TOP-K Youtube Videos / Leaderboard

Functional
- Query top-k videos (max 1000)
- 1 hour, 1 day, 1 month, all time

---- Out of scope----
- no arbitrary time periods
no arbitrary starting point

non-Functional
- <1 min staleness
- no approximation
- massive num of views
- massive num of videos
- Result in 10-100 ms
- Economical

Scale Estimation
- Throughput
  if we have a 10B views per day
  (10B views/day) / (100k sec/day)
  = 1M TPS

- Storage
  * videos/day
    = ((60mins content/sec) / (6 min content/video)) * 100K sec/day
    = 1M videos/day
  * Total videos = 1M * 365days * 10yrs
    = 3.6B or ~4B

  * With this calculation the size (Naive) of table ID and Count would be (count is views for each videoID),

    Storage = 4B * (8 bytes/ID + 8 bytes/count)
    = 64GB

    Ok, probably something we can keep in memory, especially if we use a number of hosts.

- Core Entities
  1. View
  2. Video
  3. TimeWindow

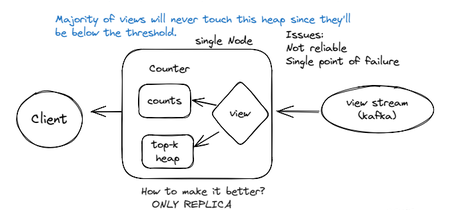
- API
  GET view/top?window={WINDOW}&k={K}

High Level (Basic Solution - All time)

simple solution for all-time top K videos which we'll build on a gigantic single host,

Counts: table of video IDs and counts. updated with count of every video.
Top-k heap: heap of the top K videos which we can update with each increment.

Majority of views will never touch this heap since they'll be below the threshold.


single Node
Counter
counts
view
top-k heap
Client
view stream (kafka)

Issues:
Not reliable
Single point of failure

How to make it better?
ONLY REPLICA
- multiple replica of Counter node
  PROS.
  * helps scale reads from our client
  * no single point of failure

CONS
* increase load on View stream
* multiply the hardware need
* with replica only, in failure we have to remove the failed instance and, new instance "catch-up" can take longer, requires stream retention.

REPLICAS + SNAPSHOTS
- All replica
- Periodically snapshot our memory
PROS
* allows us to scale out reads from our Client.
* quick recover from failure
* we'll load the most recent snapshot and re-read from our stream until we "catch up".
* we're in a much more fault-tolerant state

CONS/Challenges
* we still need retention on stream till catch-up but we can drop after that.
* snapshotting increases system overhead as they'll be multi-GB (dragging down throughput).
* We'll need 1 second to recover for every second that has passed since the last snapshot.
* need mechanism to make sure snapshots are internally consistent.

SCALE WRITE THROUGHPUT
Sharding/Partitioning (Elastic Partitioning)
* change number and size of partitions dynamically (crucial for handling varying workloads)
* When we need to add more capacity, new shards are spun up which read from different snapshots.

Challenges
* Since our partitions are no longer fixed, we need registry like "Zookeeper" so that each shards knows its assignment and the Top-K service knows which shards to query.

* Num of SHARDS becoming large requiring a large, scatter-gather operation.

Deep Dive/Improved and Final Design

HANDLING TIME WINDOWS
- We'll maintain separate Counts for each time window "Hour" and "Day" as well as separate "top-k heaps". We can utilize "Day" count for "Month".

- To handle time windows we need to age out views that happened outside that window. One solution could be is to use "2-pointers/sliding window".

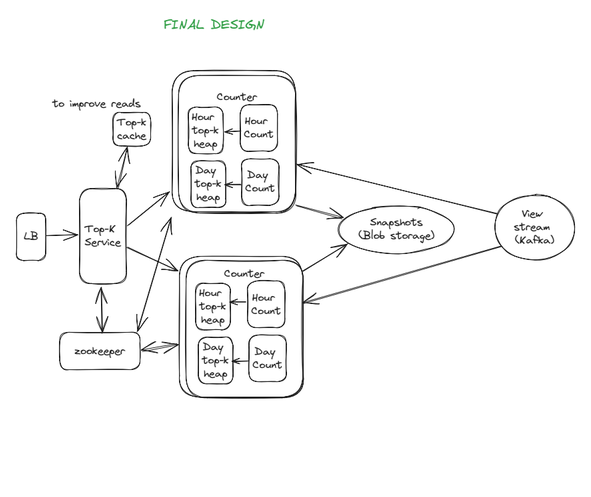
  Using 2-Pointers:
  We can have pointers pulling messages at different offsets from Kafka.
  - "Rising edge", when a new view comes in the system adds this view to the current count for all active time windows (hour, day).

  - "Falling edges" where we decrement the count.
    For each "Falling Edge",
    * start reading from the Kafka at an offset corresponding to the window (e.g., 1 hour ago, 1 month ago).
    * We pause consuming when we encounter an entry that is newer than the current time minus the duration of the time window.
    * This ensures that any views processed by the "falling edge" pointers are decremented from the respective counts, as they no longer fall within the current time window.

CHALLENGES:
- at least a 1 month (+ time since our last snapshot) retention.
- Increased read load
- Because, videos may fall out of the top 1,000, we need to make sure our heaps are storing significantly more than 1,000 (say, 2,000) accommodating new entries smoothly and expiration of old entries effectively.

HANDLING LARGE READ REQUEST
- We can add a cache setup TTL on the cache so results are never more stale than requirement. When a request comes in, we can either serve it from cache or we query all Counters for the given heap of the request and then store the merged values back in the case.

FINAL DESIGN


to improve reads
Top-k cache
Counter
Hour top-k heap
Hour Count
Day top-k heap
Day Count
LB
Top-K Service
zookeeper
Snapshots (Blob storage)
View stream (Kafka)

Design a Web Crawler

Functional Req:
- Crawl the web starting from seed URLs.
- Extract text data and store
----below the line---
- actual processing data
- Handling non-text data (e.g., images, videos, etc.).
- Handling of dynamic content (e.g., JavaScript-rendered pages).
- Handling of authentication (e.g., login-required pages)

Scale:
- 10B web pages
- Page size: 2MB each
- 5 days to scrape
- unlimited resource (within reason running cost)

Non-Functional Req:
- Fault tolerance
- Politeness
- Efficiency to crawl the web in under 5 days.
- Scalability to handle 10B pages.

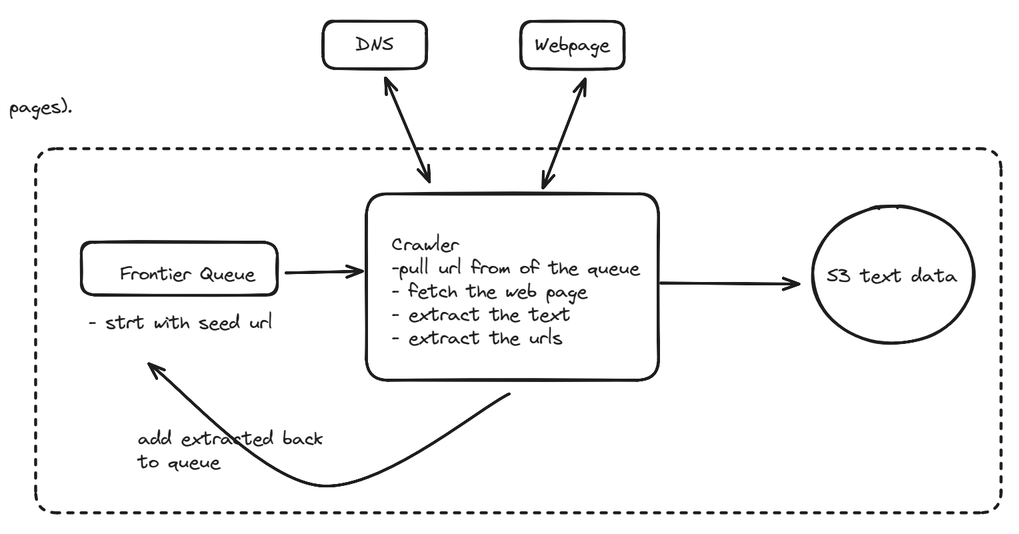
---Below the line (out of scope)---
- Security
- operating cost
- adhere to legal requirements

Core Entities:
- Text data
- URL Metadata
- Domain metadata

API or Interface:
- input: set of seed urls
- output: website data

Data Flow:
1. take the seed URL from fontier and get the IP from DNS
2. Fetch HTML
3. Extract text from HTML
4. Store text in DB
5. Extract the urls in the text and add to frontier
6. Repeat steps 1 to 5

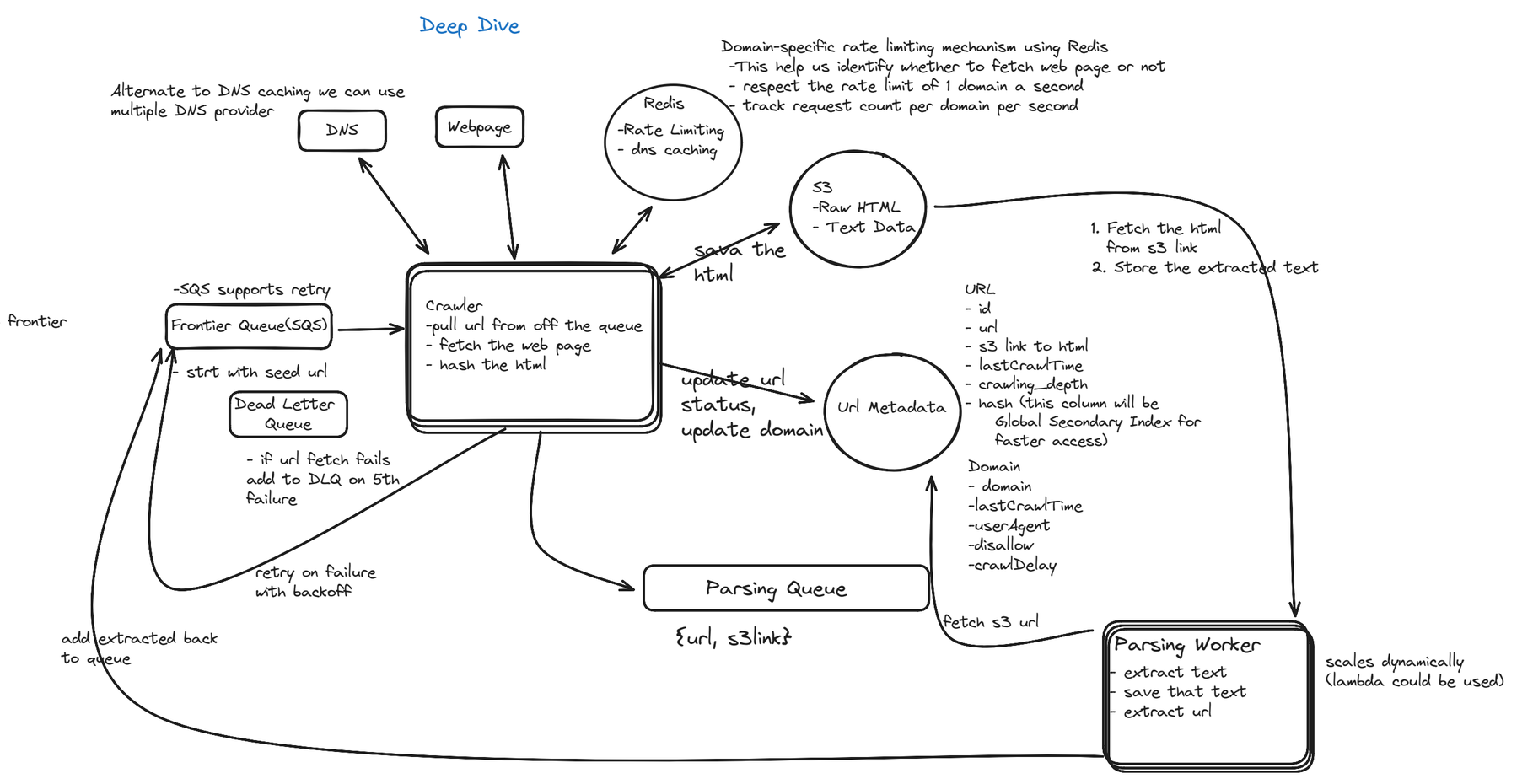
High Level Design


DNS
Webpage
Frontier Queue
- strt with seed url
Crawler
-pull url from of the queue
- fetch the web page
- extract the text
- extract the urls
S3 text data
add extracted back to queue

Deep Dive

Alternate to DNS caching we can use multiple DNS provider

Domain-specific rate limiting mechanism using Redis
-This help us identify whether to fetch web page or not
- respect the rate limit of 1 domain a second
- track request count per domain per second


DNS
Webpage
Redis
-Rate Limiting
- dns caching
S3
-Raw HTML
- Text Data
save the html
-SQS supports retry
Frontier Queue(SQS)
- strt with seed url
Crawler
-pull url from off the queue
- fetch the web page
- hash the html
Dead Letter Queue
- if url fetch fails add to DLQ on 5th failure
update url status, update domain
Url Metadata
1. Fetch the html from s3 link
2. Store the extracted text
retry on failure with backoff
Parsing Queue
{url, s3link}
fetch s3 url
add extracted back to queue
Parsing Worker
- extract text
- save that text
- extract url
scales dynamically (lambda could be used)

URL
- id
- url
- s3 link to html
- lastCrawlTime
- crawling_depth
- hash (this column will be Global Secondary Index for faster access)

Domain
- domain
-lastCrawlTime
-userAgent
-disallow
-crawlDelay

Handling URL Fetch Failure
- SQS supports retries with configurable exponential backoff.
- After a certain number of failures, the message is moved to a dead-letter queue (DLQ). At this stage, the message is considered unprocessable (say after 5 retries).

Scale to 10B pages
In the AWS ecosystem, a network optimized instance can handle about 400 Gbps. This means that a single instance, can handle about:

(400 Gbps / 8 bits/byte) / 2MB/page = 25,000 pages/second.

That's a ton. But there will be limitations on the number of requests per second dictated by factors like server response latency, DNS resolution, rate limiting, politeness, retries, etc.

But let's say we utilize 30% of the available bandwidth. This would give us:
25,000 pages/second * 30% = 7,500 pages/second i.e. around 10k or 10^4

To estimate the total number of high powered machines we need, we can divide the total number of pages by the number of pages we can crawl per second. For single machine this gives us,

10,000,000,000 pages / 7,500 pages/second = 1,333,333 seconds = 15.4 days.
OR
10^9/10^4 = 10^6 seconds
10^6/10^5(seconds in a day) = 10 days on single machine,
i.e. if we use 2 machines it will take 5 days

Video (Live) & Image Streaming Media Platform

Functional Req:
- Upload video and images from IoT devices.
- Support MP4 video format
- Smooth video streaming.
- Ability to change video quality (Adaptive (bitrate) streaming).
- Accessible only to authenticated and authorized users.
- Handle large video files, potentially through asynchronous processing.
- Support multiple video streaming protocols, starting with RTSP.

Non-Functional Req:
- 100 million DAU.
- Low latency for upload and retrieval.
- Availability > Consistency
- Resumable Upload
- Implement encryption for data in transit (TLS).
- Authentication and authorization mechanisms (OAuth2, JWT).

Back-of-the-Envelope Calculation (100M DAU):
- Assuming an average video stream of 2Mbps and an average image size of 1MB, the total daily data ingestion would be:

  * Video: 100M users * 2Mbps * 60 secs * 60 mins * 24 hrs
    = 172.8 PB (around 200 PB)
  * Images: 100M users * 1MB * 10 images per day = 1 PB

  Total daily data ingestion: 173.8 PB (around 200 PB)

- Considering a 3x replication factor for data redundancy and availability, the total storage requirement would be approximately 521.4 PB (around 600 PB).

- Assuming an average video watch time of 10 minutes per day, the total daily video streaming bandwidth would be:
  100M users * 2Mbps * 10 minutes = 20 PB

QPS ?

API Endpoints (This may not be applicable for IoT media streaming):
1. Video Upload
   POST /api/media/video
   Fields: device_id, stream_url, format

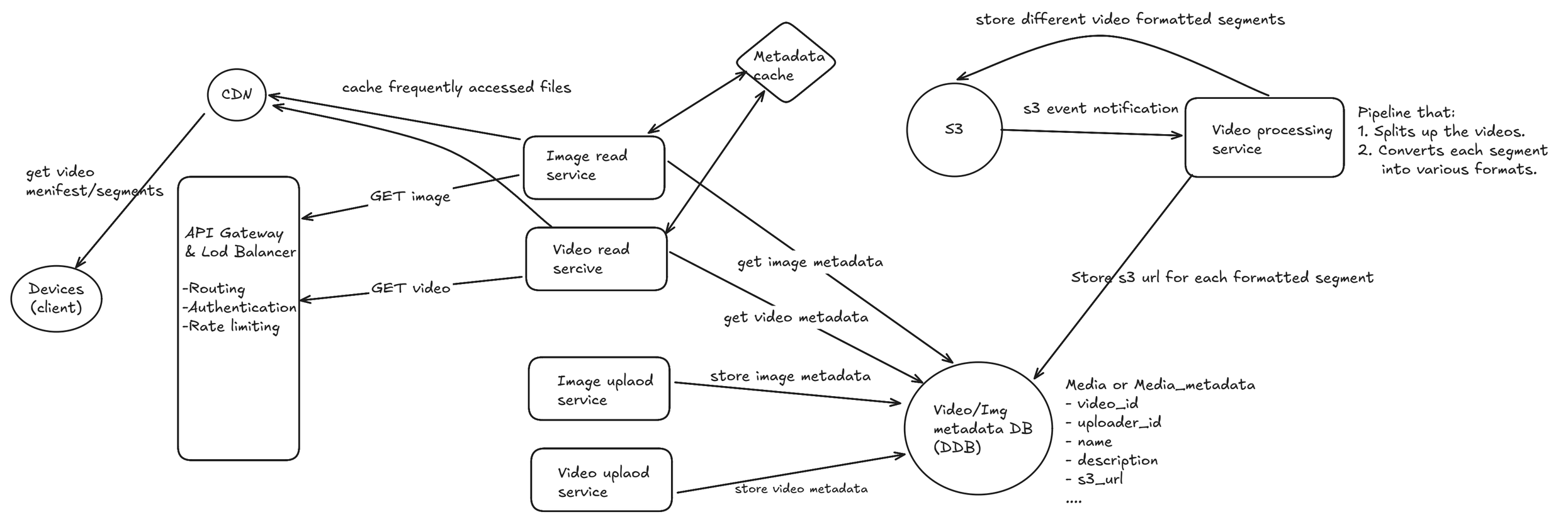
2. Image Upload
   POST /media/image
   Fields:
   device_id,
   image_data: Binary (the image file)
   format: String (default to JPEG/PNG)

3. Media Retrieval
   GET /media/{device_id}
   Response:
   media_list: (list of media files with metadata)

4. Video Stream Start
   POST /api/media/start_stream
   Request Fields:
   device_id, stream_url:

DB Design
- Users (user_id, ...)
- Devices (device_id, user_id, ...)
- Media
  media_id (PK)
  device_id (FK)
  user_id (FK)
  media_type: e.g., image, video
  media_url: URL where the media file is stored
  thumbnail_url: thumbnail image URL (for videos)
  size: Size of the media file in bytes
  format: e.g., MP4, JPEG
  created_at

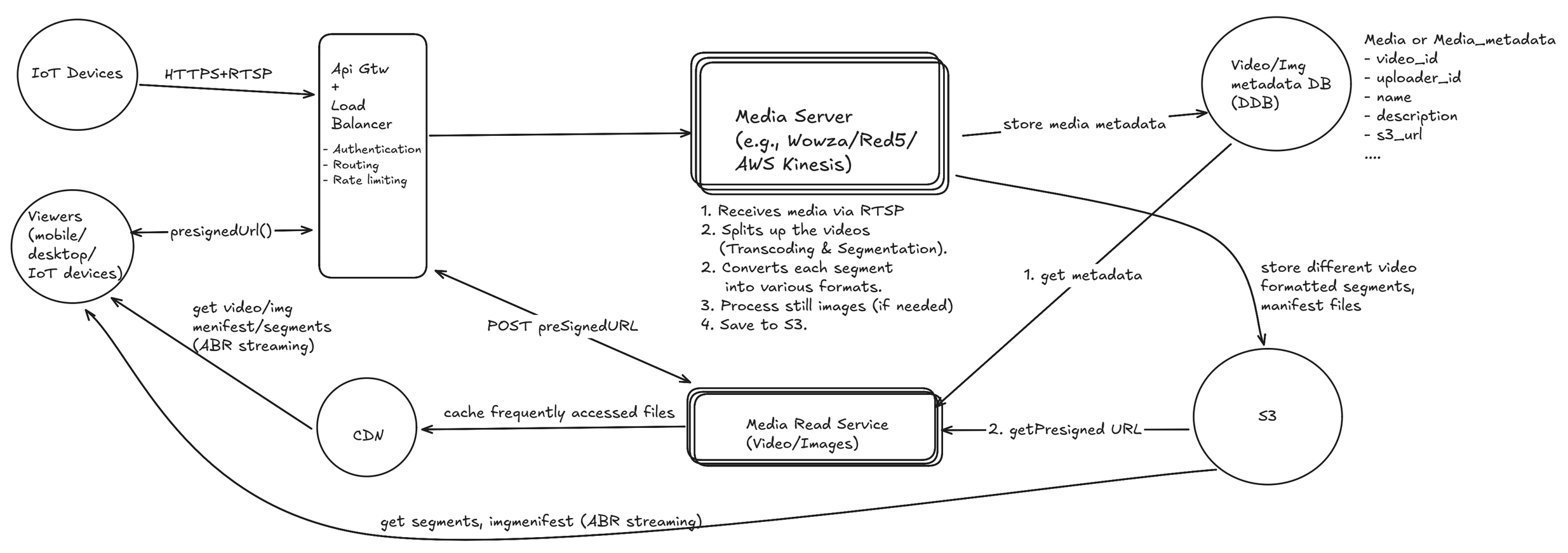
High Level Diagram


CDN
cache frequently accessed files
Metadata cache
store different video formatted segments
S3
s3 event notification
Video processing service
get video menifest/segments
API Gateway & Lod Balancer
-Routing
-Authentication
-Rate limiting
GET image
GET video
Image read service
Video read sercive
get image metadata
get video metadata
Store s3 url for each formatted segment
Devices (client)
Image uplaod service
Video uplaod service
store image metadata
store video metadata
Video/Img metadata DB (DDB)

Pipeline that:
1. Splits up the videos.
2. Converts each segment into various formats.

Media or Media_metadata
- video_id
- uploader_id
- name
- description
- s3_url
....

High Level Diagram - IoT Devices Media Streaming

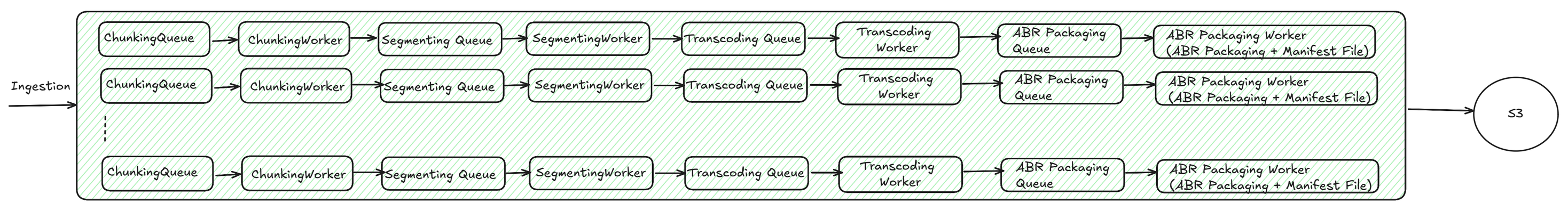

IoT Devices
HTTPS+RTSP
Api Gtw + Load Balancer
- Authentication
- Routing
- Rate limiting
Media Server (e.g., Wowza/Red5/ AWS Kinesis)
store media metadata
Video/Img metadata DB (DDB)
Viewers (mobile/ desktop/ IoT devices)
presignedUrl()
get video/img menifest/segments (ABR streaming)
POST preSignedURL
1. get metadata
store different video formatted segments, manifest files
CDN
cache frequently accessed files
Media Read Service (Video/Images)
2. getPresigned URL
S3
get segments, imgmenifest (ABR streaming)

1. Receives media via RTSP
2. Splits up the videos (Transcoding & Segmentation).
2. Converts each segment into various formats.
3. Process still images (if needed)
4. Save to S3.

Media or Media_metadata
- video_id
- uploader_id
- name
- description
- s3_url
....

Media Server

client(Ingestion) -> Chunking -> Segmentation -> Transcoding -> ABR Packaging+ManifestFile Creation -> Object Storage -> CDN


Ingestion
ChunkingQueue
ChunkingWorker
Segmenting Queue
SegmentingWorker
Transcoding Queue
Transcoding Worker
ABR Packaging Queue
ABR Packaging Worker (ABR Packaging + Manifest File)
S3

We can use Amazon SQS here. SQS offers a feature where it deletes the message onces consumed by the consumer. This will help save storage space.
Above diagram can be further updated to have a common SQS Queue (partioned) instead of dedicated queue for each stage and respective worker will read from that queue/topic.

Workflow with Multiple Kafka Topics

Ingestion/Chunking Queue:
- Incoming video streams are published to this topic by the media server.

Chunking Stage:
- Consumes from the chunking queue and Divide video streams into smaller chunks.

Segmenting Stage:
- SegmentingWorker: Consumes from the SegmentingQueue.
- Segmenting Output: Process chunks into precise segments.

Transcoding Stage:
- TranscodingWorker: Consumes from the Transcoding Queue.
- Transcoding Output: Convert segments into multiple resolutions and bitrates.

ABR Packaging Stage:
- ABR Packaging Worker: Consumes from the ABR Packaging queue.
- ABR Packaging Output:
  * Package segments for adaptive bitrate streaming.
  * Create Manifest files and store in S3.
  * Store metadata about video segments to DDB (this could be direct REST call, i.e. another service that takes the metadata and store in DDb)

IoT Devices Stream Data (RTSP)
↓
Load Balancer
↓
Media Ingestion Nodes
↓
Queue for Chunking (Kafka, RabbitMQ, etc.)
↓
Chunking
↓
Queue for Segmentation (Kafka, RabbitMQ, etc.)
↓
Segmentation
↓
Queue for Transcoding (Kafka, RabbitMQ, etc.)
↓
Transcoding
↓
Queue for ABR Packaging (Kafka, RabbitMQ, etc.)
↓
ABR Packaging
(Manifest File Creation)
↓
Queue for Storage (Kafka, RabbitMQ, etc.)
↓
Object Storage
↓
Metadata Storage (REST)
↓
Client Requests Playback (REST)
↓
Content Delivery Network (CDN)
↓
Adaptive Bit Rate Streaming

IoT Devices Stream Data (RTSP)
↓
Load Balancer
↓
Media Ingestion Nodes (Media server)
↓
Chunking
↓
Segmentation
↓
Transcoding
↓
ABR Packaging
↓
Object Storage
↓
Metadata Storage (REST)
↓
Client Requests Playback (REST)
↓
Content Delivery Network (CDN)
↓
Adaptive Bit Rate Streaming

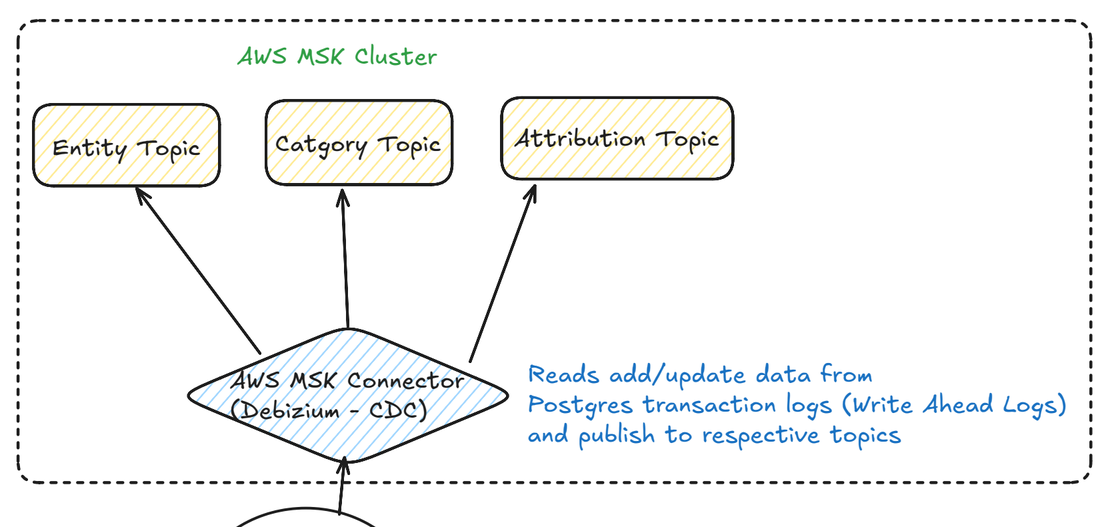
Some Usful Links:
https://aws.amazon.com/blogs/iot/build-a-cloud-gateway-to-ingest-rtsp-video-to-amazon-kinesis-video-streams/#:~:text=In%20this%20blog%20post%20you,camera%20devices%20to%20the%20cloud
https://getstream.io/blog/streaming-protocols/

Ascriber

This was not just a CRUD service connected to Kafka. I designed the authoritative transactional write path, reliable event-publication mechanism, asynchronous event distribution, and analytical data platform as one end-to-end architecture.

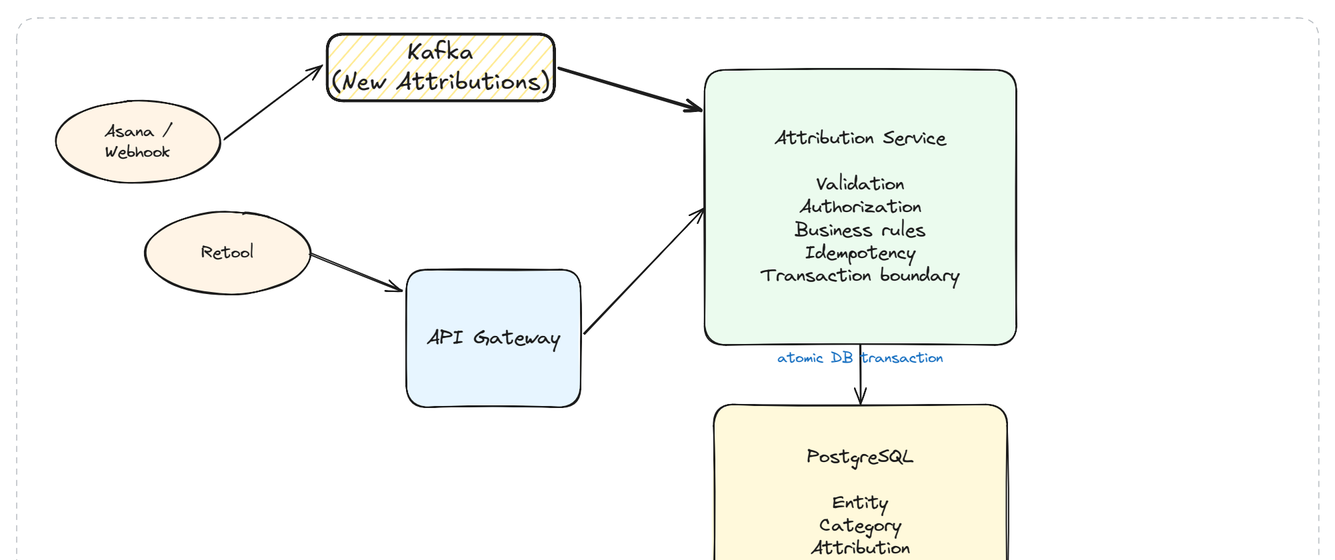
The main design challenges were consistency between the database and Kafka, replay/idempotency, schema evolution, failure recovery, scaling consumers independently, and keeping operational and analytical workloads decoupled.

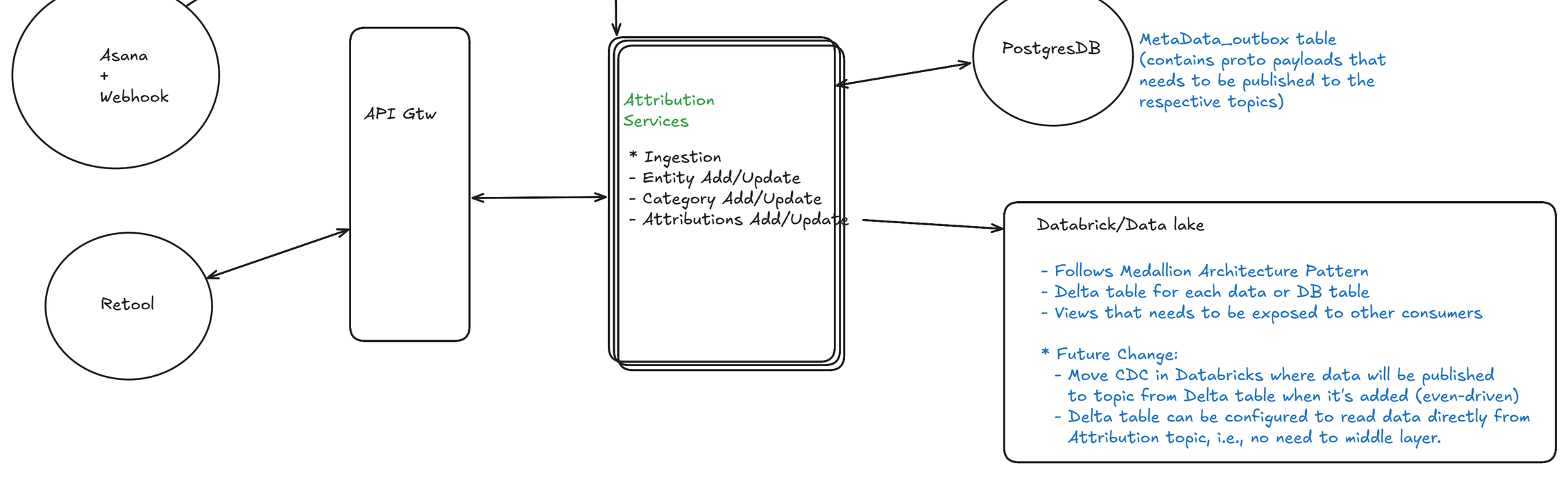
Kafka (New Attributions)


AWS MSK Cluster
Entity Topic
Catgory Topic
Attribution Topic
AWS MSK Connector (Debizium - CDC)

Reads add/update data from Postgres transaction logs (Write Ahead Logs) and publish to respective topics

TRANSACTIONAL WRITE PATH


Kafka (New Attributions)
Asana / Webhook
Retool
API Gateway
Attribution Service
Validation
Authorization
Business rules
Idempotency
Transaction boundary
atomic DB transaction
PostgreSQL
Entity
Category
Attribution

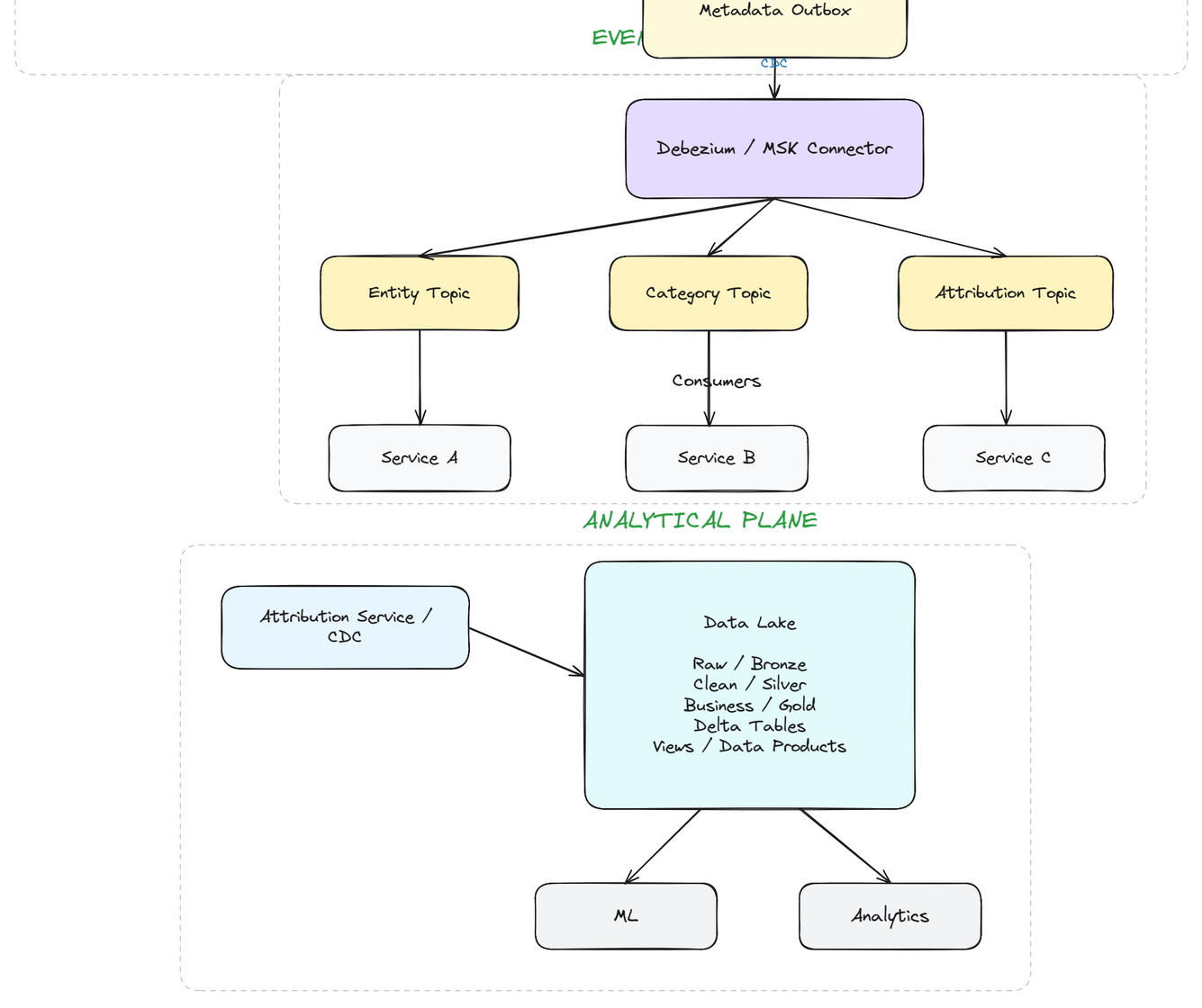

Asana
+
Webhook
Retool
API Gtw
Attribution
Services
* Ingestion
- Entity Add/Update
- Category Add/Update
- Attributions Add/Update
PostgresDB

MetaData_outbox table
(contains proto payloads that
needs to be published to the
respective topics)

Databrick/Data lake

- Follows Medallion Architecture Pattern
- Delta table for each data or DB table
- Views that needs to be exposed to other consumers

* Future Change:
  - Move CDC in Databricks where data will be published
    to topic from Delta table when it's added (even-driven)
  - Delta table can be configured to read data directly from
    Attribution topic, i.e., no need to middle layer.


EVE
Metadata Outbox
CDC
Debezium / MSK Connector
Entity Topic
Category Topic
Attribution Topic
Consumers
Service A
Service B
Service C
ANALYTICAL PLANE
Attribution Service /
CDC
Data Lake
Raw / Bronze
Clean / Silver
Business / Gold
Delta Tables
Views / Data Products
ML
Analytics

Food Delivery

Functional Requirements

- User registration and authentication
- Restaurant onboarding and menu management
- Order placement and tracking
- Payment processing
- Delivery driver assignment and tracking
- Rating and review system for restaurants and drivers
- Search and filter functionality for restaurants and menu items
- Notifications for order status updates
- Admin Panel: Restaurant manage menus, availability, and delivery settings.

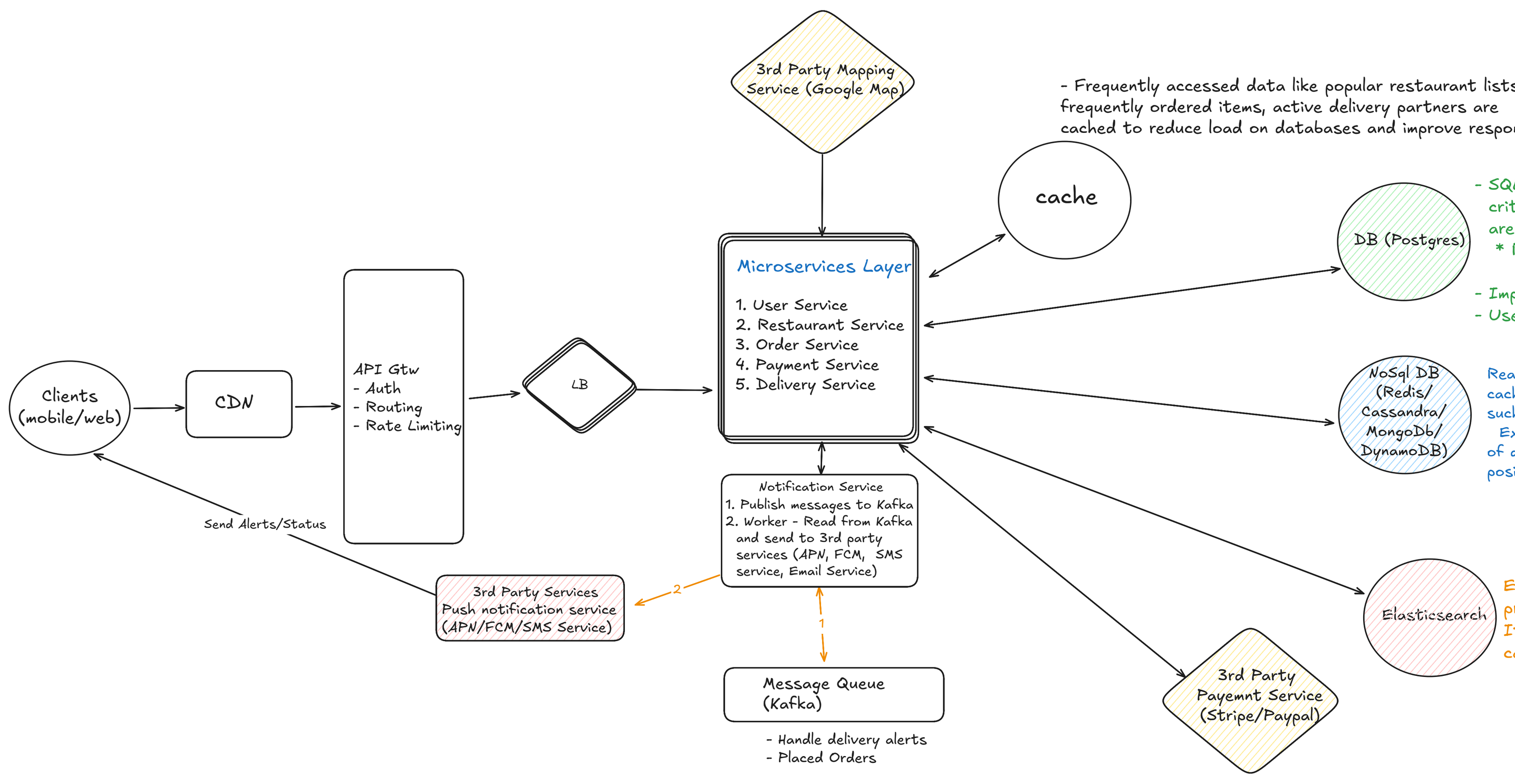
Non-Functional Requirements

- High availability (99.99% uptime)
- Low latency (< 100ms response time for most operations)
- Scalability to handle millions of users and orders
- Data consistency and integrity
- Security (data encryption, authentication, authorization)
- Fault tolerance and disaster recovery
- Compliance with data protection regulations (e.g., GDPR)

Back-of-the-Envelope

Assuming 1 Million daily active users:
- Daily orders: 500,000
- Peak orders per hour: 50,000
- Average order size: 5 KB
- Daily data generation: 500,000 * 5 KB = 2.5 GB
- Storage needed per year: 2.5 GB * 365 = 912.5 GB

Bandwidth estimation:
- Peak requests per second: 50,000 / 3600 ≈ 14 RPS
- Assuming 1 KB per request: 14 KB/s or 0.112 Mbps


3rd Party Mapping
Service (Google Map)
cache
Microservices Layer
1. User Service
2. Restaurant Service
3. Order Service
4. Payment Service
5. Delivery Service
DB (Postgres)
NoSql DB
(Redis/
Cassandra/
MongoDb/
DynamoDB)
Clients
(mobile/web)
CDN
API Gtw
- Auth
- Routing
- Rate Limiting
LB
Elasticsearch
Notification Service
1. Publish messages to Kafka
2. Worker - Read from Kafka
and send to 3rd party
services (APN, FCM, SMS
service, Email Service)
Send Alerts/Status
3rd Party Services
Push notification service
(APN/FCM/SMS Service)
2
1
3rd Party
Payemnt Service
(Stripe/Paypal)

- Frequently accessed data like popular restaurant lists,
frequently ordered items, active delivery partners are
cached to reduce load on databases and improve response time.

- SQL databases ensure ACID transactions, ensuring that critical processes like order creation and payment processing are consistent and reliable.
  * Refer DB table details for db table schema.

- Implements sharding for horizontal scaling
- Uses read replicas to improve read performance

Real Time Tracking DB: Used for real-time data like delivery partner tracking, caching frequent queries, and storing unstructured data such as session information.
  Example: A NoSQL database stores the real-time location of delivery partners, allowing the app to show current partner positions on a map.

Elasticsearch plays a crucial role in the food delivery system design, primarily serving as a powerful search and analytics engine.
Its purpose is to provide fast, scalable, and efficient search capabilities for restaurants, menu items, and related information.

Message Queue
(Kafka)

- Handle delivery alerts
- Placed Orders

* Order Service places a message in the queue to notify the delivery partner of a new order.

* Delivery Service consumes the message, matches it to the nearest delivery partner, and sends an alert

API Endpoints

* User
  POST /users/register: Register a new user (customer, restaurant, delivery partner).
  POST /users/login: Log in a user.
  GET /users/profile: Get user profile data.
  PUT /users/profile: Update user profile data.

* Restaurant
  GET /restaurants: List restaurants based on location and filters.
  GET /restaurants/{id}/menu: Get the menu for a specific restaurant.
  POST /restaurants/{id}/menu: Update restaurant menu.

* Order
  POST /orders: Place an order.
  GET /orders/{id}: Get order status.
  PUT /orders/{id}/cancel: Cancel an order.

* Delivery
  GET /delivery/partners: Get available delivery partners.
  PUT /delivery/{id}/status: Update delivery status (picked up, in transit, delivered).

* Payment
  POST /payments: Process payment for an order.

* Rating & Reviews
  POST /reviews: Submit a review for a restaurant or delivery partner.
  GET /reviews/{id}: Fetch reviews for a specific entity.

DB Tables

1. Users
   user_id, name, email, password_hash, phone, address
2. Restaurants
   restaurant_id, name, address, cuisine_type, rating
3. Menu_Items
   item_id, restaurant_id, name, description, price, category
4. Orders
   order_id, user_id, restaurant_id, driver_id, status, total_amount, created_at
5. Order_Items
   order_item_id, order_id, item_id, quantity, price
6. Drivers
   driver_id, name, email, phone, vehicle_type, current_location
7. Ratings
   rating_id, user_id, restaurant_id, driver_id, rating, comment, created_at

MISSING REQUIREMENTS

How do we prevent multiple ride requests from being sent to the same driver simultaneously?
Refer here: https://www.hellointerview.com/learn/system-design/problem-breakdowns/uber

Microservices Responsibilities:
- User Service:
  Handles user information (login, profile, etc.).
- Restaurant Service:
  Fetches restaurant menus, availability, and reviews.
- Order Service:
  Manages the lifecycle of orders (placing, updating, canceling).
- Delivery Service:
  Manages delivery partner assignments, real-time status updates, and order tracking.
  1. Queries Real-Time Tracking DB for nearby delivery partner.
  2. Matches available partner based on location and status.
  3. Updates assignment in SQL Orders DB.
  4. Sends push notification via Notification Service to driver.
  5. Tracks real-time updates from driver using Real-Time DB.

- Payment Service:
  Handles transactions securely, working with external payment gateways (e.g., Stripe, PayPal).
- Notification Service:
  Sends real-time notifications (e.g., order confirmation, delivery status updates).

Processing and Sending Alerts: Real-Time Components
* Real-Time Tracking: Delivery partners' locations are tracked using GPS data, and the data is pushed to a NoSQL database for quick access.

* Notification Layer: Push Notifications are sent via services like Firebase Cloud Messaging (FCM) to inform customers and delivery partners of updates (e.g., "Your order is on the way").

* SMS or In-App Alerts: SMS for delivery partners or customers, and in-app alerts for real-time updates.

Caching Layer
Redis/Memcached: Frequently accessed data like popular restaurant lists, frequently ordered items, or active delivery partners are cached to reduce load on databases and improve response time.

Elasticsearch plays a crucial role in the food delivery system design, primarily serving as a powerful search and analytics engine.
Its purpose is to provide fast, scalable, and efficient search capabilities for restaurants, menu items, and related information.

Purpose of Elasticsearch in the Food Delivery System:
1. Fast and Efficient Searching: Elasticsearch enables quick and relevant search results for users looking for restaurants, cuisines, or specific dishes.

2. Real-time Updates: It allows for near real-time updates of restaurant information, menus, and prices, ensuring customers always see the most current data.

3. Complex Querying: Elasticsearch supports complex queries, including geospatial searches, which are crucial for finding nearby restaurants or determining delivery areas.

4. Scalability: It can handle large volumes of data and high concurrent searches, which is essential for a food delivery platform with millions of users and restaurants.

5. Faceted Search: Elasticsearch facilitates faceted search, allowing users to filter results by cuisine type, price range, ratings, etc.

6. Fuzzy Matching: It can handle typos and provide relevant results even when search terms are not exact matches.

Distributed message queue system like Kafka, which allows us to commit the offset of the message in the queue only after we have successfully found a match. This way, if the Ride Matching Service goes down, the match request would still be in the queue, and a new instance of the service would pick it up. This approach ensures that no ride requests are lost due to service failures and provides fault tolerance to our system.

Real-Time Tracking DB (NoSQL):
The Real-Time Tracking DB stores dynamic, unstructured data such as the GPS coordinates of delivery drivers. Since driver locations change frequently, a NoSQL database is preferred over traditional relational databases because it can handle high-frequency updates, offers flexibility with unstructured data, and is highly scalable for large datasets.

Popular NoSQL Databases in the Market:

* Redis: An in-memory key-value store often used as a cache, but it can also manage real-time tracking if combined with geospatial extensions like RedisGeo, which supports storing and querying location-based data.

* MongoDB: A widely-used document-based NoSQL database, offering excellent support for geospatial queries, which are essential for tracking locations.

* Cassandra: A highly scalable NoSQL database, optimized for write-heavy workloads and capable of handling massive amounts of data. Ideal for real-time tracking in large systems like Uber Eats.

* Firebase Realtime Database: A NoSQL cloud database offered by Google, designed for real-time data syncing, commonly used for tracking locations in mobile applications.

* DynamoDB (AWS): [Expensive] A fully managed NoSQL database that offers high performance and scales automatically. Its integration with AWS services makes it a great choice for large-scale real-time tracking systems.

*** Design Diagram: Delivery Service Real-Time Tracking Flow ***

```
+------------------------------------------------------------------------+
|                  Driver's Mobile App (GPS)                              |
|------------------------------------------------------------------------|
| - Sends driver's GPS coordinates to backend periodically     |
| - Uses WebSocket / HTTP API to send real-time data       |
+------------------------------------------------------------------------+
                                    |
                                    v
+---------------------------------------------------------------------------+
|                        WebSocket / HTTP API                              |
|---------------------------------------------------------------------------|
| - Facilitates real-time data transmission                               |
| - Receives GPS coordinates and forwards them to backend    |
+---------------------------------------------------------------------------+
                                    |
                                    v
+---------------------------------------------------------------------+
|                    Real-Time Tracking DB (NoSQL)             |
|---------------------------------------------------------------------|
| - Stores driver's current GPS location                          |
| - Tracks real-time driver status (e.g., Available, On Trip)  |
| - Supports geospatial queries (e.g., nearby drivers)          |
+---------------------------------------------------------------------+
                                    |
                                    v
+---------------------------------------------------------------------+
|                    Delivery Service                                   |
|---------------------------------------------------------------------|
| - Matches available drivers based on proximity                 |
| - Tracks real-time driver location using Real-Time DB       |
| - Updates Order Service on delivery progress                  |
+---------------------------------------------------------------------+
                                    |
                                    v
+---------------------------------------------------------------------+
|                    Map Service (Google Maps API)               |
|---------------------------------------------------------------------|
| - Displays real-time driver location on a map                  |
| - Calculates ETA and route optimization                         |
+---------------------------------------------------------------------+
                                    |
                                    v
+-----------------------------------------------------------------------+
|                    Order Service & Notification Service           |
|-----------------------------------------------------------------------|
| - Updates order status (Picked Up, Out for Delivery)       |
| - Sends notifications to customers with delivery updates  |
+-----------------------------------------------------------------------+
```

Stock Price Change Notifications/Alerts

** Functional Requirements **
- Users can subscribe to X no of stocks and set rules based on stock prices.

- Rule Types:
  Users can define rules based on stock metrics such as:
  - Opening Price: Notify if it goes above/below a threshold.
  - Current Price: Notify if it crosses a certain value or changes by a %.
  - Moving Average: Notify if stock price deviates from the moving average.

- Real-Time Stock Data Processing:
  Continuously scan stock price changes from an external service/API.

- Rule Evaluation:
  Check more than 1 million user-defined rules against stock price changes.

- Notify user if any rule is satisfied. Notifications must be sent within a minute.

** Non-Functional Requirements **
- Scalability: System should handle millions of rules and thousands of stock updates per second.

- Low Latency: Notifications should be sent to users within one minute.

- High Availability: System must remain available during high traffic or stock market volatility.

- Fault Tolerance: The system should be resilient to failures in external stock APIs and notification services.

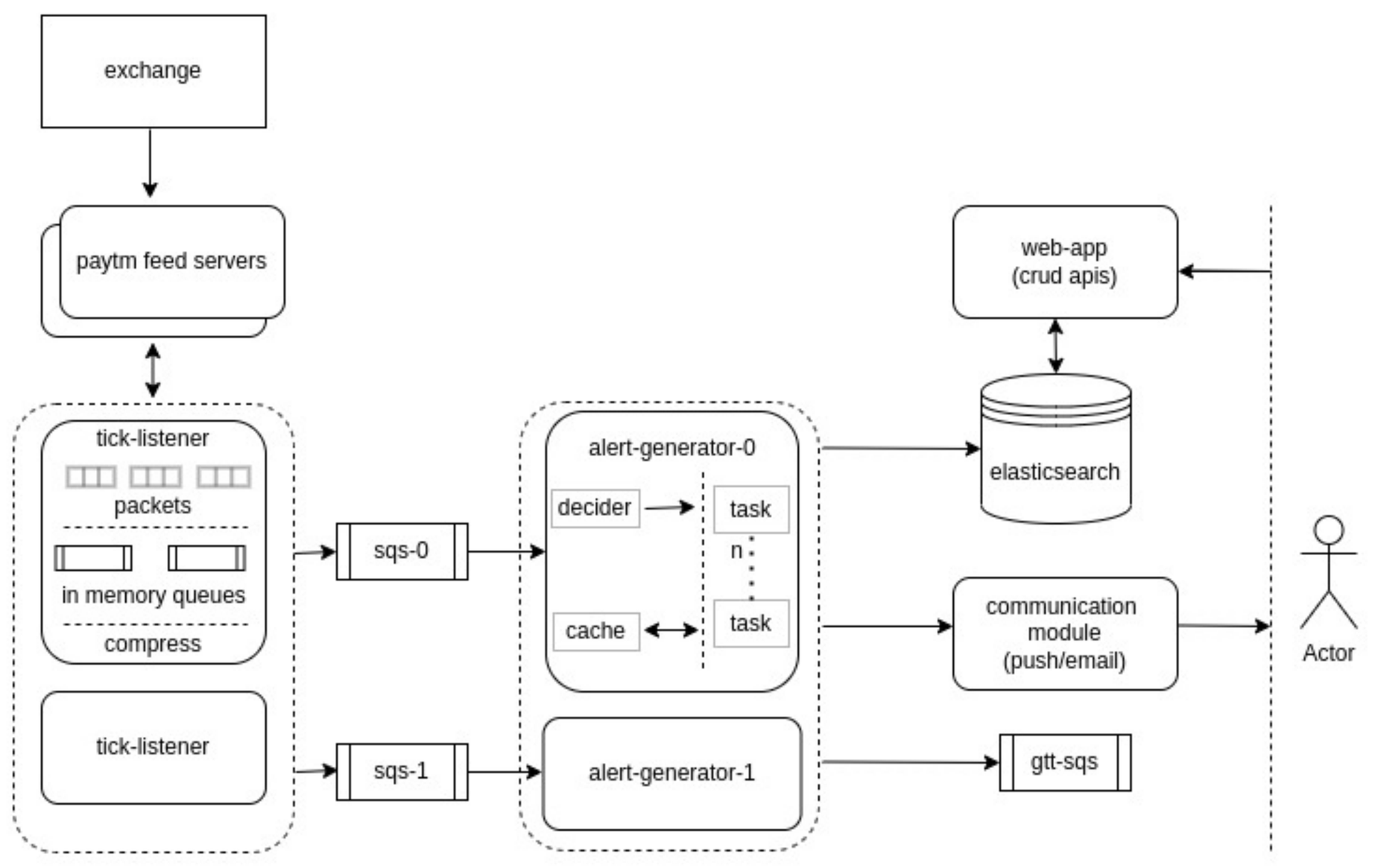
- Data Consistency: Rules and stock data must remain consistent even in a distributed setup.

- Security: Users' data and subscriptions must be secured, particularly notifications and stock preferences.

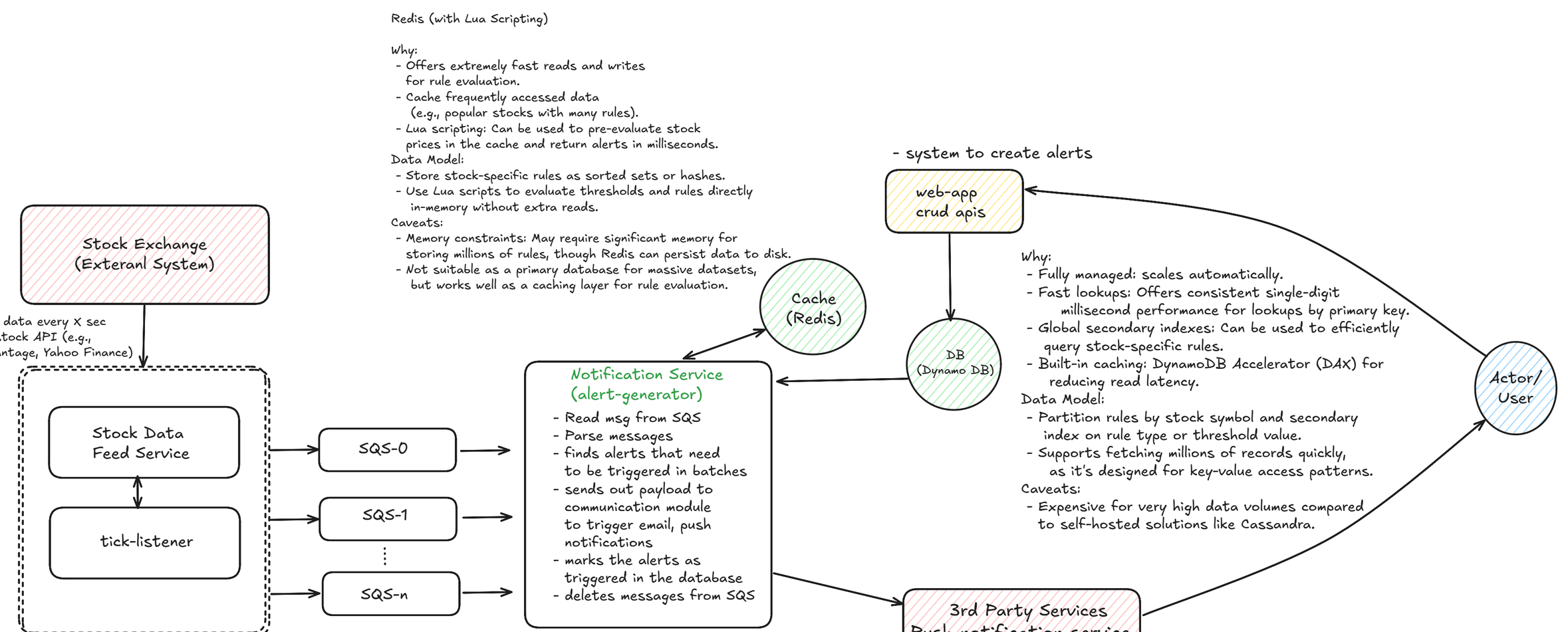
** Back-of-the-Envelope Estimation **
- Users: Assume 10 million users, each tracking 10 stocks.

- Rules: 1 million users may define 3 rules per stock, l
  eading to 30 million rules to scan.

- Stock Data: With ~5000 stocks, receiving updates every


exchange
paytm feed servers
tick-listener
packets
in memory queues
compress
tick-listener
sqs-0
sqs-1
alert-generator-0
decider
task
n
task
cache
alert-generator-1
web-app
(crud apis)
elasticsearch
communication
module
(push/email)
gtt-sqs
Actor

Source: https://www.paytmmoney.com/blog/how-we-built-an-in-house-price-alerts-system/


Stock Exchange
(Exteranl System)
Fetches data every X sec
from a stock API (e.g.,
Alpha Vantage, Yahoo Finance)
Stock Data
Feed Service
tick-listener
SQS-0
SQS-1
SQS-n
Cache
(Redis)
DB
(Dynamo DB)
- system to create alerts
web-app
crud apis
Actor/
User
3rd Party Services

Redis (with Lua Scripting)

Why:
- Offers extremely fast reads and writes for rule evaluation.
- Cache frequently accessed data (e.g., popular stocks with many rules).
- Lua scripting: Can be used to pre-evaluate stock prices in the cache and return alerts in milliseconds.

Data Model:
- Store stock-specific rules as sorted sets or hashes.
- Use Lua scripts to evaluate thresholds and rules directly in-memory without extra reads.

Caveats:
- Memory constraints: May require significant memory for storing millions of rules, though Redis can persist data to disk.
- Not suitable as a primary database for massive datasets, but works well as a caching layer for rule evaluation.

Notification Service
(alert-generator)
- Read msg from SQS
- Parse messages
- finds alerts that need to be triggered in batches
- sends out payload to communication module to trigger email, push notifications
- marks the alerts as triggered in the database
- deletes messages from SQS

Why:
- Fully managed: scales automatically.
- Fast lookups: Offers consistent single-digit millisecond performance for lookups by primary key.
- Global secondary indexes: Can be used to efficiently query stock-specific rules.
- Built-in caching: DynamoDB Accelerator (DAX) for reducing read latency.

Data Model:
- Partition rules by stock symbol and secondary index on rule type or threshold value.
- Supports fetching millions of records quickly, as it's designed for key-value access patterns.

Caveats:
- Expensive for very high data volumes compared to self-hosted solutions like Cassandra.

sec for price changes, that's around 5000 updates per sec.

- Notification Throughput: 1% of rules trigger notifications per minute (10,000 notifications/min).

- Storage: Each rule can take 200 bytes, resulting in ~6GB of storage for all user rules.

- Latency: Each rule check and notification should occur within 1 sec.

** API Endpoints **

User Management:
- POST /users/register: Register a new user.
- POST /users/login: Authenticate the user.

Stock Subscription:
- Subscribe to stock price changes.
  POST /users/{userId}/stocks/subscribe
  Payload: {

```
            "stockSymbol": "ABC",
            "rules": [{
                        "type": "price",
                        "operator": ">",
                        "value": 100
                      }]
          }
```

- Un-Subscribe to stock price changes.
  POST /users/{userId}/stocks/unsubscribe

- Get list of subscribed stocks.
  GET /users/{userId}/stocks

Rule Management:
- POST /users/{userId}/rules: Create a rule for a stock.
- GET /users/{userId}/rules: List all rules set by the user.
- DELETE /users/{userId}/rules/{ruleId}: Delete a specific rule.

Notification Management:
- GET /users/{userId}/notifications: Retrieve user notifications.

** Database Tables **
- Users: user_id, email, name, phone_number

- Stocks: stock_id, symbol, name, last_price, timestamp

- Subscriptions: subscription_id, user_id, stock_id

- Rules:
  rule_id, subscription_id,
  rule_type (e.g., price above, price below, moving average),
  operator (e.g., >, <),
  threshold_value, status (active, triggered)
  created_at

- Notifications:
  notification_id, user_id, rule_id, message,
  timestamp, status (sent, failed)

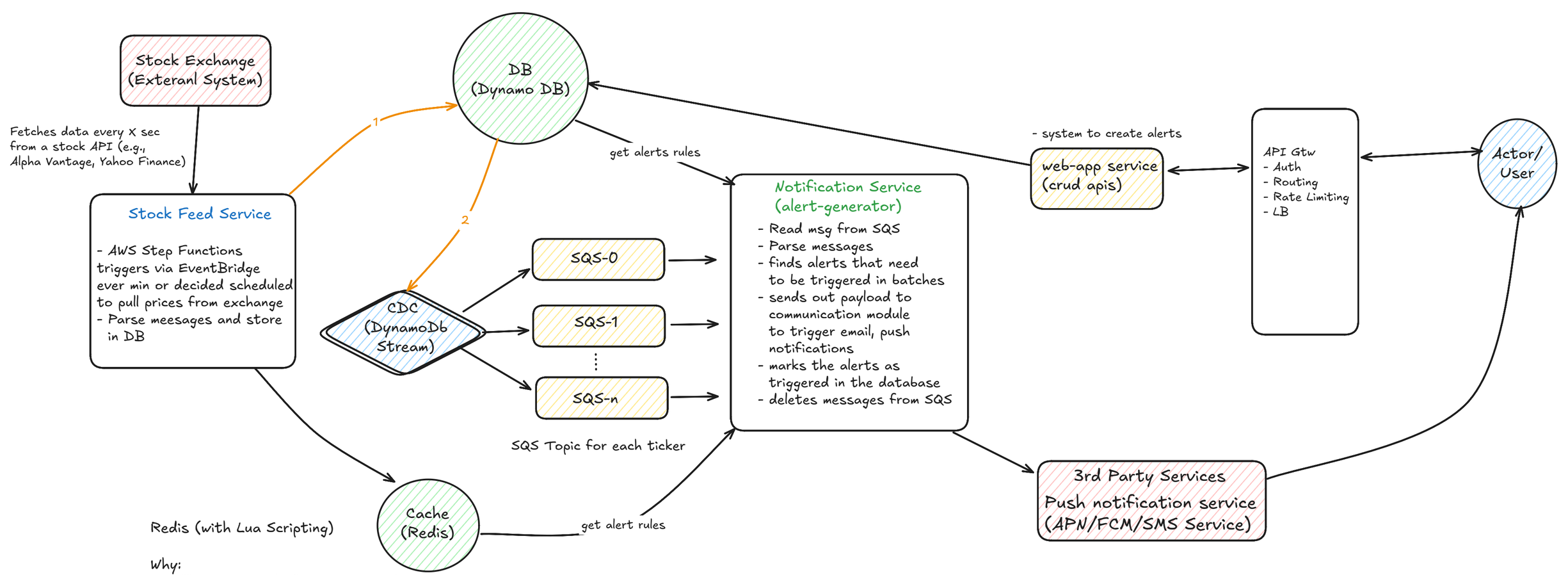
Useful URLs:
https://leetcode.com/discuss/interview-question/system-design/5745135/Design-Stock-price-Notification-System/
https://www.paytmmoney.com/blog/how-we-built-an-in-house-price-alerts-system/

- Here, AWS step function can be used to fetch stock prices from exchange, then publish the msg to SQS

SQS Topic for each ticker

(APN/FCM/SMS Service)

*** 2nd Approach (with CDC/Dynamo DB Stream) ***


Stock Exchange (Exteranl System)
Fetches data every X sec from a stock API (e.g., Alpha Vantage, Yahoo Finance)
Stock Feed Service
- AWS Step Functions triggers via EventBridge ever min or decided scheduled to pull prices from exchange
- Parse meesages and store in DB
DB (Dynamo DB)
1
2
CDC (DynamoDb Stream)
SQS-0
SQS-1
SQS-n
SQS Topic for each ticker
get alerts rules
Notification Service (alert-generator)
- Read msg from SQS
- Parse messages
- finds alerts that need to be triggered in batches
- sends out payload to communication module to trigger email, push notifications
- marks the alerts as triggered in the database
- deletes messages from SQS
- system to create alerts
web-app service (crud apis)
API Gtw
- Auth
- Routing
- Rate Limiting
- LB
Actor/ User
3rd Party Services
Push notification service (APN/FCM/SMS Service)
Cache (Redis)
get alert rules

Redis (with Lua Scripting)

Why:
- Offers extremely fast reads and writes for rule evaluation.
- Cache frequently accessed data (e.g., popular stocks with many rules).
- Lua scripting: Can be used to pre-evaluate stock prices in the cache and return alerts in milliseconds.

Data Model:
- Store stock-specific rules as sorted sets or hashes.
- Use Lua scripts to evaluate thresholds and rules directly in-memory without extra reads.

Caveats:
- Memory constraints: May require significant memory for storing millions of rules, though Redis can persist data to disk.
- Not suitable as a primary database for massive datasets, but works well as a caching layer for rule evaluation.

** Another Recommended Approach for DB **

Primary Storage: Cassandra/ScyllaDB
* Why: Cassandra's distributed nature, horizontal scalability, and low-latency reads make it an excellent choice for storing and querying millions of stock rules.
* How It Works:
  - Partition rules by stock symbol.
  - Query for rules related to a specific stock symbol and rule type in constant time (using Cassandra's partition key).
  - Efficiently scale as the number of rules grows, without worrying about pagination or scroll mechanisms.

Secondary Storage: Redis for Caching Hot Data
* Why: Redis can act as a caching layer for frequently accessed stocks and rules. For example, if a certain stock price changes frequently or has a high number of subscribers, caching those rules in Redis would allow faster rule evaluation.
* How It Works:
  - Cache stock rules in Redis for quick lookups during evaluation.
  - Use Redis for frequently accessed rules, while Cassandra serves as the primary store for all rules.

Notification Service - Dsign

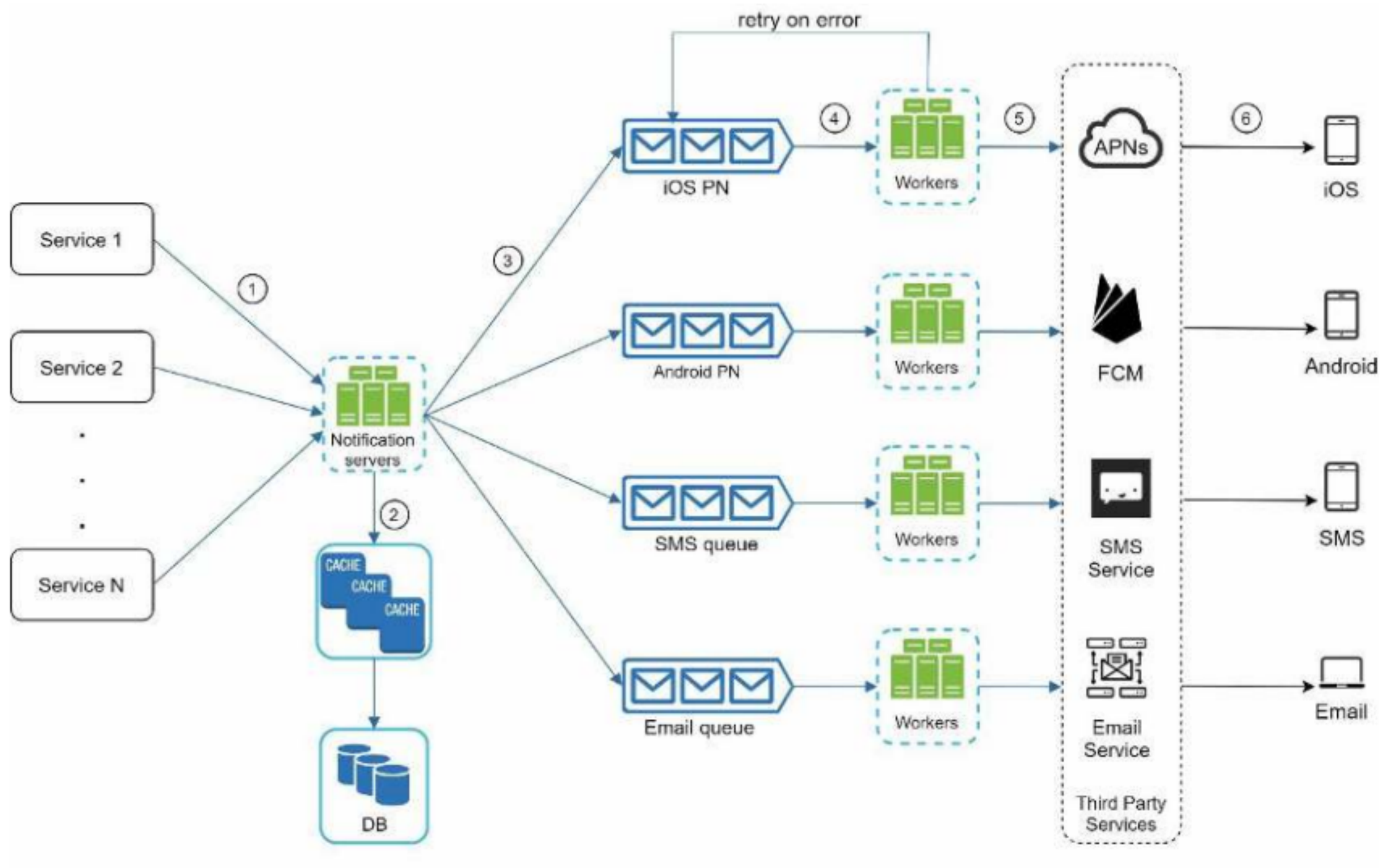

retry on error
Service 1
Service 2
Service N
①
②
③
④
⑤
⑥
Notification servers
CACHE
DB
iOS PN
Android PN
SMS queue
Email queue
Workers
APNs
FCM
SMS Service
Email Service
Third Party Services
iOS
Android
SMS
Email

Figure 10-10

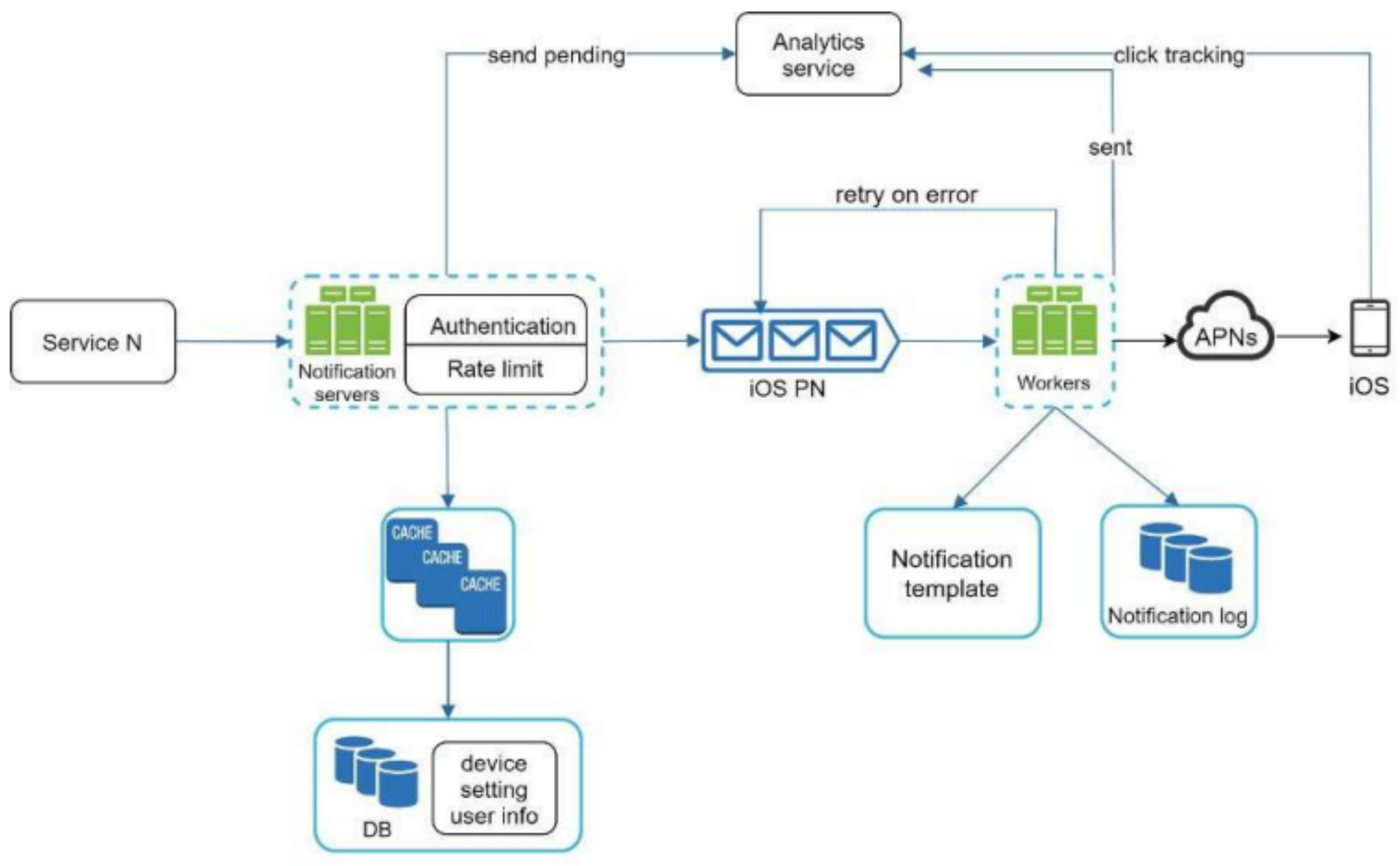

send pending
Analytics service
click tracking
sent
retry on error
Service N
Notification servers
Authentication
Rate limit
iOS PN
Workers
APNs
iOS
CACHE
DB
device setting user info
Notification template
Notification log

Figure 10-14

Uber Driver Location (2 requirements)

In ride hailing service, Drivers upload their locations every second. Design a system that takes data and allows us to serve a heat map of their locations

Requirement 1:
We want to build a tool for our operations folks. They need to monitor the health of their city in real time. We need to serve the last 20 minutes of data for this tool. We want to split up the data into one minute buckets for each of those 20 minutes (thus yielding 20 different sets of data we can visualize).

Requirement 2:
We want to expose this data for consumption by cities and research institutions. Since it will be used for research purposes, we do not need data in real time. The data need only be available after 24 hours. Furthermore, we will summarize the data over the hour (rather than the minute above) and this data can only be requested in full hour increments (12:00-16:00 but not 12:05-16:05)

Requirement 1: Real-Time Heat Map for Operations

Functional Req:
- Ingest driver location data every second.
- Store the last 20 minutes of location data.
- Aggregate data into 1-minute buckets.
- Serve the data as a heat map for operations to visualize real-time city health.

Non-Functional Req:
- Low Latency: The system must provide near-real-time updates for location data.
- Scalability: Handle high data ingestion rates due to thousands of drivers reporting locations every second.
- Availability: The system should be highly available to ensure the tool is accessible at all times.
- Fault Tolerance: Ensure data persistence in the case of failures.

Back-of-the-Envelope:
- Data Ingestion Rate: Assume 10,000 drivers per city, with each driver sending 1 update per second
  Data rate = 10,000 updates/sec.

- Storage for 1 minute of data:
  Each location update contains driver_id, timestamp, lat, lon (40 bytes).
  Storage per minute = 10,000 * 40 bytes = 400 KB.
  Storage for 20 minutes = 20 * 400 KB = 8 MB per city.

- Queries per Second: Assume 100 operations staff accessing the tool.
  If each user generates 1 query every 5 seconds, then 20 QPS.

API Endpoints:

- POST /location: Accepts real-time location updates from drivers.
  Request: { "driver_id": "string", "timestamp": "ISO 8601", "lat": float, "lon": float }

- GET /heatmap?city_id={city_id}&start={timestamp}&end={timestamp}: Retrieves heat map data for a given time range.
  Response: { "timestamp": "ISO 8601", "buckets": [{ "minute": int, "data": [{ "lat": float, "lon": float, "count": int }] }] }

Database Tables:
- RealTime_Locations (city_id, driver_id, timestamp, lat, lon)

- Minute_Buckets:
  city_id,
  minute_timestamp (rounded to the minute)
  location_count
  lat_lon_grid (serialized data representing location densities)

Final Design Diagram for Requirement 1:

Components:
- API Gateway:
  Handles requests and routes them to relevant services.
- Location Ingestor Service:
  Accepts location data, validates it, and sends it to a stream.
- Stream Processor (e.g., Apache Kafka, Flink):
  Aggregates location data into minute buckets and writes to a time-series DB.
- Time-Series Database (e.g., InfluxDB):
  Stores the aggregated data for 20 minutes, allowing for efficient retrieval.
- Heat Map Service:
  Fetches the data from the time-series database and formats it for visualization.
- Cache (e.g., Redis):
  Caches recent minute-level data for quick access by the Heat Map Service.

Deep Dive into Major Components:

- Location Ingestor Service:
  *Responsible for processing high-frequency location updates from drivers.
  *Writes the data into a stream for further processing.
  *Performs basic validation (e.g., checking timestamp format).

- Stream Processor:
  *Aggregates data into 1-minute buckets based on time and geospatial regions.
  *Writes aggregated data to a time-series database.

- Time-Series Database:
  *Optimized for time-based queries, allowing efficient retrieval of data for the last 20 minutes.
  *Retains only the last 20 minutes of data, evicting older data.

- Heat Map Service:
  *Queries the time-series database for recent data.
  *Processes data into a heat map format, using geospatial libraries to map densities.

Req 1:

```
+----------------------+
|    API Gateway    |
+----------+-----------+
           |
           v
+----------------------+             +---------------------+
| Location Ingestor|             | Cache (e.g., Redis)|
| Service               |-----------| Stores recent      |
|                          |             | minute-level data |
+----------+-----------+             +---------------------+
           |
           v
+----------------------+
|  Stream Processor|
| (e.g., Kafka, Flink) |
| Aggregates data  |
| into 1-minute        |
| buckets               |
+----------+-----------+
           |
           v
+----------------------------+
| Time-Series Database|
| (e.g., InfluxDB)         |
| Stores last 20 mins    |
| of data                     |
+----------+-----------------+
           |
           v
+-------------------------+
|  Heat Map Service    |
| Queries DB and       |
| Cache for data        |
| Generates heat map |
+----------+--------------+
           |
           v
+-----------------------+
|    Real-Time Tool   |
|   for Operations     |
| Displays heat map   |
+-----------------------+
```

Requirement 2: Summary Data for Research Consumption

1. Functional Requirements:
   - Summarize driver location data over 1-hour intervals.
   - Store data for 24 hours.
   - Provide data for download in full-hour increments (e.g., 12:00-13:00).

2. Non-Functional Requirements:
   - Data Consistency: Ensure that the summarized data is accurate.
   - Durability: Summarized data should be stored securely for 24 hours.
   - Accessibility: Provide a simple interface for external entities to access and download data.

3. Back-of-the-Envelope Estimation:
   - Data Summarization Rate: Assume 10,000 drivers sending location data every second.
     *36,000,000 updates per hour.
     *Summarization reduces the data to lat-lon buckets with counts.

   - Storage for 1 hour of summarized data:
     *Aggregated data per hour ≈ 100 KB.
     *For 24 hours = 2.4 MB per city.

4. API Endpoints:
   - Retrieves summarized data for a given time range.
     GET /summary?city_id={city_id}&start={timestamp}&end={timestamp}:
     Response: { "hour": "ISO 8601", "data": [{ "lat": float, "lon": float, "count": int }] }

5. Extended Database Tables:

   Hourly_Summary:
     city_id
     hour_timestamp (rounded to the hour)
     location_count
     lat_lon_grid (serialized data representing location densities)

6. Final Design Diagram for Requirement 2:

   - Components (New/Updated):
     Batch Processor: Processes data from the stream every hour to create summarized data.
     Object Storage (e.g., S3): Stores hourly summarized data files for 24 hours.
     Research Data Service: Provides an interface for external entities to access and download the summarized data.

7. Deep Dive into Major Components (Extension):
   - Batch Processor:
     *Runs every hour to process and summarize the last hour's location data.
     *Writes summarized data into object storage for further retrieval.

   - Object Storage (S3):
     *Stores hourly data, providing durable storage and allowing for easy retrieval of data by external clients.

   - Research Data Service:
     *Handles access control and data retrieval requests.
     *Ensures that data is delivered in the correct format and respects full-hour time boundaries.

This design ensures a scalable and efficient solution for both real-time operational monitoring and delayed research data access, optimizing for different latency and storage requirements in each case.

Req 2:

```
+----------------------+
|    API Gateway    |
+----------+-----------+
           |
           v
+----------------------+            +---------------------+
| Location Ingestor|            | Cache (e.g., Redis)|
| Service               |-----------| Stores recent      |
|                          |            | minute-level data |
+----------+-----------+            +---------------------+
           |
           v
+----------------------+
|  Stream Processor|
| (e.g., Kafka, Flink) |
| Aggregates data  |
| into 1-minute        |
| buckets               |
+----------+-----------+
           |
           v
+----------------------------+
| Time-Series Database|
| (e.g., InfluxDB)         |
| Stores last 20 mins    |
| of data                     |
+----------+-----------------+
           |
           v
+----------------------+            +-------------------------+
|  Heat Map Service. |            |   Batch Processor  |
| Queries DB and      |            | Processes hourly     |
| Cache for data       |----------| summaries from the |
| Generates heat map|            | stream every hour   |
+----------+-----------+            +---------+---------------+
           |                                   |
           v                                   v
+----------------------+            +----------------------+
|    Real-Time Tool    |             |   Object Storage    |
|   for Operations     |             | (e.g., S3)             |
| Displays heat map   |             | Stores hourly         |
+----------------------+            | summaries for 24 hrs|
                                    +---------+------------+
                                              |
                                              v
                                    +----------------------+
                                    | Research Data        |
                                    | Service                |
                                    | Provides access to  |
                                    | summarized data      |
                                    +---------+------------+
                                              |
                                              v
                                    +----------------------+
                                    | External Consumers  |
                                    | (Cities, Researchers)|
                                    | Access summarized   |
                                    | data by hour          |
```

*** Design Diagram: Delivery Service Real-Time Tracking Flow ***

```
+---------------------------------------------------------------------+
|                   Driver's Mobile App (GPS)                        |
|---------------------------------------------------------------------|
| - Sends driver's GPS coordinates to backend periodically   |
| - Uses WebSocket / HTTP API to send real-time data         |
+---------------------------------------------------------------------+
                              |
                              v
+----------------------------------------------------------------------+
|                      WebSocket / HTTP API                           |
|----------------------------------------------------------------------|
| - Facilitates real-time data transmission                          |
| - Receives GPS coordinates and forwards them to backend   |
+----------------------------------------------------------------------+
                              |
                              v
+-----------------------------------------------------------------+
|                 Real-Time Tracking DB (NoSQL)                 |
|-----------------------------------------------------------------|
| - Stores driver's current GPS location                        |
| - Tracks real-time driver status (e.g., Available, On Trip)  |
| - Supports geospatial queries (e.g., nearby drivers)        |
+-----------------------------------------------------------------+
                              |
                              v
+-----------------------------------------------------------------+
|                  Delivery Service                                 |
|-----------------------------------------------------------------|
| - Matches available drivers based on proximity               |
| - Tracks real-time driver location using Real-Time DB       |
| - Updates Order Service on delivery progress                |
+-----------------------------------------------------------------+
                              |
                              v
+----------------------------------------------------------------+
|                 Map Service (Google Maps API)                 |
|----------------------------------------------------------------|
| - Displays real-time driver location on a map                |
| - Calculates ETA and route optimization                       |
+----------------------------------------------------------------+
                              |
                              v
+------------------------------------------------------------------+
|              Order Service & Notification Service            |
|------------------------------------------------------------------|
| - Updates order status (Picked Up, Out for Delivery)       |
| - Sends notifications to customers with delivery updates  |
+------------------------------------------------------------------+
```

Distributed Job/Task Scheduler

[Google Doc: Look here for more details: https://docs.google.com/document/d/1Hplx3vXIbXQE_rY_1T5rRKDZN4NGvA8lAdWUCiWDHSY/edit?tab=t.0
Useful Links: https://bugfree.ai/system-design/distributed-job-scheduler
]

1. Functional Requirements
- Job Registration:
  Users can create, update, and delete jobs. Jobs may have specific schedules, recurrence intervals, and dependencies on other jobs.
- Job Execution:
  Execute jobs at their specified time or upon meeting conditions (e.g., cron jobs, interval-based jobs, or event-driven tasks).
- Job Monitoring:
  Track the status of each job (pending, running, completed, failed, or retried).
  Allow users to query the status and history of jobs.
- Job Retry:
  Automatically retry failed jobs based on pre-configured retry policies.
- Job Priority:
  Handle jobs with varying priorities to ensure that critical jobs are processed first.
- Task Dependencies:
  Support dependencies between jobs, ensuring jobs that rely on other jobs only execute once their dependencies are complete.
- Job Cancellation: Allow users to cancel or pause running jobs.
- Concurrency Control:
  Handle concurrent job executions while avoiding resource conflicts.
- Alerting and Notifications: Notify users of job failures, successes, or retry attempts through email or webhooks.
- Scalability: Ability to scale horizontally to handle a large number of jobs and distribute them across multiple nodes.

2. Non-Functional Requirements
- Scalability:
  The system should support horizontal scaling to handle an increasing number of jobs and execute them efficiently.
- High Availability:
  Ensure the scheduler is highly available and can tolerate node failures without job loss.
- Fault Tolerance:
  The system should gracefully recover from failures and continue processing jobs.
- Low Latency:
  Minimize the delay between job scheduling and execution, especially for time-sensitive jobs.
- Data Consistency:
  Ensure that job state (e.g., pending, running, completed) is consistent across all nodes.
- Throughput:
  Support a high number of job executions per second to meet user demand.
- Security:
  Ensure secure access to APIs, data encryption, and secure storage of user credentials and job data.
- Monitoring and Logging:
  Provide detailed logging and metrics for monitoring job executions and system performance.
- Multi-tenancy: Support multiple users or teams, ensuring data isolation between different tenants.

3. Back-of-the-Envelope Estimation
- Number of Jobs:
  Assume 1 million jobs scheduled daily, with peaks of 10,000 jobs per second.
- Job Storage:
  Each job record (metadata, status) is around 1 KB. Storing 1 million job records requires around 1 GB of storage per day.
- Job History Retention:
  Retain job history for 30 days, leading to a data storage of ~30 GB.
- API Requests:
  Assuming 10,000 API requests per second for job management (create, update, delete, status).
- Number of Nodes:
  With an average job execution time of 2 seconds, around 500 worker nodes could handle the load, considering 20 jobs per node per second.

4. API Endpoints
- Job Management

  POST /jobs: Create a new job.
  PUT /jobs/{job_id}: Update job details.
  DELETE /jobs/{job_id}: Delete a job.
  GET /jobs/{job_id}: Get details of a specific job.
  GET /jobs: List all jobs with filtering options (status, priority, time range).

- Job Execution and Status

  POST /jobs/{job_id}/run: Trigger a job immediately.
  GET /jobs/{job_id}/status: Get the current status of a job.
  GET /jobs/{job_id}/logs: Retrieve execution logs of a job.
  POST /jobs/{job_id}/cancel: Cancel a running job.

- Job Monitoring

  GET /jobs/history: Get historical data for jobs within a time range.
  POST /jobs/{job_id}/retry: Manually retry a failed job.

- Alerts and Notifications

  POST /alerts: Configure alerts for job completion/failure.

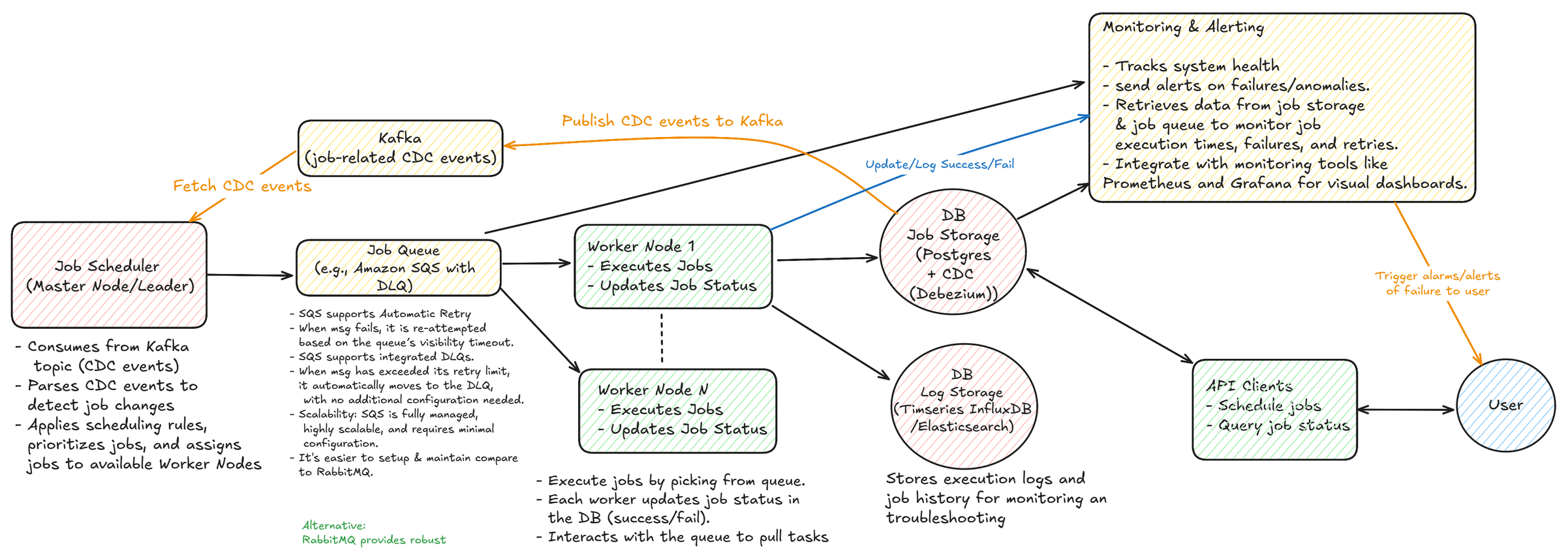
5. Database Tables (Field Names Only)
- Jobs
  job_id (UUID)
  name
  schedule (cron expression or time interval)
  priority (integer)
  retry_policy (max_attempts, delay between retries)
  status (pending, running, completed, failed)
  created_at, updated_at

- Job Dependencies
  job_id (UUID)
  depends_on (UUID of the job this job depends on)

- Job Executions
  execution_id (UUID)
  job_id (UUID)
  status (running, completed, failed)
  start_time, end_time
  logs (text or URL to logs)

- Job Alerts
  alert_id (UUID)
  job_id (UUID)
  type (success, failure)
  notification_method (email, webhook)
  contact_details (email address or webhook URL)

Final Design Diagram


Kafka
(job-related CDC events)
Fetch CDC events
Publish CDC events to Kafka
Job Scheduler
(Master Node/Leader)
Job Queue
(e.g., Amazon SQS with DLQ)
Worker Node 1
- Executes Jobs
- Updates Job Status
Worker Node N
- Executes Jobs
- Updates Job Status
Update/Log Success/Fail
DB
Job Storage
(Postgres
+ CDC
(Debezium))
DB
Log Storage
(Timseries InfluxDB
/Elasticsearch)
Monitoring & Alerting
- Tracks system health
- send alerts on failures/anomalies.
- Retrieves data from job storage
& job queue to monitor job
execution times, failures, and retries.
- Integrate with monitoring tools like
Prometheus and Grafana for visual dashboards.
Trigger alarms/alerts
of failure to user
API Clients
- Schedule jobs
- Query job status
User

- Consumes from Kafka topic (CDC events)
- Parses CDC events to detect job changes
- Applies scheduling rules, prioritizes jobs, and assigns jobs to available Worker Nodes

- SQS supports Automatic Retry
- When msg fails, it is re-attempted based on the queue's visibility timeout.
- SQS supports integrated DLQs.
- When msg has exceeded its retry limit, it automatically moves to the DLQ, with no additional configuration needed.
- Scalability: SQS is fully managed, highly scalable, and requires minimal configuration.
- It's easier to setup & maintain compare to RabbitMQ.

Alternative:
RabbitMQ provides robust message routing, reliable acknowledgment, and retry mechanisms, making it ideal for managing job distribution and ensuring jobs are processed accurately in a distributed env.

- Execute jobs by picking from queue.
- Each worker updates job status in the DB (success/fail).
- Interacts with the queue to pull tasks and with DB to update job statuses.
- Workers should be stateless to enable horizontal scaling.
- Use thread pools within workers for parallel job execution.
- Logs to log storage.

Stores execution logs and job history for monitoring an troubleshooting

Why Separate Databases?
- Different Access Patterns:
  Job storage requires frequent reads, updates, and transactions, while log storage is write-heavy with occasional reads, often in the form of batch analysis.

- Optimized for Different Workloads:
  Using a database like PostgreSQL or Redis for job storage ensures fast job updates and queries, while Elasticsearch or InfluxDB for log storage allowsfor efficient log indexing, searching, and analysis.

- Scalability Considerations:
  Using different databases allows each system to scale independently.
  Log storage systems like Elasticsearch can scale horizontally as the log volume grows, while job storage can focus on maintaining consistency and fast access times.

Deep Dive into Major Components

- Job Scheduler (Master Node/Leader)

  * Role:
    The central component that manages job distribution to worker nodes.
    It maintains a global job queue and assigns jobs to workers.
  * Interactions:
    Reads job data from the database and pushes tasks into the job queue.
    Continuously monitors worker nodes for job completion or failures.
  * Considerations:
    Leader election mechanism is needed for high availability, e.g., using Zookeeper or etcd.

- Job Queue (e.g., Redis, RabbitMQ)

  * Role:
    Stores jobs in a queue for workers to pick up based on their load. Supports prioritized queues to ensure high-priority jobs are executed first.
  * Interactions:
    Worker nodes consume jobs from the queue, and the scheduler pushes new jobs or retries into the queue.
  * Considerations:
    Choosing a queue with support for priority handling and message acknowledgment is crucial to prevent job loss.

- Worker Nodes

  * Role:
    Execute jobs by picking them from the job queue. Each worker updates the status of the job in the database upon completion or failure.
  * Interactions:
    Interacts with the job queue to pull tasks and with the database to update job statuses.
  * Considerations:
    Workers should be stateless to enable horizontal scaling. Use thread pools within workers for parallel job execution.

- Job Storage (SQL/NoSQL Database) :
  Use different DB for Job storage (Postgres/DynamoDB/MongoDb) and for log storage (TimeSeries InfluxDB/Elasticsearch).
  Store archived or old logs to S3.

  * Role: Stores job metadata, schedules, execution logs, and history.
  * Interactions: Used by the scheduler to read job configurations and by worker nodes to update job statuses.
  * Considerations:
    A relational database like PostgreSQL or MySQL is ideal for structured job data, while NoSQL options like MongoDB can be used for logs or metadata.

  Separate Databases.
  Different Access Patterns: Job storage requires frequent reads, updates, and transactions, while log storage is write-heavy with occasional reads, often in the form of batch analysis.

  Optimized for Different Workloads: Using a database like PostgreSQL or Redis for job storage ensures fast job updates and queries, while Elasticsearch or InfluxDB for log storage allows for efficient log indexing, searching, and analysis.

  Scalability Considerations: Using different databases allows each system to scale independently.
  Log storage systems like Elasticsearch can scale horizontally as the log volume grows, while job storage can focus on maintaining consistency and fast access times.

- Monitoring & Alerting

  * Role: Tracks the health of the system and job executions, sending alerts for failures or anomalies.
  * Interactions: Retrieves data from job storage and job queue to monitor job execution times, failures, and retries.
  * Considerations: Integration with monitoring tools like Prometheus and Grafana for visual dashboards.

- User/API Clients

  * Role: Interfaces used by users to interact with the scheduler through REST APIs for creating, updating, or querying jobs.
  * Interactions: Sends requests to the job scheduler and receives updates on job statuses.
  * Considerations: Ensure authentication and authorization for secure access to job data.

This design provides a scalable, fault-tolerant solution for scheduling and managing distributed tasks while supporting varied job types, priorities, and dependencies.

- Monitoring & Alerting

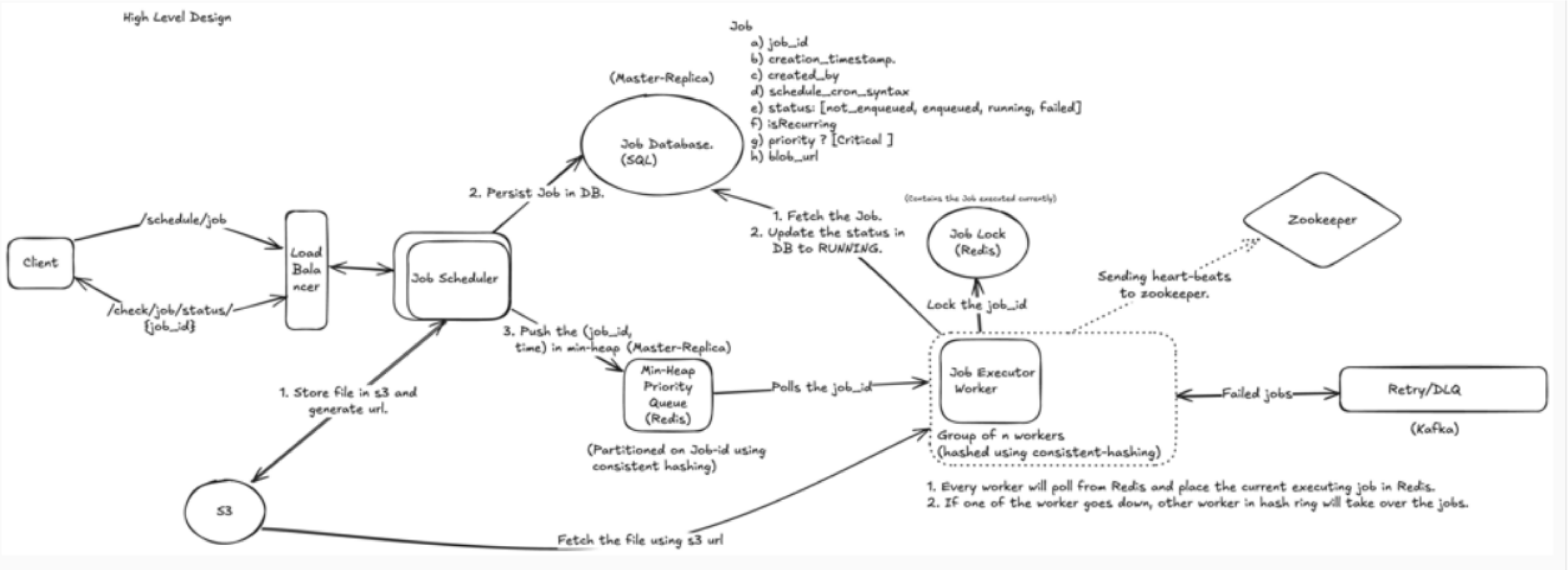
  * Role: Tracks the health of the system and job executions, sending alerts for failures or anomalies.
  * Interactions: Retrieves data from job storage and job queue to monitor job execution times, failures, and retries.
  * Considerations: Integration with monitoring tools like Prometheus and Grafana for visual dashboards.

- User/API Clients

  * Role: Interfaces used by users to interact with the scheduler through REST APIs for creating, updating, or querying jobs.
  * Interactions: Sends requests to the job scheduler and receives updates on job statuses.
  * Considerations: Ensure authentication and authorization for secure access to job data.

This design provides a scalable, fault-tolerant solution for scheduling and managing distributed tasks while supporting varied job types, priorities, and dependencies.

Task/Job Scheduler [Another Approach From: https://leetcode.com/discuss/post/6428187/design-distributed-job-scheduler-by-anon-retx/]


High Level Design
Client
/schedule/job
/check/job/status/{job_id}
Load Balancer
Job Scheduler
2. Persist Job in DB.
(Master-Replica)
Job Database. (SQL)
Job
a) job_id
b) creation_timestamp.
c) created_by
d) schedule_cron_syntax
e) status: [not_enqueued, enqueued, running, failed]
f) isRecurring
g) priority ? [Critical]
h) blob_url
1. Fetch the Job.
2. Update the status in DB to RUNNING.
(Contains the Job executed currently)
Job Lock (Redis)
Lock the job_id
Zookeeper
Sending heart-beats to zookeeper.
3. Push the (job_id, time) in min-heap (Master-Replica)
Min-Heap Priority Queue (Redis)
(Partitioned on Job-id using consistent hashing)
Polls the job_id
Job Executor Worker
Group of n workers (hashed using consistent-hashing)
Failed jobs
Retry/DLQ
(Kafka)
1. Store file in s3 and generate url.
S3
Fetch the file using s3 url
1. Every worker will poll from Redis and place the current executing job in Redis.
2. If one of the worker goes down, other worker in hash ring will take over the jobs.

* Functional Requirements

- Submit an immediate or future scheduled job
- Execute eligible jobs using horizontally scalable workers
- Track status: SCHEDULED → READY → RUNNING
  → terminal (COMPLETE / FAILED / CANCELLED)
(terminal is not a status. It means the task has reached a final state and will not transition further in the normal flow.)

- Retry failed or abandoned jobs with configurable policy
- Cancel scheduled/running jobs where possible
- Support recurring schedules

FOLLOW-UP — DAG dependencies, priority scheduling

* Non-Functional Requirements

Durability — Accepted jobs must not disappear
Availability — Scheduling continues despite individual scheduler/worker failures
Execution semantics — At-least-once; duplicate-safe
Schedule latency — Eligible jobs begin within seconds under normal load
Scalability — Millions of jobs/day, tens of thousands concurrently running

MUST SAY
Durable acceptance before acknowledging submission.
Scheduler/worker failures must not permanently lose jobs.
At-least-once execution means duplicate handling is part of correctness.

* Back-of-the-Envelope Estimation
Assume 10M task executions/day

Average start rate: 10M / 86,400 ≈ 116 tasks/sec

10x peak: ≈ 1,200 task starts/sec

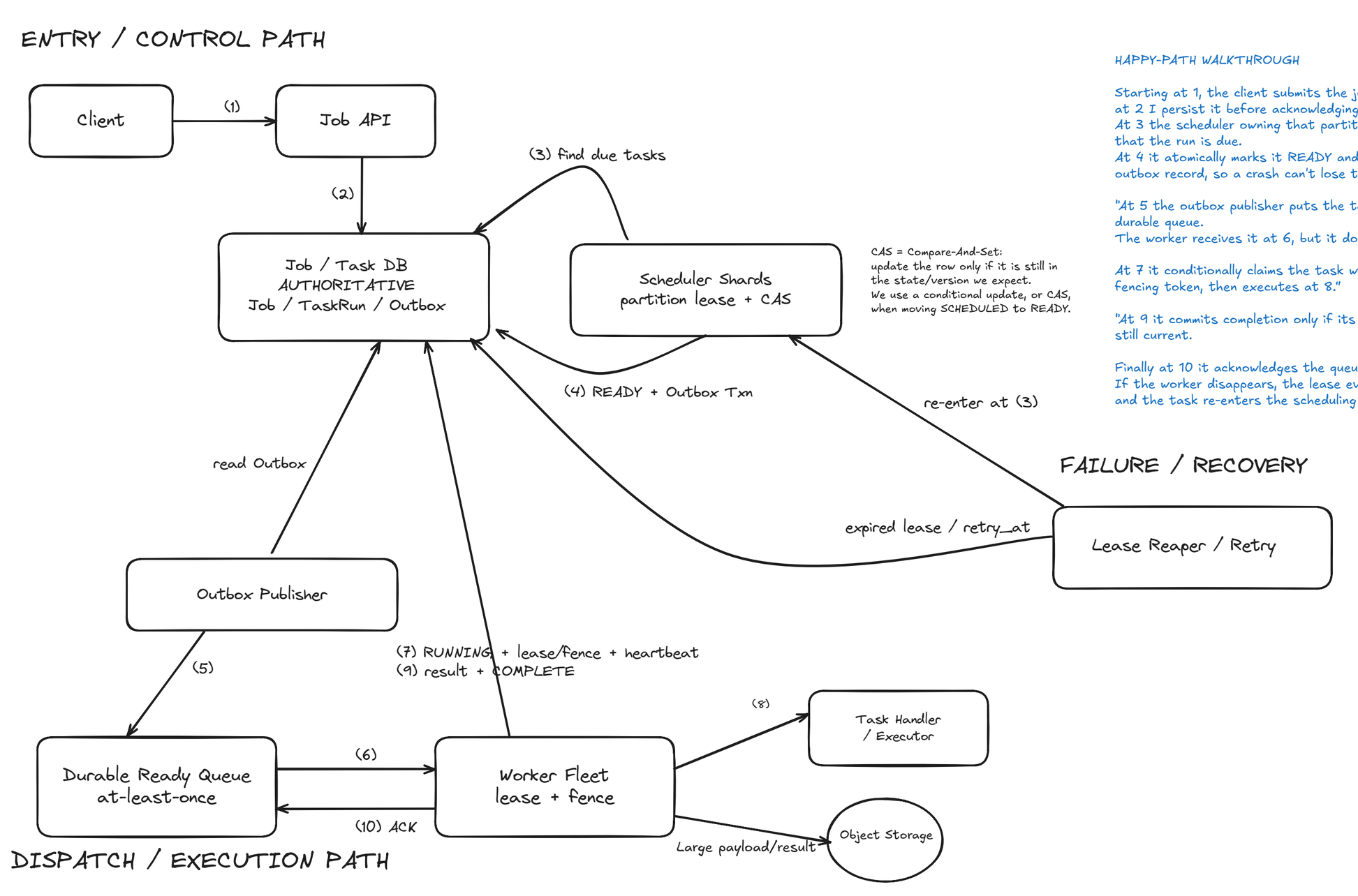
Average execution = 60 sec
Peak concurrent executions: 1,200 × 60 ≈ 72,000

~2 KB metadata/task:
10M × 2 KB ≈ 20 GB/day of task metadata

Architecture Implications

~72K concurrent executions → workers must scale horizontally.
~1.2K peak starts/sec → one scheduler process should not own the entire schedule space.
Millions of future tasks → don't hold timers in memory; scheduled state must remain durable/indexed.
~20 GB/day metadata → retention/archival matters, but isn't on the hot execution path.

What I Say
"The important number isn't submission QPS; it's concurrency. Around seventy thousand simultaneous executions means workers are naturally horizontally scaled. And because millions of timers may be pending, I want scheduling represented durably in storage rather than millions of in-memory timers."


ENTRY / CONTROL PATH
Client
(1)
Job API
(2)
Job / Task DB
AUTHORITATIVE
Job / TaskRun / Outbox
(3) Find due tasks
Scheduler Shards
partition lease + CAS
(4) READY + Outbox Txn
re-enter at (3)
FAILURE / RECOVERY
expired lease / retry_at
Lease Reaper / Retry
read Outbox
Outbox Publisher
(5)
(7) RUNNING + lease/fence + heartbeat
(9) result + COMPLETE
Durable Ready Queue
at-least-once
(6)
(10) ACK
Worker Fleet
lease + fence
(8)
Task Handler
/ Executor
Large payload/result
Object Storage
DISPATCH / EXECUTION PATH

CAS = Compare-And-Set:
update the row only if it is still in the state/version we expect.
We use a conditional update, or CAS, when moving SCHEDULED to READY.

CORRECTNESS
Scheduler = partition lease + CAS
Dispatch = transactional outbox
Delivery = at-least-once
Worker = lease + fencing token

HAPPY-PATH WALKTHROUGH

Starting at 1, the client submits the job, and at 2 I persist it before acknowledging.
At 3 the scheduler owning that partition discovers that the run is due.
At 4 it atomically marks it READY and creates an outbox record, so a crash can't lose the dispatch."

"At 5 the outbox publisher puts the task onto the durable queue.
The worker receives it at 6, but it doesn't yet own execution.

At 7 it conditionally claims the task with a lease and fencing token, then executes at 8."

"At 9 it commits completion only if its fencing token is still current.

Finally at 10 it acknowledges the queue message.
If the worker disappears, the lease eventually expires and the task re-enters the scheduling path.

Why DB Table with partition details over Zookeeper/etcd

Because the DB is already required for job/task state, and it already gives us the primitives we need:

conditional updates / CAS
transactions
unique constraints
lease timestamps
version / epoch

So instead of:
```
Scheduler
   |
   +--> Postgres
   |
   +--> etcd / ZooKeeper
```

we can initially have:
Scheduler -> Postgres

That means fewer things to operate and fewer failure boundaries.

Main benefits of using the DB:
- no extra distributed system
- simpler operations, easier debugging
- task state and scheduler ownership live near each other
- easy CAS/lease implementation

sufficient when partition count and lease churn are moderate

When would ZooKeeper/etcd be better?
If coordination itself becomes a high-scale or low-latency concern.

For example:
100K+ scheduler partitions
very frequent ownership changes
many scheduler nodes
sub-second failover requirement
lots of ephemeral membership/leader-election operations

Then etcd/ZooKeeper starts making more sense because they are built specifically for:
- ephemeral leases/sessions
- leader election
- watches
- strongly consistent coordination
- fast ownership changes

The DB approach is simpler, but coordination writes compete with normal workload.

So the trade-off is:
DB table = simpler architecture

etcd / ZooKeeper =

The Easiest Phrase to Remember:
LEASE says how long you own it.
FENCING proves you're still the latest owner.

Why we need FENCING: A lease by itself isn't enough because an expired worker can wake up. So every ownership change increments a fencing token. Once worker B owns token 12, worker A with token 11 can no longer commit authoritative completion.

What is FENCING?
Lease alone has an important race.

FINAL 45-SECOND SUMMARY

The flow is essentially 1 through 10 on the board. A client submits a job through the API, and the job database is the authoritative source of schedule and execution state. Scheduler shards own partitions and determine when work

API Design
POST /jobs — Submit immediate, scheduled, or recurring job
- Req (one time scheduling):
  { "idempotencyKey": "req-12345", "tenantId": "tenant-42",
  "taskType": "GENERATE_REPORT",
  "scheduledAt": "2026-09-05T18:00:00Z",
  "retryPolicy": { "maxAttempts": 3 }
  }
- Req (recurring scheduling):
  { "idempotencyKey": "req-12345", "tenantId": "tenant-42",
  "taskType": "DAILY_REPORT",
  "schedule": { "cron": "0 9 * * *",
  "timezone": "America/Los_Angeles" } }

GET /jobs/{jobId} — Get definition/status
(jobId, taskType, status, scheduledAt, taskId)
GET /tasks/{taskId} — Inspect a particular execution
POST /jobs/{jobId}/cancel — Cancel future/current execution
POST /tasks/{taskId}/retry — Explicitly retry failed execution

Data Model
JOB(job_id PK, tenant_id, schedule/cron, payload_ref,
next_run_at, status, scheduler_partition, version)

TASK_RUN(task_id PK, job_id FK → JOB.job_id, scheduled_for,
status, attempt, lease_owner, lease_until, fence_token)

OUTBOX(event_id PK, task_id FK → TASK_RUN.task_id, event_type, published_at)

Critical index: (scheduler_partition, status, scheduled_for) on TASK_RUN
Critical recurring-job constraint: UNIQUE(job_id, scheduled_for) on TASK_RUN
Submission idempotency: UNIQUE(tenant_id, idempotency_key) on JOB

and execution state. Scheduler shards own partitions and determine when work becomes runnable; they atomically mark it READY and create an outbox event, which safely crosses into the asynchronous durable queue."

"Workers consume from that queue, but they only become the authoritative executor after a conditional DB claim that gives them a lease and fencing token.
They execute the task, persist completion using that fence, and then acknowledge the queue message. That gives us at-least-once execution without losing jobs, scales through scheduler partitioning and independent worker scaling, and recovers crashed workers by expiring leases and feeding the task back through the same scheduling path.

SCHEDULED = wait until scheduled time

READY = time has arrived; waiting for worker

RUNNING = some worker currently owns execution

COMPLETE = execution succeeded

Imagine:
10:00 Worker A starts Task, fence = 10
10:02 Worker A freezes maybe long GC pause/network partition
10:07 A's lease expires
10:08 Worker B takes Task fence = 11
10:09 Worker B executes
10:10 Worker A wakes up!

Now both A and B may think they're executing Task 123.
Lease expiration tells us that B may take ownership.
But it cannot physically stop A from waking up.
That's why we add a fencing token.

Every new owner gets a larger token:
Worker A -> token 10
Worker B -> token 11

DB knows: current token = 11

Worker A tries: COMPLETE task
token = 10

DB says: REJECT, 10 != 11

Worker B: COMPLETE task, token = 11
DB says: ACCEPT

That's fencing.

better dedicated coordination semantics
extra infrastructure + another failure domain

Metrics Monitoring and Aggregation System

Functional
- Ingestion (push) / Collection
- Aggregate metrics (min, max, avg, sum, p95, p99)
  over time windows (1m, 5m, 1h).
- Storage tiers: hot (recent, high-res),
  warm (compressed, downsampled), cold (long-term).
- Should be able Query metrics: time-range queries,
  label filters, group-by, downsampling hint.
- Alerts & notifications
- Cardinality control: guardrails against unbounded
  label explosion. [Set rules ""You can't use high-cardinality
  labels like user_id directly. You must aggregate them
  first into something like user_country.""]
- Multi-Tenancy: The warehouse has separate sections
  for each team (Tenants). The "Search Team" has their
  own quota and can't use all the space and slow down
  the "Notifications Team."

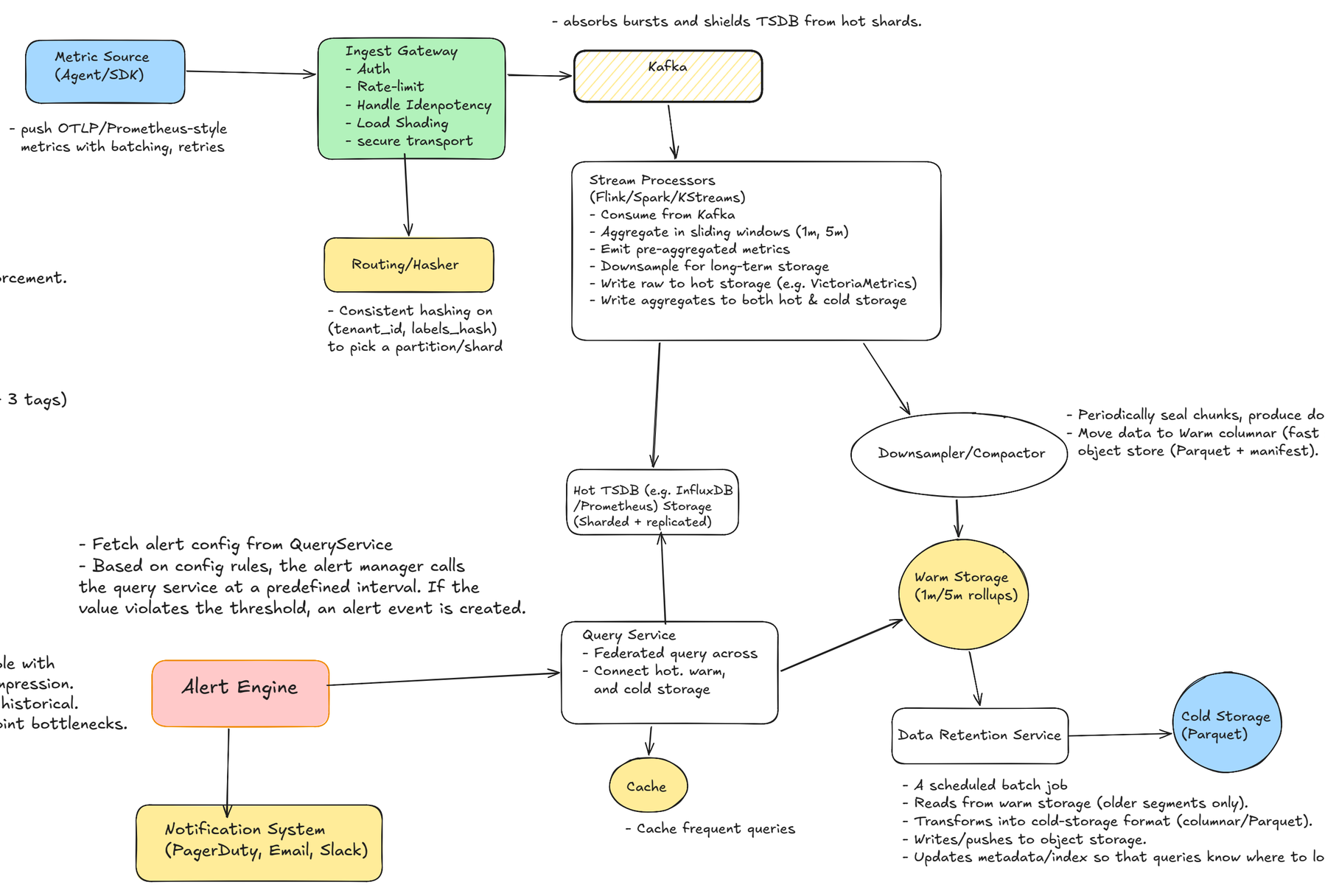
Non-Functional
- Scalability: 1M+ metrics/sec ingestion
- High availability: 99.95% / 99.99% (4 nine) for query &
  alerting; ingestion should buffer during partial outages.
- Low Latency: <1s ingestion → queryable, <100ms query (P99)
  Real-time alerting needs low latency.
- Durability: No data loss after commit and during node failure.
  WAL + replication (RF=3), RF=3 stands for Replication
  Factor 3, means data is stored on three different nodes.
- Throughput: sustain multi-million samples/sec global ingestion.
- Cost Efficiency: Old data stored in cheaper archive (S3/AWS glacier)
- Governance: schema registry + metric linting; deprecation & TTL enforcement.

Back-of-the-Envelope: Let's estimate scale to size components.
Assumptions:
- 10,000 servers/microservices
- Each emits 100 metrics/sec (e.g., latency, QPS, errors)
- Average metric size: 100 bytes (metric name + value + timestamp + 3 tags)

Calculations:
- Total metrics per second = 10,000 × 100 = 1M/sec
- Data rate = 1M × 100 bytes = 100 MB/sec
- Daily volume = 100 MB/sec × 86,400 sec ≈ 8.64 TB/day
- Yearly volume (raw) ≈ 3.15 PB
- With 10:1 compression → ~315 TB/year
- Retention: 15 days raw, 1 year downsampled → ~315 TB total
  - Hot: 15 days with 10secs downsampled
  - Warm: next 75 days with 1min downsampled
  - Cold: 365 days with 5min downsampled

Interview Script:
"We're looking at ~100 MB/sec of incoming data — that's manageable with distributed queues. But yearly storage is ~300+ TB even after compression. So we'll need tiered storage: hot storage for recent data, cold for historical. Also, 1M metrics/sec means we need parallel ingestion — no single-point bottlenecks. This informs our choice of message queues and storage engines."

"The Ingest Gateway is not the same as a general-purpose API Gateway. While an API Gateway handles user-facing API calls for dashboards, alert rules, or queries, the Ingest Gateway is specialized for high-throughput telemetry ingestion. It validates, deduplicates, batches, and securely forwards metrics into the pipeline. For example, OpenTelemetry Collector or Cortex/Mimir Ingest Gateway is best suited here. This separation allows us to scale the metrics ingestion independently from the control-plane APIs."


Metric Source (Agent/SDK)
Ingest Gateway
- Auth
- Rate-limit
- Handle Idenpotency
- Load Shading
- secure transport
Kafka
Routing/Hasher
Stream Processors (Flink/Spark/KStreams)
- Consume from Kafka
- Aggregate in sliding windows (1m, 5m)
- Emit pre-aggregated metrics
- Downsample for long-term storage
- Write raw to hot storage (e.g. VictoriaMetrics)
- Write aggregates to both hot & cold storage
Downsampler/Compactor
Hot TSDB (e.g. InfluxDB /Prometheus) Storage (Sharded + replicated)
Warm Storage (1m/5m rollups)
Query Service
- Federated query across
- Connect hot. warm, and cold storage
Alert Engine
Notification System (PagerDuty, Email, Slack)
Cache
Data Retention Service
Cold Storage (Parquet)

- absorbs bursts and shields TSDB from hot shards.

- push OTLP/Prometheus-style metrics with batching, retries

- Consistent hashing on (tenant_id, labels_hash) to pick a partition/shard

- Periodically seal chunks, produce downsampled aggregates (1m, 5m).
- Move data to Warm columnar (fast scans, cheap storage), then Cold object store (Parquet + manifest).

- Fetch alert config from QueryService
- Based on config rules, the alert manager calls the query service at a predefined interval. If the value violates the threshold, an alert event is created.

- Cache frequent queries

- A scheduled batch job
- Reads from warm storage (older segments only).
- Transforms into cold-storage format (columnar/Parquet).
- Writes/pushes to object storage.
- Updates metadata/index so that queries know where to look.

API:
Ingest (Push/write): POST /v1/metrics
Query:
GET /v1/metrics/query?metric=cpu.usage&start=ts&end=ts&step=1m
→ Query aggregated metrics

POST /v1/query — instant query (expr, ts)
POST /v1/query_range — range query (expr, start, end, step, downsample_hint)

GET /v1/labels
GET /v1/label/<name>/values

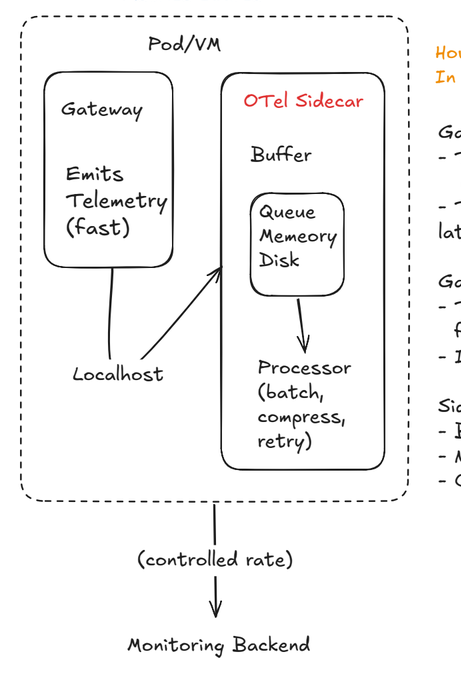
Alerting & SLO:
POST /v1/alerts — Define alert rules (e.g., CPU > 80% for 5m)

GET /alerts/status
→ Fetch current triggered alerts

GET /dashboard/:id
→ Render saved dashboards

Metadata:
GET /v1/usage — samples/sec, active series, storage by tenant

Metrics Source



How It Achieves "Asynchronous Shipping"
In the sidecar implementation:

Gateway emits telemetry locally:
- The Gateway sends metrics/logs/traces to localhost where the sidecar is listening.
- This is extremely fast (in-memory or very low latency)

Gateway immediately continues:
- The Gateway doesn't wait for acknowledgment from the remote monitoring system.
- It trusts the sidecar to handle the delivery.

Sidecar handles the heavy lifting:
- Buffering, batching, retry logic
- Network communication to the monitoring backend
- Compression, authentication, etc.

DB:
Storage choices (why)
*Hot Storage: Hot storage ensures rapid data access when you need it most, while cold storage offers cost-effective solutions for infrequently accessed data.

- TimeSeries (TSDB) - Hot Storage, e.g., VictoriaMetrics/Cortex/Mimir
  These are designed for high-write, high-cardinality workloads.

  T_series_index - stores metadata about each unique series
  Fields/columns: (tenant_id, metric_name, labels, first_ts, last_ts)

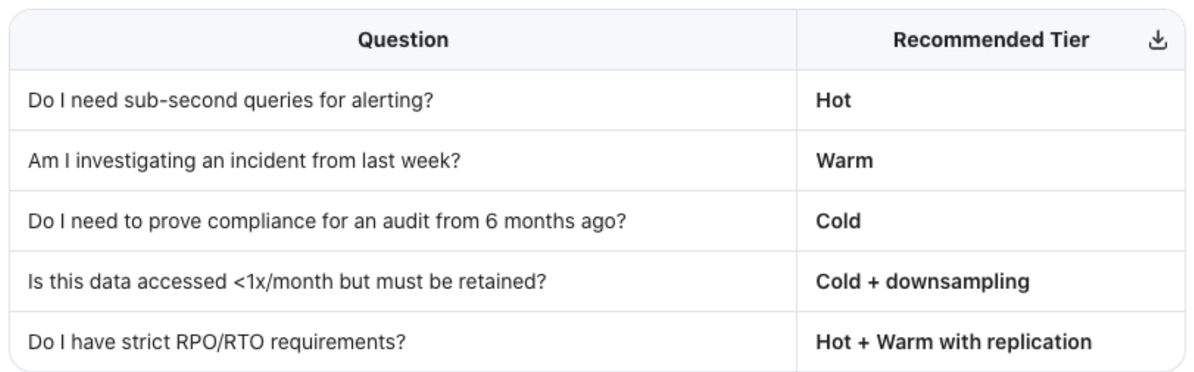
  T_samples - stores actual metric data points (timestamp + value)
  Fields/Columns: (tenant_id, series_id, ts, value)

*Warm Storage (ClickHouse / Bigtable)
- T_Metrics_1m (tenant_id, metric, labels, ts, avg, min, max, p95)
(1-minute aggregates for mid-term queries, ~75d retention)

LabelsCatalog (tenant_id, label_name, label_value, last_seen)
(all label keys/values for filtering and governance)

* Cold Storage (S3/Parquet)
T_Metrics_5m (partitioned by date)
(tenant_id, metric, labels, window_start, window_end, sum, count
(5-minute rollups for long-term retention, 1y+)

✅ Quick Decision Guide



Question	Recommended Tier
Do I need sub-second queries for alerting?	Hot
Am I investigating an incident from last week?	Warm
Do I need to prove compliance for an audit from 6 months ago?	Cold
Is this data accessed <1x/month but must be retained?	Cold + downsampling
Do I have strict RPO/RTO requirements?	Hot + Warm with replication

Notification Service - Dsign

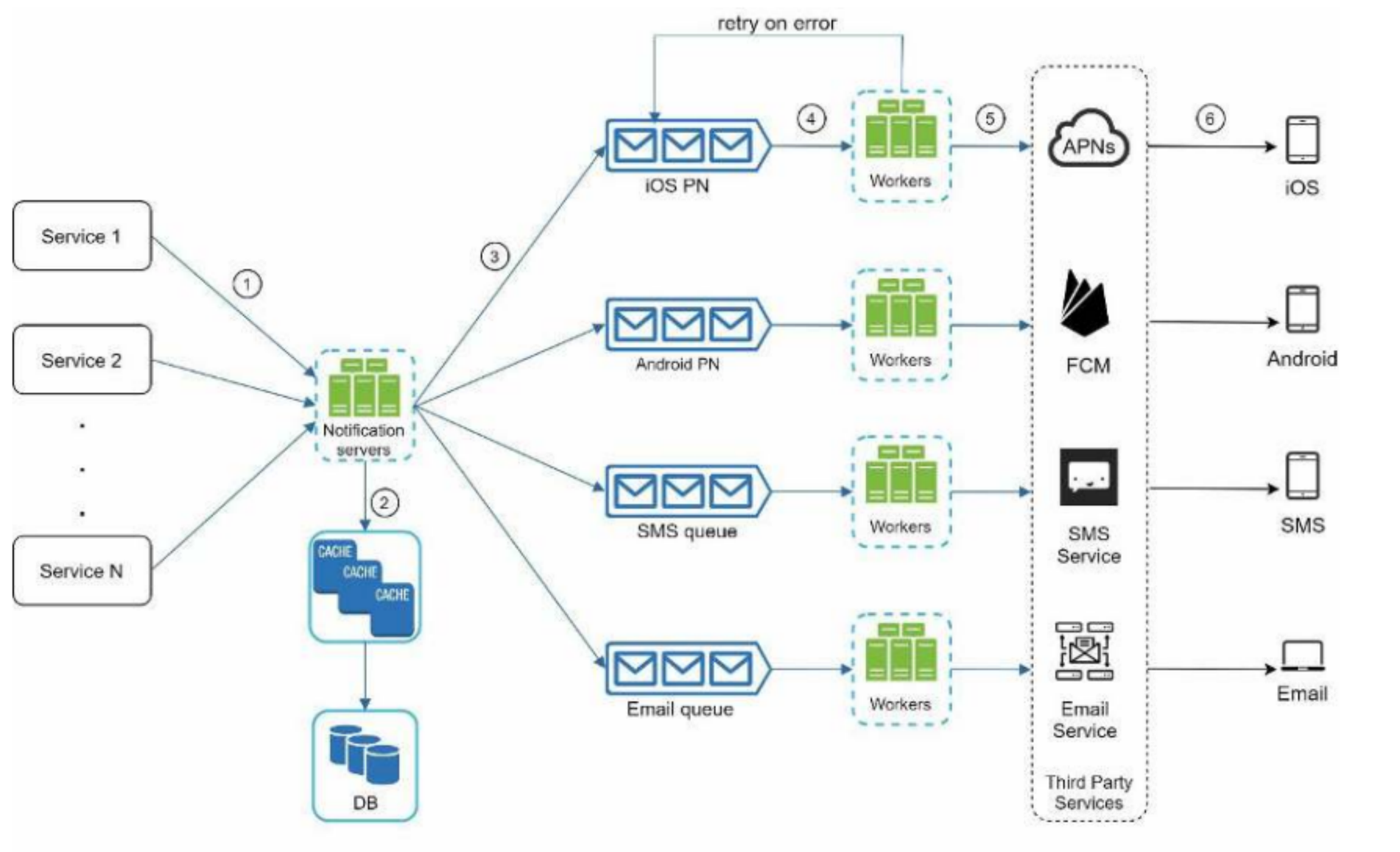


Figure 10-10

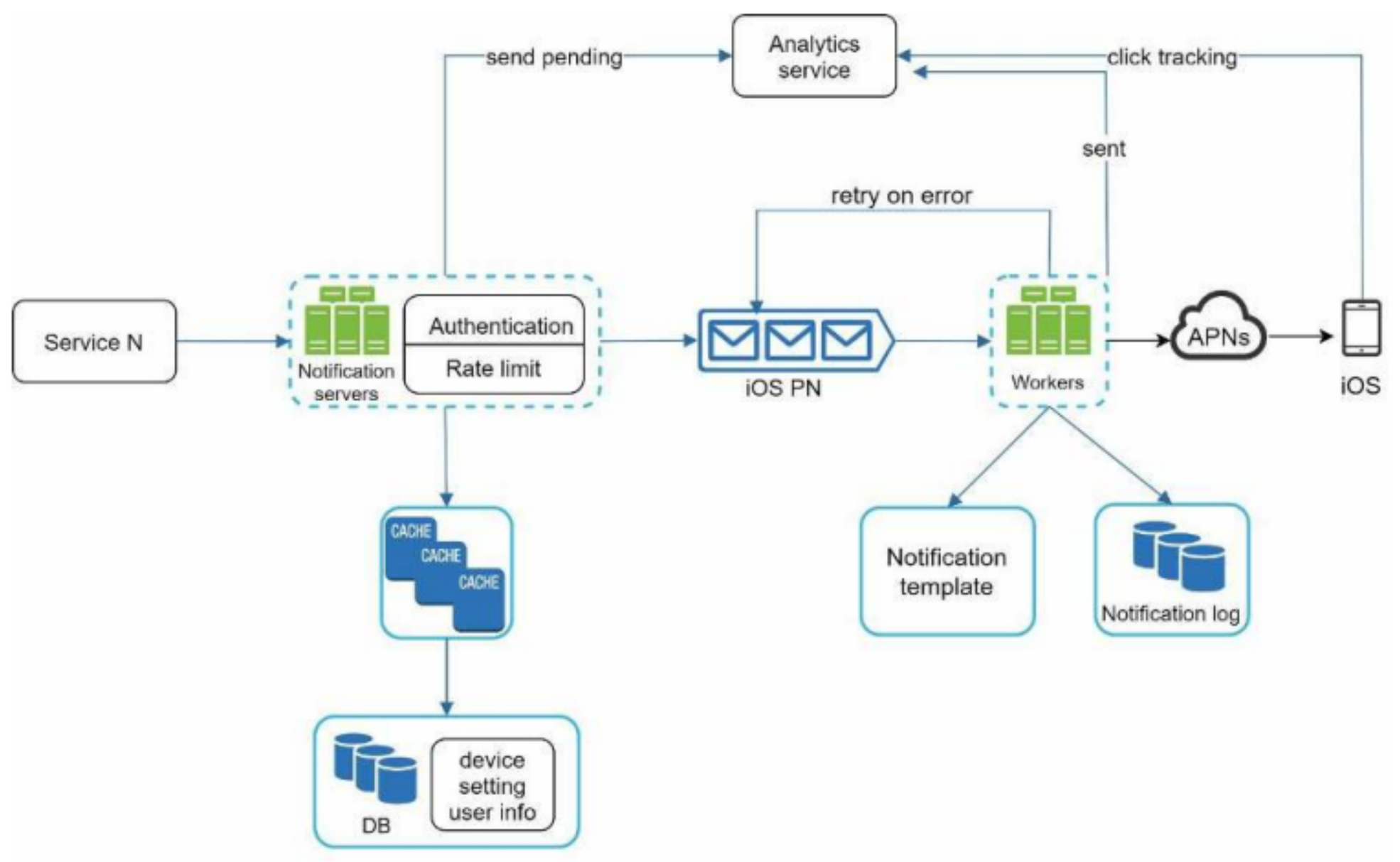


Figure 10-14

Inverted Index LinkedIn Posts

Q. Design a system which takes stream of linkedin posts and form a inverted index of all the words in the post. The user will query a list of words, you have to return all the post ids which has all the given list of words.

Interview script:
"I'll trade strict read-your-writes for high availability. Deletes/visibility changes propagate synchronously to a 'visibility filter' so they take effect on queries immediately, while index updates can be eventually consistent."

Functional:
- Consumes a stream of LinkedIn posts.
- Extracts words from post and Tokenize (remove punctuation, lowercase, etc.).
- Builds and maintains inverted index, e.g. word → [post_id1, post_id2, ...]
- Update inverted index as new posts arrive.
- Supports conjunctive queries: given a list of words, return all post IDs that contain all of them.

Non-Functional:
- Latency Query: <100ms, Ingestion: <1s, so new posts appear in search quickly.
- Throughput: 10K posts/sec, 5K queries/sec — LinkedIn has massive engagement.
- High availability (>99.9%) and fault tolerance, using replication and redundancy.
- Durability: Replication factor ≥ 3; WAL/commit log; S3 snapshots.
- Consistency: Eventually consistent index. Trade-off for availability and speed.
- Fault Tolerance: Survive node failures
- Observability via metrics, logging, and tracing — because if it breaks, we need to know why fast."
- Cost Efficiency: Old data stored in cheaper archive (S3/AWS glacier)

Back-of-the-envelop:
* Assumptions:
- 10K posts per sec (peak) with AVG size: ~50 words
- Avg word length: 6 characters → ~300 bytes per post
- Total words/sec: 10K × 50 = 500K words/sec

- Unique words: Assume 1M distinct words (English + jargon)
- Post lifetime: 1 year → ~315M posts/year
- Query rate: 5K QPS (complex queries)

* Storage Estimation:
- Each word maps to a list of post IDs.
- Assume average 10K posts per word (for popular words it will be higher).
- Each post ID: 8 bytes (long)
- Total storage: 1M words × 10K post IDs × 8 bytes = 80 GB
  But skewed: "hiring" may have 10M entries → hot keys.
  → Estimate: 100–200 GB of index data.

* Throughput:
Need to process 500K word updates/sec.
Query: 5K QPS, each query intersects 2–5 word lists.

API:
- GET /search?q=word1+word2
  Response: { "post_ids": [123, 456, 789], "took_ms": 45 }

*Internal (Admin)
- POST /posts (internal, for ingestion) - ingestion done via stream
  but keep it for testing. Not public.
  Payload: { "post_id": 123, "text": "I am a machine learning engineer..." }

- POST /v1/reindex?from=timestamp — The definition of a "word" might change, or they might want to improve how they're stored. This tells the system, "Go back and rebuild the entire index for all posts from a certain time." (This is a "rolling reindex").

- POST /v1/refresh_visibility — A user might change a post from "Public" to "Connections Only," or delete their account. This forces the search index to immediately double-check with the "visibility service" to ensure hidden posts don't appear in search results..

Interview script
"I'll lean on OpenSearch for the inverted index. It's the right lever for postings and AND queries, leaving us to focus on correctness, visibility, and cost. Kafka + Flink keeps the pipeline exactly-once; Postgres/Redis hold operational metadata."

DB:
* Choice
- Primary (Option A): OpenSearch/Elasticsearch for inverted index. -- we'll choose OpenSearch
  Why: battle-tested full-text engine, shard routing, compression, skip data structures, query DSL, ops tooling.

- Metadata/controls: PostgreSQL (or Aurora) for job state, schema versions.
- Cache: Redis for term DF & small postings caches.
- Streaming: Kafka (ingestion buffer, replay, backfill).
- Object store: S3 for snapshots, cold tiers.

* Logical Tables (field names only)
- Post stream (Kafka value schema)
  post_id, text, lang, author_id, visibility, created_at, updated_at, op (upsert/delete)

- OpenSearch index mapping (document = post)

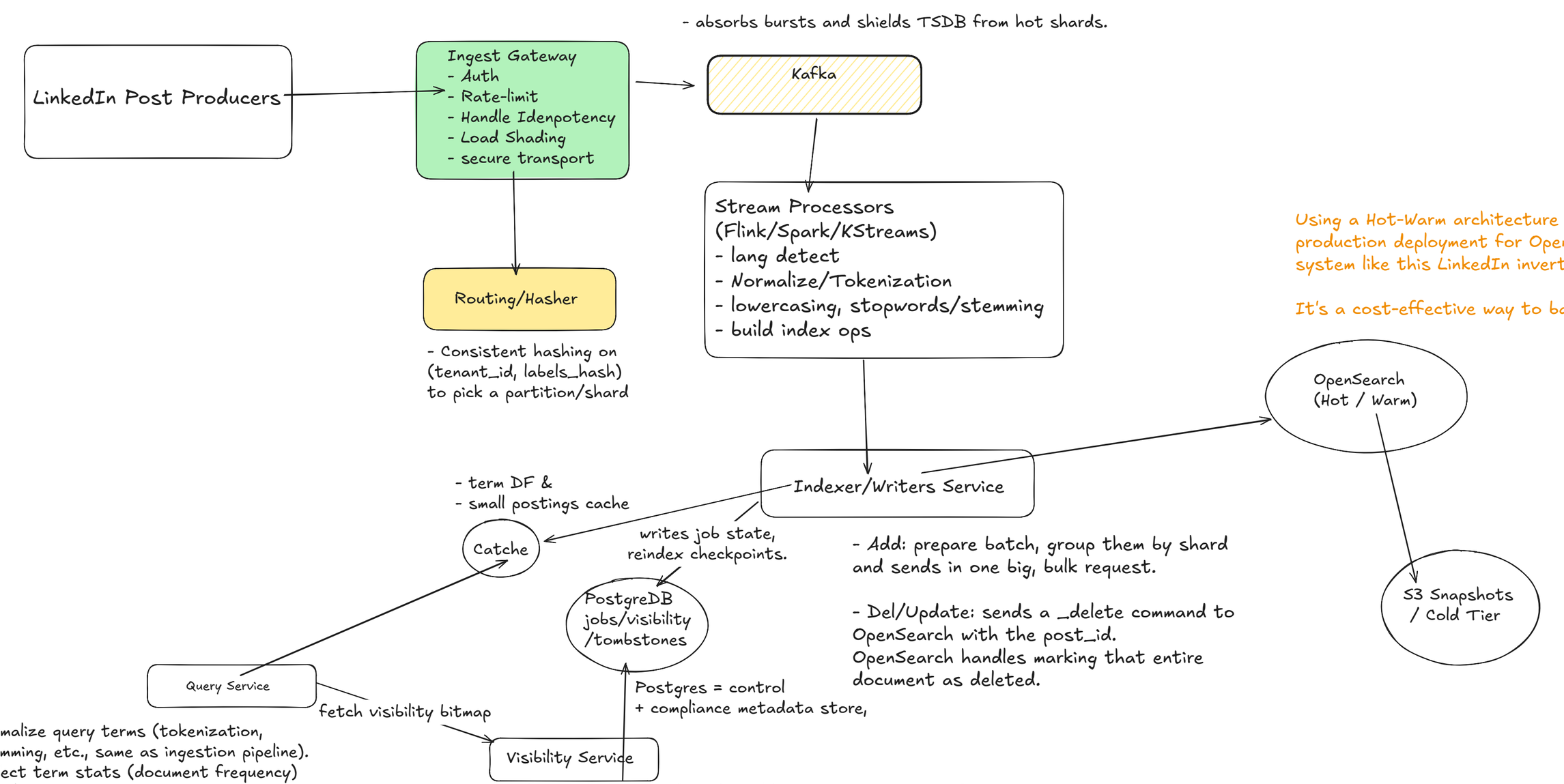
_id = post_id
text: analyzed (custom analyzer per language)
lang: keyword
visibility: keyword
created_at: date

- PostgreSQL (operational metadata)
  reindex_jobs(job_id, status, start_ts, end_ts, from_ts, shard_plan_json)
  -- log of every time an admin triggers a "reindex."

term_hotset(term, df, updated_at)
-- A small table of the most popular search terms (term) and how many posts contain them (df - Document Frequency). This is for monitoring.

delete_tombstones(post_id, ts) (mirror to ensure SLA)
visibility_overrides(post_id, visibility, ts)
-- A guaranteed list of every post that was deleted or had its visibility changed.
Ensures the system never forgets to process a delete, even if something goes wrong temporarily.

Why these choices?
- Search engine handles postings, compression, skip lists, intersection efficiently.
- SQL keeps operator state reliable; Kafka is the durable bus; Redis cuts tail on hot metadata.



- absorbs bursts and shields TSDB from hot shards.

- Consistent hashing on (tenant_id, labels_hash) to pick a partition/shard

- term DF &
- small postings cache

writes job state, reindex checkpoints.

- Add: prepare batch, group them by shard and sends in one big, bulk request.

- Del/Update: sends a _delete command to OpenSearch with the post_id. OpenSearch handles marking that entire document as deleted.

Postgres = control
+ compliance metadata store,

- Normalize query terms (tokenization, stemming, etc., same as ingestion pipeline).
- Collect term stats (document frequency) to plan execution.
- Fan-out sub-queries to the underlying storage (OpenSearch).
- Merge results → intersect postings lists (since requirement is AND semantics).
- Apply filters (visibility, time ranges).
- Return post IDs with pagination.

- Handles authorization (e.g., "should user X see this post?").
- Provides denormalized post view (joins post text, user info, visibility rules).
- Prevents Query Service from directly hitting Postgres.

Using a Hot-Warm architecture is a very common and recommended production deployment for OpenSearch/Elasticsearch, especially for a system like this LinkedIn inverted index.

It's a cost-effective way to balance performance needs with the large volume of data.

Interview-ready script:
"The Query Service is the coordinator. Clients hit this instead of going directly to OpenSearch. It first normalizes the query terms using the same analyzer as ingestion — so query tokens match index tokens. Then it fetches term statistics: for hot terms we use Redis cache; otherwise we call OpenSearch. Based on DF, we plan to intersect the smallest postings list first to minimize work.

For example, if the query is ['search', 'system'], Redis tells us 'system' is rarer than 'search'. We fetch postings for both, then intersect them. At this point we apply a visibility filter — powered by a deny-bitmap service — so posts deleted or private are immediately excluded. Finally, we paginate and return post IDs.

Directly, Query Service talks to:
- Clients for requests/responses
- Redis for DF and hot postings cache
- OpenSearch for main postings lists
- Visibility filter service for access control.

This separation keeps query latency predictable (<100ms P99), while ensuring privacy is never violated and allowing us to scale query and index layers independently."

User Query to Query Service Flow

```
     +-----------------+
     |   User Query     |
     +--------+--------+
              |
              v
     +--------+--------+
     |  Query Service   |
     +----+---+---+-----+
          |   |   |
   +--------+  |  +----------+
   |           |             |
   v           v             v
+-----+  +----------+  +-------------+
|Redis |  |SearchIndex|  |Visibility Svc|
|(hot  |  |(Elastic)  |  | (auth/meta)  |
|cache)|  |           |  |--------------|
+-----+  +----------+  | -> Postgres  |
                       +-------------+

              ^
              |
     +-----------------+
     |    Postgres      |
     | (posts, metadata) |
     +-----------------+
              ^
              |
     +-----------------+
     |   Index Writer   |
     +----+------------+
              ^
              |
     +----+------------+
     |      Kafka       |
     | (stream of posts) |
     +-----------------+
              ^
              |
      +-------+------+
      |  Ingest Svc   |
      | (posts → Kafka|
      +--------------+
```

Resource Allocation/Capacity Management System

Resource Allocation / Capacity Management
└── Amazon Locker — specialization
└── Prime Day Inventory — specialization
└── Capacity-Aware Routing — related specialization

FR:
- Allocate a resource that satisfies CPU, memory, resource-type, region/AZ, and tenant constraints.
- Guarantee exclusivity — no two concurrent requests can claim the same resource.
- Release & reclaim — free resources on request, and reclaim reservations abandoned without provisioning/heartbeat.
- Track lifecycle — every resource moves through Available → Reserved → Provisioning → Allocated → Released, with state transitions observable.

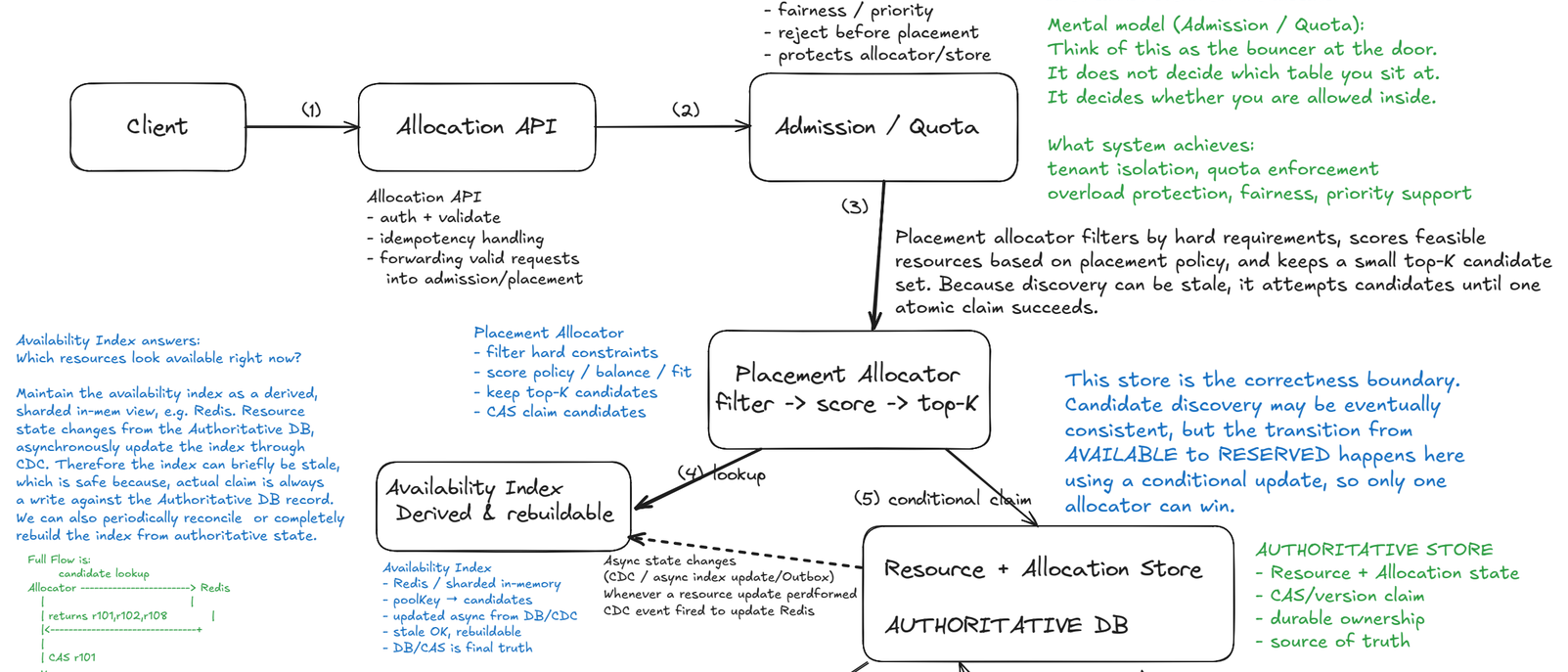
Non-FR
- Handle bursts of concurrent allocation requests/traffic.
- Strong consistency for the final resource claim;
  stale reads are acceptable during candidate discovery.
- Low allocation latency without scanning the full inventory.

BACK-OF-THE-ENVELOPE:
Assume 10M managed resources.
Peak burst: 500K allocation requests / 10 sec
≈ 50K allocation attempts/sec.

If each request examines only ~20 candidates:
50K × 20 ≈ 1M candidate evaluations/sec.

REQUEST/
ADMISSION

DISCOVERY /
PLACEMENT



Admission / Quota
- quota + rate limit
- fairness / priority
- reject before placement
- protects allocator/store

Allocation API
- auth + validate
- idempotency handling
- forwarding valid requests
  into admission/placement

Availability Index answers:
Which resources look available right now?

Maintain the availability index as a derived, sharded in-mem view, e.g. Redis. Resource state changes from the Authoritative DB, asynchronously update the index through CDC. Therefore the index can briefly be stale, which is safe because, actual claim is always a write against the Authoritative DB record. We can also periodically reconcile or completely rebuild the index from authoritative state.

```
Full Flow is:
      candidate lookup
Allocator -----------------------> Redis
  |                                |
  | returns r101,r102,r108          |
  |<------------------------------+
  |
  | CAS r101
```

Placement Allocator
- filter hard constraints
- score policy / balance / fit
- keep top-K candidates
- CAS claim candidates

Availability Index
- Redis / sharded in-memory
- poolKey → candidates
- updated async from DB/CDC
- stale OK, rebuildable
- DB/CAS is final truth

Async state changes
(CDC / async index update/Outbox)
Whenever a resource update perdformed
CDC event fired to update Redis

Placement allocator filters by hard requirements, scores feasible resources based on placement policy, and keeps a small top-K candidate set. Because discovery can be stale, it attempts candidates until one atomic claim succeeds.

Before doing any expensive placement work,
I run admission control. It enforces tenant quotas, rate limits, and priority so one tenant or a traffic burst cannot overwhelm the allocator.

Mental model (Admission / Quota):
Think of this as the bouncer at the door.
It does not decide which table you sit at.
It decides whether you are allowed inside.

What system achieves:
tenant isolation, quota enforcement
overload protection, fairness, priority support

This store is the correctness boundary. Candidate discovery may be eventually consistent, but the transition from AVAILABLE to RESERVED happens here using a conditional update, so only one allocator can win.

AUTHORITATIVE STORE
- Resource + Allocation state
- CAS/version claim
- durable ownership
- source of truth

Mental model (Placement Allocator)
Placement is like booking a seat on a busy flight.
The search system says: Seats 12A, 12B, 14C appear free
But another customer may grab 12A milliseconds before you.
So the allocator keeps several candidates and tries them one by one.

What system achieves:
efficient placement, reduced fragmentation, AZ balancing
scalability, tolerance of stale discovery

FILTER: Discard resources that can't satisfy hard requirements.
Request:
region = us-west-2, cpu >= 8, memory >= 32, type = compute-large

The allocator should only consider resources satisfying all of those.
For example:
r1: west, 16 CPU, 64 GB ✓
r2: west, 4 CPU, 64 GB ✗
r3: west, 8 CPU, 32 GB ✓
Candidates: r1, r3

SCORE: Now multiple candidates working.
We rank/score them based on policy.

Possible scoring factors:

Resource state ~1 KB × 10M
≈ 10 GB logical metadata, easily partitionable.

APIs
POST /allocations - Request capacity matching constraints.
Sample Req: { "tenantId": "tenant-42",
"resourceType": "compute",
"cpu": 8,
"memoryGb": 32,
"region": "us-west-2",
"preferredAzs": ["us-west-2a", "us-west-2b"] }

Resp: { "allocationId": "alloc-781", "resourceId": "host-5512",
"state": "RESERVED", "reservationExpiresAt": "2026-09-05T14:20:00Z" }

GET /allocations/{allocationId} - Read reservation/provisioning status.
DELETE /allocations/{allocationId} - Release/cancel allocated capacity.

DB:
RESOURCE
Purpose: Authoritative record for each allocatable resource and its current ownership/capacity state.

- resourceId — unique ID of the machine/resource being allocated.
- poolKey — identifies the placement pool this resource belongs to, such as us-west-2 / AZ-1 / GPU; used to quickly narrow candidate searches instead of scanning all resources.
- cpuTotal — total CPU capacity this resource can provide.
- memoryTotal — total memory capacity this resource can provide.
- state — tells whether this resource can currently be selected: AVAILABLE, RESERVED, ALLOCATED, etc.
- version — increments whenever the resource state changes; an allocator updates the resource only if the version it read is still current, preventing two allocators from successfully claiming the same resource.

ALLOCATION
Purpose: Tracks one customer's allocation request from reservation through provisioning/release.

- allocationId — unique ID for this allocation request; also lets retries refer to the same allocation.
- tenantId — customer that owns the allocation; used for quotas, priority, and fairness.
- resourceId — resource successfully selected/reserved for this allocation.
- state — current lifecycle stage, such as RESERVED, PROVISIONING, ACTIVE, FAILED, or RELEASED.
- expiresAt — time after which an unfinished reservation can be reclaimed so capacity is not leaked forever.
- requestHash — fingerprint of the important request parameters; if the same request ID is retried with different parameters, we can detect that it is not the same logical request.

CAPACITY_POOL
Purpose: Small summary of capacity for a placement pool, used to decide which pools are worth searching before looking at individual resources.

- poolKey — identifies a pool such as region + AZ + resource class.
- availableCount — approximate number of currently available resources in that pool; helps skip pools that are likely full.
- capacityClass — normalized/bucketed capacity characteristics used to match requests to suitable pools.
- updatedAt — when this summary was last refreshed; lets the allocator judge whether the approximate capacity information may be stale.

AVAILABILITY_INDEX
Purpose: Derived lookup structure containing currently usable resource candidates so allocation does not scan the authoritative RESOURCE table.

- poolKey — pool under which this resource is searchable.
- resourceId — candidate resource that may satisfy an allocation request.
- freeCpu — CPU currently believed to be available on this candidate.
- freeMemory — memory currently believed to be available on this candidate.
- version — RESOURCE version from when this index entry was created; helps detect that the index entry is stale relative to authoritative resource state.

Authoritative DB
| AVAILABLE → RESERVED
+------ change event / CDC ------+
v
Index Updater
v
Redis
remove r101

PROVISION / RECOVERY

CORRECTNESS
- candidate index may be stale
- AVAILABLE -> RESERVED via version / CAS
- completion / expiry check allocationId + state

The full request flow in simple language

You can memorize the whole architecture like this:

1 Client asks for capacity.

2 API validates and deduplicates the request.

3 Admission decides whether the tenant is allowed to allocate.

4 Allocator asks the availability index for suitable candidates.

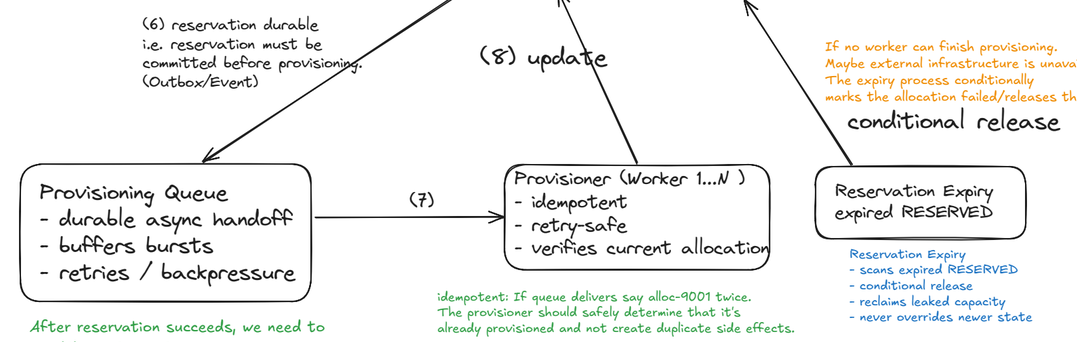
5 Allocator atomically claims one candidate in the authoritative DB.

6 Once reservation is durably stored, enqueue provisioning work.

7 Provisioner consumes the task and configures the resource.

8 Provisioner conditionally marks the allocation ALLOCATED.

If provisioning never completes:
Reservation Expiry conditionally releases the resource.



If no worker can finish provisioning.
Maybe external infrastructure is unavailable.
The expiry process conditionally marks the allocation failed/releases the resource.

After reservation succeeds, we need to provision resources.
Those operations may take seconds or minutes. Provisioning of resources is done by 'Provisioner' and Provisioning Queue makes the process asynch, and absorbs traffic burst that Provisioner might not be able to handle.
So:
durable reservation
↓
Provisioning Queue
↓
Provisioner

idempotent: If queue delivers say alloc-9001 twice.
The provisioner should safely determine that it's already provisioned and not create duplicate side effects.

A Worker must verify DB state first. Before doing provisioning, Worker should not blindly trust the queue message. if in DB state != RESERVED/PROVISIONING, then ignore message

WORKER CRASH and Reallocation Example:
10:00:00 Queue sends alloc-9001 → Worker 17
10:00:05 Worker configures network ✓
10:00:10 Worker attaches disk ✓
10:00:12 Worker crashes ✗

No ACK

10:01:00 queue visibility timeout expires
10:01:01 Queue sends alloc-9001 → Worker 24

Here, Idempotency comes to play after Worker 24 picks the job.

Worker 24 will not execute every task from start instead it will check if the resource alraeady allocated and if it has, it SKIPS that task and moves to next.

EXAMPLE: Suppose provisioning a compute resource means:

desired state:
network = configured
disk = attached
image = installed
health = healthy

Worker 1 starts:
network ✓
disk ✓
then crashes.

Worker 2 receives the same job.
It checks current state:

network already configured → skip
disk already attached → skip
image missing → install
health not ready → configure/check

Eventually:
resource = fully provisioned

Then Worker 2 updates the authoritative DB:

alloc-9001:
PROVISIONING → ALLOCATED

and ACKs the queue message.

Reservation Expiry
- scans expired RESERVED
- conditional release
- reclaims leaked capacity
- never overrides newer state

Reservations have a TTL. A background expiry process reclaims reservations that never complete, but release is conditional on the allocation still being RESERVED so a stale timeout cannot undo a successful allocation.

I persist the reservation deadline as expiresAt on the authoritative allocation record. The expiry service does not scan the whole table; it uses an index on state and expiration time, or time-bucketed expiry partitions at larger scale, to fetch only due reservations. For every candidate it performs a conditional state transition before releasing the resource, because the reservation may have completed after the expiry worker read it."

fit efficiency, AZ balance, cost, latency etc

Example: r1 = 16 CPU, 64 GB; r4 = 8 CPU, 32 GB
If the request needs exactly: 8 CPU / 32 GB
r4 may score higher because it avoids wasting a larger machine.

TOP-K: Do not return one candidate only.
Return, say: r4, r8, r17, r21, r9

Why? Because the index may be stale. By the time we try r4, another allocator may have claimed it. Then we immediately try r8.

This avoids performing a brand-new expensive search after every conflict.

Mental model (Availability Index, Derived & rebuildable)
Think of Zillow listings.
A listing may say: House available
but somebody could just have bought it.

The listing helps you find candidates.
The official property transaction determines ownership.

What system achieves:
no full-table scans, fast candidate discovery,
scalable reads, flexible search dimensions

The Availability Index stores a fast-searchable, derived view of resources that appear available so Placement Allocator does not query/scan the authoritative Resource DB for every allocation request.

A good mental model is:

Authoritative DB = truth
Availability Index = fast lookup/cache of candidates
Where can the Availability Index live?

For this design, Redis is a very reasonable choice, especially in a 30-minute interview.

For example, partition it by poolKey:

poolKey = us-west-2#az-a#compute-large

↓

Redis
----
r101 → cpu=8, memory=32, version=17
r102 → cpu=16, memory=64, version=12
r103 → cpu=8, memory=64, version=21

Then the allocator can quickly ask:

Give me candidates from:
us-west-2 / AZ-A / compute-large

instead of querying millions of Resource records.

Depending on the exact search requirements, alternatives could be DynamoDB secondary indexes, OpenSearch, or another purpose-built in-memory/index structure. For the interview, Redis/sharded in-memory index is easy to explain.

But how does Redis know which resources are available?

This is the more important part.
I would not make a periodic full DB-scanning job the primary mechanism.

Instead, update the Availability Index asynchronously whenever authoritative Resource state changes.

Think:
authoritative change
|
v
Resource DB ----> change event / CDC ----> Index Updater ----> Redis

For example, initially:

RESOURCE DB
r101
state = AVAILABLE
cpu = 8
memory = 32
version = 17

The index updater gets that change and adds:
Redis:
poolKey → r101
Now r101 becomes discoverable.
When the resource is reserved
Allocator performs the authoritative CAS:

r101:
AVAILABLE/v17
↓
RESERVED/v18

The database is updated first.

Then that change eventually reaches the index updater:

DB change:
r101 RESERVED
↓
Index Updater
↓
remove r101 from Redis availability index

So eventually:

Redis no longer contains r101

There can be a short period where Redis still says r101 is available.

That's exactly why we say:

The availability index may be stale.

What happens during that stale window?

Suppose:

DB:
r101 = RESERVED/v18

Redis:
r101 = AVAILABLE/v17 ← hasn't caught up yet

Another allocator searches Redis.

Redis returns:

r101

That's fine.

The allocator then goes to the authoritative DB:

claim r101
if state == AVAILABLE
and version == 17

But the DB contains:

RESERVED/v18

So CAS fails.

Allocator simply tries another top-K candidate:

r102

This is why the design works even when Redis is eventually consistent.

What produces these change events?
Below are a few reasonable implementations.

Option 1 — CDC

Resource DB
|
DynamoDB Streams / DB CDC
|
v
Index Updater
|
v
Redis

Or:

Option 2 — Transactional Outbox

Resource DB transaction
├── update Resource
└── write Outbox event
|
v
Event Publisher
|
v
Index Updater
|
v
Redis

Amazon Locker - Resource Allocation/Capacity Management System

# Functional Requirements:
Core (must-have):

- Find nearby locker with a slot that fits the pkg
- Reserve exactly one slot
  — no double-booking under concurrent requests
- Keep reservation/package state in sync with lockers that connect intermittently
- Notify customer when package is ready, and free the slot after pickup or expiry

Follow-up (secondary, good to mention if asked):
- Handle locker maintenance / disabled slots / temporarily unavailable lockers
- Reroute to a different locker if delivery or provisioning fails

Non-FR:
- Strong correctness for slot ownership; never assign the same slot to two active pkg.
- High availability of cloud allocation despite edge-device disconnects.
- Low-latency locker/slot discovery across a large fleet.

BACK-OF-THE-ENVELOP
100K lockers
100 slots / locker ≈ 10M physical slots

10M package-placement requests/day
≈ 115 requests/sec average
Peak 20x during promotions:
≈ 2–3K placement requests/sec

Locker heartbeat every 30 sec:
100K / 30 ≈ 3.3K heartbeat messages/sec

APIs
POST /pkg-reservations - Find and reserve a locker slot for a package.

Req: { "packageId": "pkg-1001", "customerId": "cust-42",
"packageSize": "MEDIUM",
"destination": {"latitude": 37.7749, "longitude": -122.4194 },
"maxDistanceKm": 5}
Resp: { "reservationId": "res-901", "lockerId": "locker-201",
"slotId": "slot-17", "state": "RESERVED",
"expiresAt": "2026-09-07T18:00:00Z" }

GET /pkg-reservations/{id}
- Read reservation/delivery/pickup state.
POST /locker-events
- Locker reports door, slot, heartbeat, pickup events.
POST /pkg-reservations/{id}/cancel
- Cancel or release unused reservation.

DB
LOCKER (lockerId, location, connectivityState, lastHeartbeatAt, operationalState)
connectivityState — ONLINE, DEGRADED, OFFLINE based on heartbeat.
operationalState — ACTIVE, MAINTENANCE, DISABLED.

SLOT (slotId, lockerId, sizeClass, state, reservationId, version)
sizeClass — SMALL, MEDIUM, LARGE, etc.
state — AVAILABLE, RESERVED, OCCUPIED, DISABLED.

PACKAGE_RESERVATION (reservationId, packageId, lockerId, slotId, state, expiresAt, notificationState)

LOCKER_COMMAND (commandId, lockerId, reservationId, commandType, state, createdAt)

commandType — PREPARE_SLOT, OPEN_DOOR, RELEASE_SLOT, etc.
state — PENDING, ACKED, FAILED.

Find nearby available locker for MEDIUM/LARGE/SMALL slot.
A Geo+Capacity-indexed data structure (e.g., DynamoDB with geohash buckets, or an OpenSearch geo index) can be used

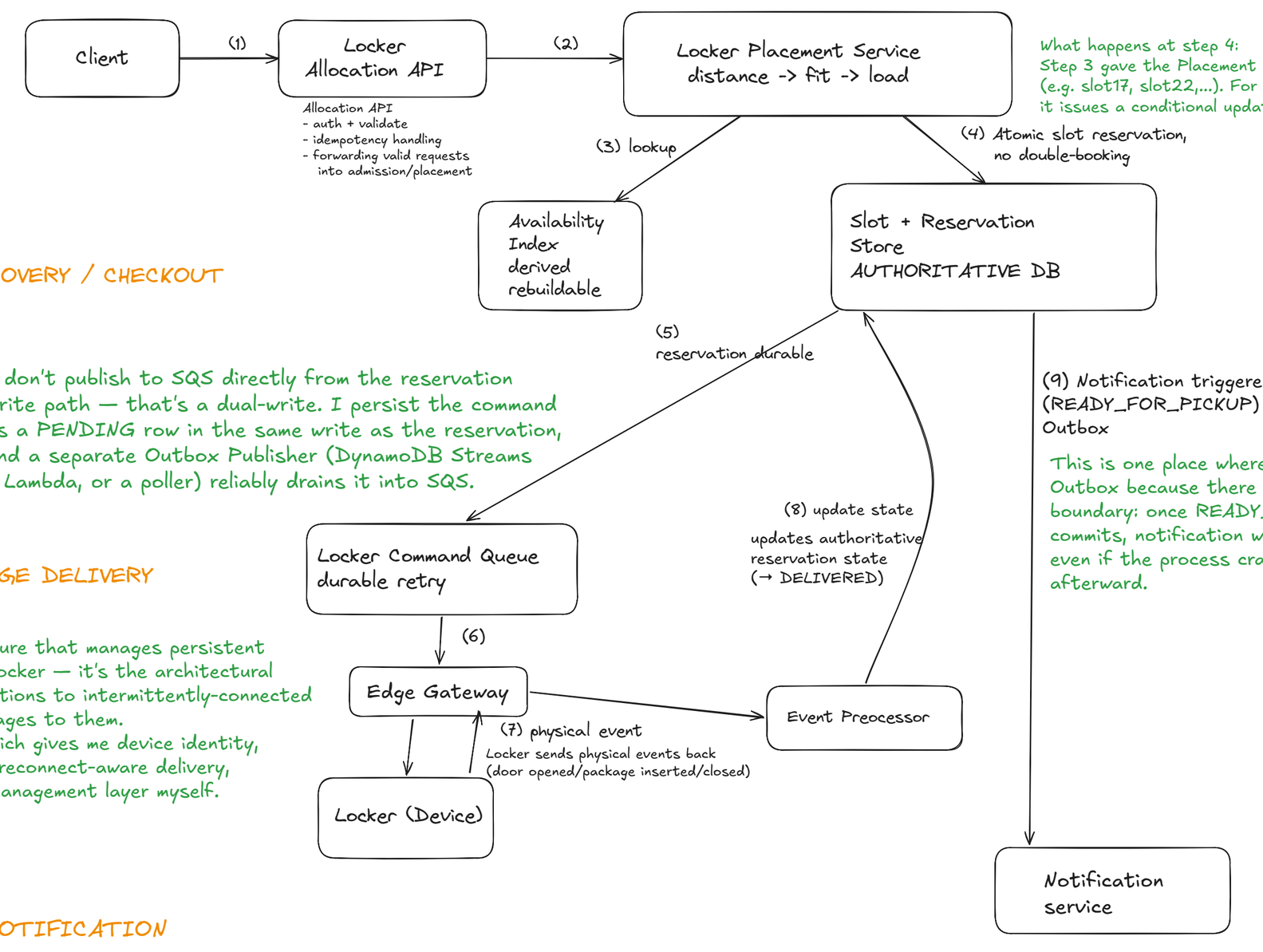
Instead of scanning 10M physical slots across 100K lockers, the placement service queries this index and gets back a short candidate list — maybe 10–20 lockers.

CUSTOMER / DELIVERY FLOW

DISCOVERY / CHECKOUT

EDGE DELIVERY

EVENT/NOTIFICATION


Client
(1)
Locker Allocation API
(2)
Locker Placement Service
distance -> fit -> load
(3) lookup
(4) Atomic slot reservation, no double-booking
Availability Index derived rebuildable
Slot + Reservation Store AUTHORITATIVE DB
(5) reservation durable
(8) update state
updates authoritative reservation state (→ DELIVERED)
(9) Notification triggered (READY_FOR_PICKUP) and Outbox
Locker Command Queue durable retry
(6)
Edge Gateway
(7) physical event
Locker sends physical events back (door opened/package inserted/closed)
Event Preocessor
Locker (Device)
Notification service

Allocation API
- auth + validate
- idempotency handling
- forwarding valid requests into admission/placement

What happens at step 4:
Step 3 gave the Placement Service Top-K slots (e.g. slot17, slot22,...). For each candidate, in order, it issues a conditional update directly against the DB.

I don't publish to SQS directly from the reservation write path — that's a dual-write. I persist the command as a PENDING row in the same write as the reservation, and a separate Outbox Publisher (DynamoDB Streams + Lambda, or a poller) reliably drains it into SQS.

This is one place where I would use an Outbox because there is a real dual-write boundary: once READY_FOR_PICKUP commits, notification work must be emitted even if the process crashes immediately afterward.

EDGE GATEWAY - piece of infrastructure that manages persistent connections to thousands of physical locker — it's the architectural role of managing live, persistent connections to intermittently-connected devices and routing durable cloud messages to them.
I'd implement it with AWS IoT Core, which gives me device identity, MQTT-based persistent sessions, and reconnect-aware delivery, rather than building that connection-management layer myself.

Correctness:
- Slot checkout: AVAILABLE + expected version
- Edge commands: idempotent commandId
- Notification retry does not change reservation state

HAPPY PATH — Amazon Locker Fleet Mgmt

1. Delivery system requests placement (package + destination)
2. API converts request → slot-size + location constraints
3. Placement service queries derived availability index → gets candidate lockers
4. Optimistic CAS reserves ONE slot (AVAILABLE→RESERVED, version-checked)
5. Idempotent command queued to locker (durable, retried if offline)
6. Edge gateway delivers command when locker reconnects
7. Locker sends physical events back (door opened/package inserted/closed)
8. Event processor updates authoritative reservation state (→ DELIVERED)
9. Notification triggered (READY_FOR_PICKUP) — retried independently if it fails

KEY RULE: Steps 1-4 = logical lock (cloud authoritative).
Steps 5-7 = physical confirmation (locker authoritative).
Steps 8-9 = sync + notify (never rolls back physical truth).

COMPONENT → TECHNOLOGY MAP — Amazon Locker Fleet Mgmt

[ALLOCATION]
Locker Allocation API → Java on ECS/EKS → stateless
Accepts placement requests, entry point to control plane

[DISCOVERY]
Locker Placement Service → Java service → reads eventual, writes strong
Filters + scores candidates, drives top-K CAS retry

Availability Index → DynamoDB+geohash / OpenSearch geo → eventually consistent
Fast nearby-locker lookup, rebuildable, never grants ownership

[CHECKOUT — CORRECTNESS BOUNDARY]
Slot + Reservation Store → DynamoDB → STRONGLY consistent (CAS)
Only place slot ownership legally changes (state+version conditional write)

[OUTBOX]
Command Publisher → DynamoDB Streams + Lambda → at-least-once
Drains PENDING LOCKER_COMMAND rows → queue (avoids dual-write)

[EDGE DELIVERY]
Locker Command Queue → SQS → durable, at-least-once
Buffers commands while locker is offline

Edge Gateway → AWS IoT Core (MQTT) → at-least-once, reconnect-aware
Persistent device identity + live session, routes cmd to right locker

[EVENT SYNC]
Event Processor → ECS/EKS consumer → idempotent conditional updates
Dedupes device events, reconciles physical truth into authoritative state

[NOTIFY — DECOUPLED]
Notification Queue → SQS → → at-least-once
Isolates notification failures from delivery state

Notification Service → SNS / Pinpoint → best-effort, independent retry
Push → email → SMS fallback/escalation

KEY RULE: Only Slot+Reservation Store needs STRONG consistency.
Everything else (index, queues, gateway, notify) = eventually consistent + independently retryable. No component's staleness or outage can corrupt slot ownership.

Amazon Prime Day Inventory/ Flash Sale - Resource Allocation/Capacity Management System

FR:
- Read approx. product availability (fast, can be stale)
- Reserve inventory atomically during checkout (NO oversell)
- Confirm reservation → permanent sale (idempotent)
- Release inventory on payment failure / abandonment / expiry

FOLLOW-UP/Out Of Scope
- Fair handling of extreme flash-sale contention
- Inventory reconciliation with fulfillment systems

Non-FR:
- Strong consistency for inventory ownership-changing ops (reserve / confirm / release — never read-then-write)

- Handle extreme bursts concentrated on a very small number of hot SKUs (skew, not just total QPS)

- Keep read/checkout latency low while protecting the inventory authority from overload

ONE-LINER:
"Strong consistency where ownership changes hands; everything else (availability reads) can be cheap, cached, and eventually consistent — the system is built to survive skew, not just scale."

Back-Of-the-envelop
100M daily product-detail views

Normal checkout:
~10K inventory reservations/sec

Flash sale:
1M buyers target one SKU in 10 sec
≈ 100K reservation attempts/sec for ONE SKU

Suppose SKU has only 50K units:
most requests are guaranteed to lose.

# APIs
GET /inventory/{sku}
→ approximate sellable availability (cached/derived)

POST /inventory/reservations
→ atomically reserve qty for checkout (idempotent via requestId)

Req: { "requestId": "checkout-7821", "customerId": "cust-42",
"sku": "PS5-BUNDLE", "quantity": 1, "fulfillmentRegion": "us-west"}

Resp: {"reservationId": "res-901", "sku": "PS5-BUNDLE", "qty": 1,
"state": "HELD", "expiresAt": "2026-09-06T21:05:00Z"}

POST /inventory/reservations/{id}/confirm
→ HELD -> CONFIRMED (payment succeeded)

DELETE /inventory/reservations/{id}
→ HELD -> RELEASED (payment failed / cancelled)

DB
INVENTORY
- inventoryKey PK (sku + fulfillmentPool [+ shard])
- sellableQty physical qty eligible for sale
- reservedQty held by active checkouts
- committedQty committed to confirmed orders
- version optimistic concurrency token (if using CAS)

(alt minimal form: availableQty + reservedQty, with invariant-preserving transitions)

RESERVATION
- reservationId PK stable hold identity
- requestId UNIQUE client idempotency key
- inventoryKey FK sku/pool/shard reserved from
- quantity
- state HELD | CONFIRMED | RELEASED | EXPIRED
- expiresAt

ORDER
- orderId PK
- reservationId inventory hold backing this order
- customerId
- state PENDING_PAYMENT | CONFIRMED | CANCELLED
- paymentId

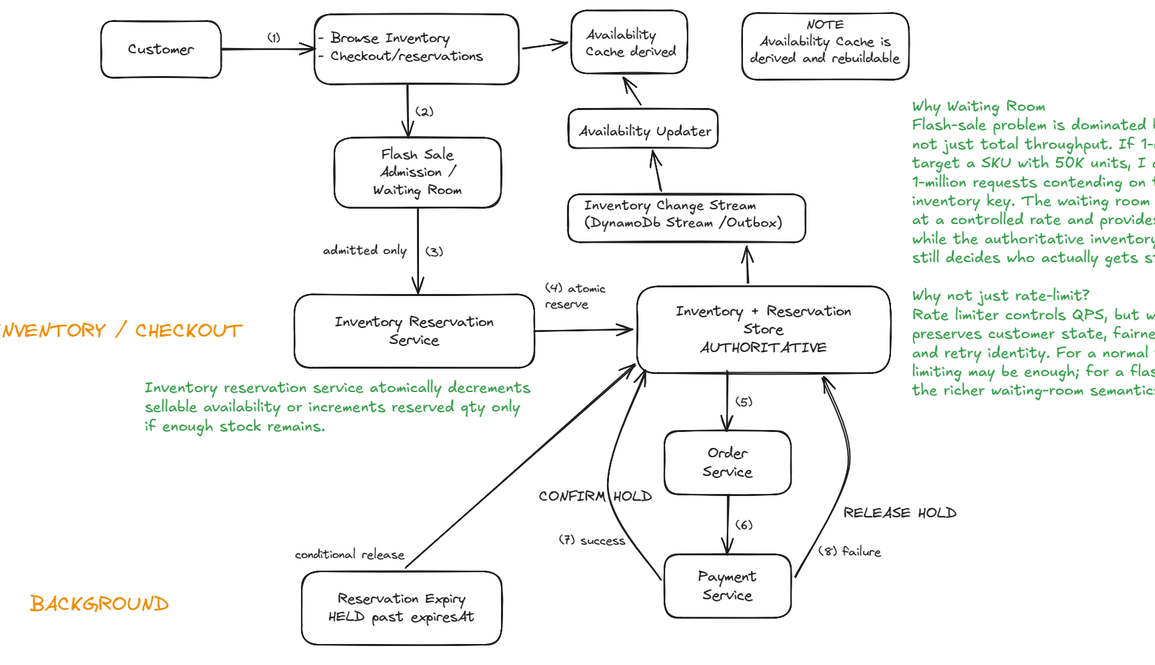
INVENTORY_EVENT / OUTBOX (only if commit must reliably notify downstream)
- eventId PK unique publication identity
- reservationId
- eventType RESERVED | CONFIRMED | RELEASED
- published bool

BROWSE / ADMISSION

INVENTORY / CHECKOUT

BACKGROUND

serve availability from a derived cache because exact consistency is unnecessary until checkout.


Customer
(1)
- Browse Inventory
- Checkout/reservations
Availability Cache derived
NOTE
Availability Cache is derived and rebuildable
Availability Updater
Inventory Change Stream (DynamoDb Stream /Outbox)
(2)
Flash Sale Admission / Waiting Room
admitted only
(3)
Inventory Reservation Service
(4) atomic reserve
Inventory + Reservation Store AUTHORITATIVE
(5)
Order Service
(6)
Payment Service
CONFIRM HOLD
(7) success
RELEASE HOLD
(8) failure
conditional release
Reservation Expiry HELD past expiresAt

Inventory reservation service atomically decrements sellable availability or increments reserved qty only if enough stock remains.

Why Waiting Room
Flash-sale problem is dominated by skew, not just total throughput. If 1-million buyers target a SKU with 50K units, I don't want all 1-million requests contending on the same inventory key. The waiting room admits requests at a controlled rate and provides fairness, while the authoritative inventory reservation still decides who actually gets stock.

Why not just rate-limit?
Rate limiter controls QPS, but waiting room also preserves customer state, fairness, admission order, and retry identity. For a normal traffic spike rate limiting may be enough; for a flash sale I'd use the richer waiting-room semantics.

CORRECTNESS
- reserve only if availableQty >= requestedQty
- requestId / reservationId idempotency
- expiry / confirm operate only on HELD reservation

HAPPY-PATH WALKTHROUGH
1. Browse → Checkout API reads Availability Cache (stale OK)
2. Checkout attempt → enters Flash Sale Admission / Waiting Room
3. Admitted request → reaches Inventory Reservation Service
4. Atomic hold → Inventory Store (AUTHORITATIVE) — reserve
5. Hold → Order → Order Service creates order against reservation
6. Order → Payment → Payment Service processes payment
7. Payment success → HELD -> CONFIRMED
8. Payment failure → HELD -> RELEASED (+ restore qty)

PHASES:
1-2 cheap / stale-tolerant (cache + admission gate)
3-4 strict / strongly consistent (the ONE hold moment)
5-8 slow / external (order + payment, OUTSIDE inventory txn)

ONE-LINER:
"Secure the unit fast (steps 3-4), then do the slow stuff (payment) only after it's already safe — never hold inventory open while waiting on an external gateway."

PREVENT OVERSELLING
SETUP: availableQty = 1, two buyers arrive simultaneously

WRONG (read-then-write / TOCTOU race):
read availableQty
if > 0: write availableQty - 1
→ both can read 1, both decrement → oversell / negative qty

RIGHT (atomic conditional mutation):
UPDATE Inventory
SET availableQty = availableQty - 1
WHERE inventoryKey = ?
AND availableQty >= 1;

DynamoDB:
UpdateItem SET availableQty = availableQty - :q
Condition: availableQty >= :q
→ fails with ConditionalCheckFailedException if not enough stock

WHY IT WORKS:
check + mutation = ONE atomic op in the datastore.
No gap for two requests to both "pass." Exactly one concurrent update succeeds; all others' condition fails and are rejected outright.

DOES NOT SOLVE:
hot-key throughput under massive contention (→ §14/§16:
admission + sharding — a separate problem)

ONE-LINER:
"Never read-then-write for inventory. The decrement itself is the check — atomic in the store, not split across two application-level round-trips."

HOT SKU STRATEGIES
OPTION 1 — ADMISSION TOKENS (§15)
Problem: 1M buyers, 50K units → don't let all 1M hit inventory directly
Fix: Issue ~50K (+ small margin) admission tokens via waiting room; only token-holders may attempt reservation
Key rule: Token = permission to ATTEMPT, not ownership.
Real correctness check still happens at the authoritative inventory decrement.

OPTION 2 — SHARD INVENTORY (§16)
Problem: One inventory counter/partition becomes a hot key even with correct atomic decrement
Fix: Split stock into N shards
(e.g. PS5#0..PS5#9, 5K units each)
Requests hash/spread across shards.
Global availability = sum(shard qtys)
Trade-off: Uneven depletion — one shard may hit zero while others still have stock. On failure, retry a different shard.

RELATIONSHIP:
Admission → controls HOW MANY requests reach inventory
Sharding → controls HOW FAST inventory can absorb the requests that do arrive
Different problems. Use BOTH at extreme scale.

MENTAL MODEL:
"Bouncer at the door (admission) + ten registers instead of one (sharding). Bouncer thins the crowd; multiple registers process it faster. Neither alone survives a true Prime Day spike."

Design Youtube

Functional
- Upload video
- watch video

Below the line (out of scope)
view information about a video, such as view counts.
search for videos.
comment on videos.
see recommended videos.
make a channel and manage their channel.
subscribe to channels.

Non-Functional
- Availability > Consistency
- Low latency streaming, even in low bandwidth environments.
- support uploading, streaming large videos (10s of GBs).
- should scale to a high number of videos uploaded, watched per day
- Should support resumable uploads

Back-of-the-envelop
100M daily viewers × 10 plays/day
≈ 1B plays/day
≈ 12K average playback starts/sec
≈ 50K–100K/sec at peak

5M uploads/day × 200 MB average
≈ 1 PB/day raw uploaded video

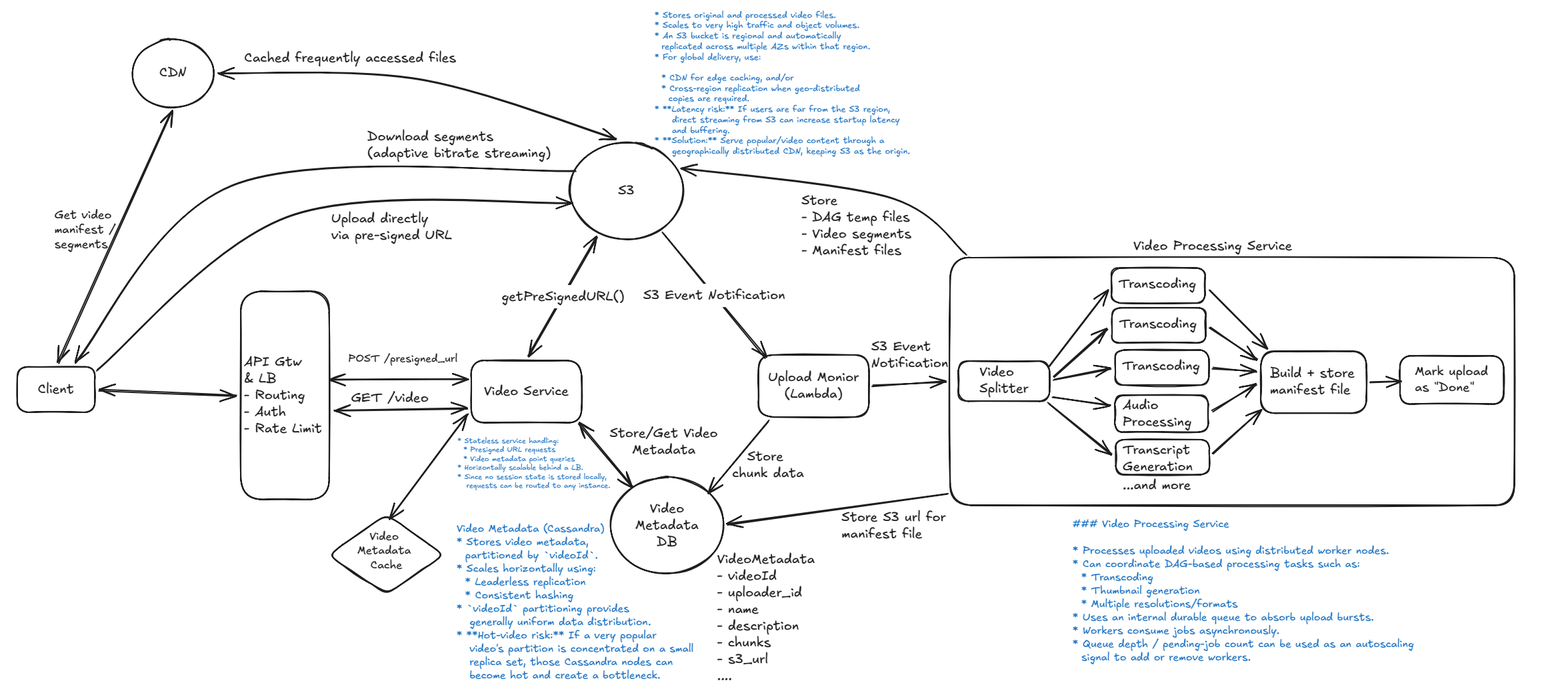
With multiple encoded renditions:
≈ 2–3 PB/day of generated video objects

DB
User: A user of the system (uploader or viewer).
Video: A video that is uploaded / watched.
VideoMetadata: This is metadata associated with the video, such as the uploading user, URL reference to a transcript, etc.
- videoId, uploader_Id, name, desc, chunks, s3_url

API
- To upload a video, server will create a pre-signed URL to enable the client to upload directly to S3. The request payload will just be the video metadata.
POST /presigned_url
Request:
{Video, VideoMetadata}

- POST /presigned_url
Request:
{ Video, VideoMetadata }

- To stream a video, retrieve the video data to play it on device:
GET /videos/{videoId} -> Resp: {VideoMetadata}
VideoMetadata, will contain URL(s) necessary to watch video.



S3 — Video/Object Storage

* Stores original and processed video files.
* Scales to very high traffic and object volumes.
* An S3 bucket is regional and automatically replicated across multiple AZs within that region.
* For global delivery, use:

  * CDN for edge caching, and/or
  * Cross-region replication when geo-distributed copies are required.
* **Latency risk:** If users are far from the S3 region, direct streaming from S3 can increase startup latency and buffering.
* **Solution:** Serve popular/video content through a geographically distributed CDN, keeping S3 as the origin.

* Stateless service handling:
  * Presigned URL requests
  * Video metadata point queries
* Horizontally scalable behind a LB.
* Since no session state is stored locally, requests can be routed to any instance.

Video Metadata (Cassandra)
* Stores video metadata, partitioned by `videoId`.
* Scales horizontally using:
  * Leaderless replication
  * Consistent hashing
* `videoId` partitioning provides generally uniform data distribution.
* **Hot-video risk:** If a very popular video's partition is concentrated on a small replica set, those Cassandra nodes can become hot and create a bottleneck.

VideoMetadata
- videoId
- uploader_id
- name
- description
- chunks
- s3_url
....

Video Processing Service

* Processes uploaded videos using distributed worker nodes.
* Can coordinate DAG-based processing tasks such as:
  * Transcoding
  * Thumbnail generation
  * Multiple resolutions/formats
* Uses an internal durable queue to absorb upload bursts.
* Workers consume jobs asynchronously.
* Queue depth / pending-job count can be used as an autoscaling signal to add or remove workers.

Handling Hot / Viral Videos

A viral video creates an extreme **read hotspot**:
it may be uploaded once but viewed millions of times.

To handle this:

* **Cache metadata aggressively**

  * Use a distributed cache such as **Redis / Memcached**.
  * Key by `videoId`.
  * Cache hot video metadata to avoid repeatedly hitting Cassandra.
  * Use TTL/LRU-style eviction.

* **Leverage Cassandra replication**

  * Cassandra already replicates each partition across multiple nodes.
  * Tune the **replication factor and read consistency** so reads can be served from multiple replicas.
  * This reduces pressure on a single replica set, although extremely hot keys can still create hotspots.

* **Use CDN for actual video content**

  * Store video files in S3, but serve them through a CDN such as **CloudFront / Akamai**.
  * Viral videos are cached at edge locations close to users.
  * This prevents repeated requests from reaching S3 and reduces latency and buffering.

* **Scale stateless read services**
  * Video Service remains stateless and horizontally scales behind a load balancer.

Interview Script

> "The key issue with a viral video is that the system becomes extremely read-heavy. A video may be written once but read millions of times.

> For metadata, I would first serve hot entries from a distributed cache like Redis, keyed by videoId, so most requests never reach Cassandra.

> Cassandra already replicates partitions, so I would tune replication and read consistency to distribute database reads across replicas.

> For the actual video bytes, the main solution is the CDN. Popular videos get cached at edge locations, so viewers are served close to their geography instead of repeatedly hitting S3.

> So the main scaling strategy is: **cache metadata, CDN-cache video content, and horizontally scale the stateless read path.**"

What does each box actually run?

DAG step	Real implementation	Typical technology
Detect completed upload	Event emitted after original object arrives	S3 Event Notification, EventBridge or SQS
Start processing	Create one durable workflow per uploaded video	**Temporal**
Inspect video	Determine codec, duration, dimensions, streams	**FFprobe**
Split/process source	Create GOP/segment boundaries or prepare chunks	**FFmpeg**
Transcode video	Encode different resolutions/bitrates/codecs	**FFmpeg** running on CPU/GPU workers
Audio processing	Extract/normalize/transcode audio	FFmpeg
Thumbnail generation	Extract selected frames	FFmpeg
Speech-to-text	Generate captions/transcript	AWS Transcribe or ML service
Package ABR output	Build HLS/DASH/CMAF segments and manifests	FFmpeg, Shaka Packager, or MediaConvert
Store artifacts	Original, intermediate and final outputs	S3
Update status	`PROCESSING → READY`	Metadata DB
Deliver video	Cache/serve segments globally	CloudFront/CDN

Interview script:

"Once the original upload completes in S3, an object-created event starts a durable video-processing workflow. I would use something like Temporal as the DAG orchestrator.

Temporal itself doesn't process the media. It schedules work to different worker pools. We first inspect and logically split the video using FFprobe or FFmpeg. Then the expensive work fans out across many FFmpeg workers, which generate multiple renditions such as 360p, 720p and 1080p. Audio, thumbnails and transcription can run in parallel as well.

These workers can run as containers on Kubernetes or ECS, and the transcoding pools can scale independently based on task backlog and CPU or GPU utilization.

Large intermediate files are always written to S3. The workflow only passes S3 object references, not the actual media bytes.

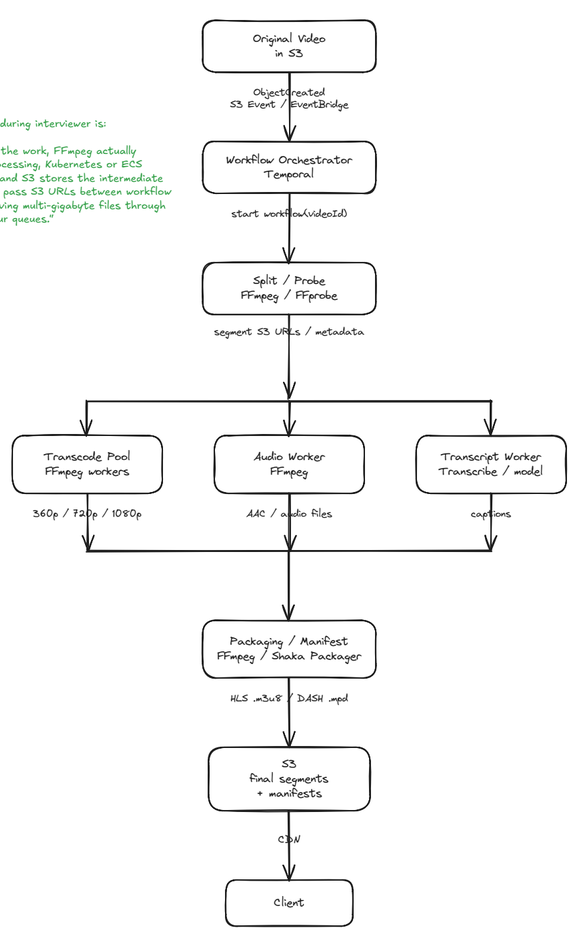
Once all required renditions complete, the workflow fans back in and a packaging step generates HLS or DASH manifests referencing the different segments. We store those manifests and segments in S3, update the video's metadata to READY, and then serve the assets through the CDN.

The key reason for the DAG is that transcoding is CPU-intensive but highly parallelizable. Temporal gives us dependency management, retries and durable execution, while FFmpeg does the actual media work."

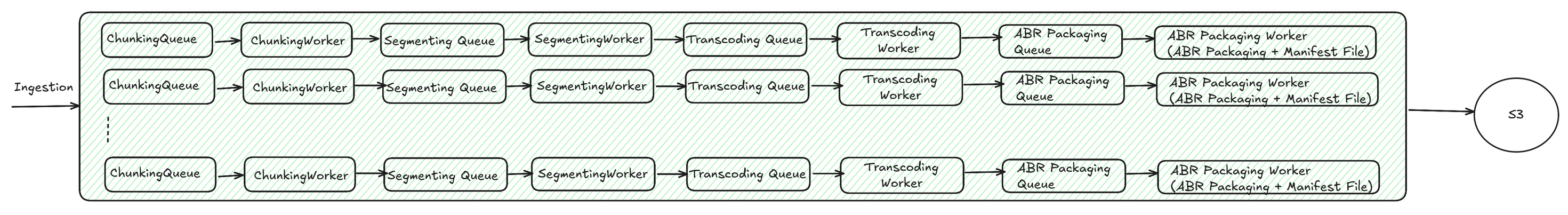
The main thing to tell during interviewer is:

"Temporal coordinates the work, FFmpeg actually performs the media processing, Kubernetes or ECS runs the worker fleet, and S3 stores the intermediate and final artifacts. We pass S3 URLs between workflow stages rather than moving multi-gigabyte files through Temporal or through our queues."

DAG to concrete components like below:


Original Video in S3
ObjectCreated
S3 Event / EventBridge
Workflow Orchestrator Temporal
start workflow(videoId)
Split / Probe FFmpeg / FFprobe
segment S3 URLs / metadata
Transcode Pool FFmpeg workers
Audio Worker FFmpeg
Transcript Worker Transcribe / model
360p / 720p / 1080p
AAC / audio files
captions
Packaging / Manifest FFmpeg / Shaka Packager
HLS .m3u8 / DASH .mpd
S3 final segments + manifests
CDN
Client

client(Ingestion) -> Chunking -> Segmentation -> Transcoding -> ABR Packaging+ManifestFile Creation -> Object Storage -> CDN



Workflow with Multiple Kafka Topics

Ingestion/Chunking Queue:
- Incoming video streams are published to this topic by the media server.

Chunking Stage:
- Consumes from the chunking queue and Divide video streams into smaller chunks.

Segmenting Stage:
- SegmentingWorker: Consumes from the SegmentingQueue.
- Segmenting Output: Process chunks into precise segments.

Transcoding Stage:
- TranscodingWorker: Consumes from the Transcoding Queue.
- Transcoding Output: Convert segments into multiple resolutions and bitrates.

ABR Packaging Stage:
- ABR Packaging Worker: Consumes from the ABR Packaging queue.
- ABR Packaging Output:
  * Package segments for adaptive bitrate streaming.
  * Create Manifest files and store in S3.
  * Store metadata about video segments to DDB (this could be direct REST call, i.e. another service that takes the metadata and store in DDb)

Ticketmaster

# Functional
- view upcoming events
- search upcoming events
- book upcoming events

Below the line (out of scope)
- view their booked events
- Admins flow (adding, editing, etc)
- Dynamic pricing for popular events

# Non-Functional
- Availability for searching & viewing events,
  prioritize consistency for booking events
  (no double booking)
- handle popular events (10 million users, one event)
- low latency search (< 500ms)
- Read >> write (read heavy)

Below the line (out of scope): system should,
- adhere to GDPR
- fault tolerant
- secure transactions for purchases
- regular backups

# Back-of-the-envelop

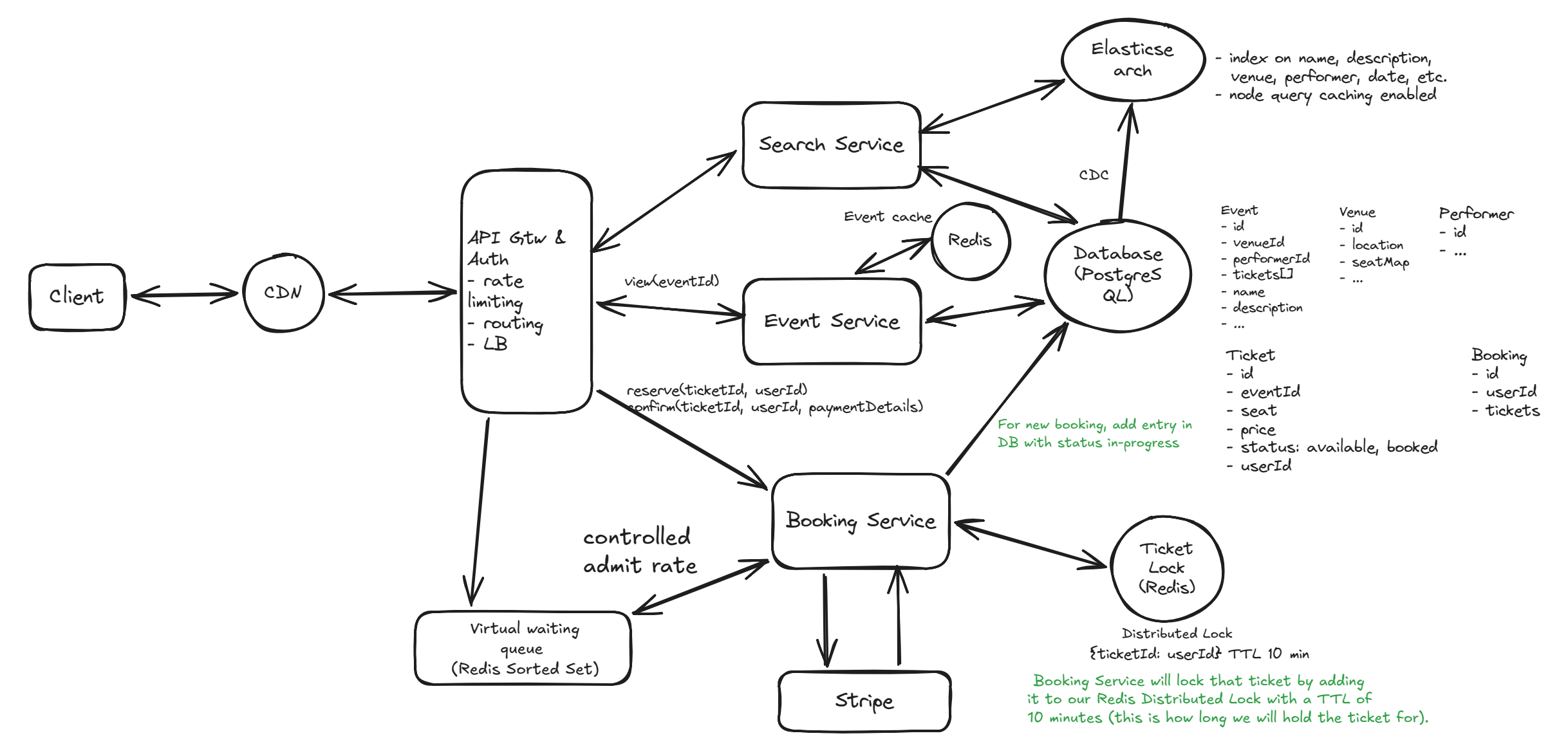
# DB
- Events: stores info about an event, including details like date, description, type, and the performer/team involved.
  (id, venueId, performerId, name, desc)
- Users:
- Performer: performer's name, a brief description, and potentially links to their work or profiles.
- Venues: (id,location,seatMap...)
  physical location where an event is held.
- Tickets:
  (id, eventId, seat, price, status:booked, available)
  Info of individual tickets, event ID,
  seat details, pricing, and status (available/sold).
  When an event is created, tickets are generated from the venue's seat map. Client combines Venue seat-map data + Ticket status to render interactive seat selection.

- Bookings: (userId, ticketId, total_price,
  booking_status (in-progress/confirmed)).

# API
- GET /events/:eventId -> Event,Venue,Performer,Ticket[]
  tickets are to render the seat map on the Client
- GET /events/search?
  keyword={keyword}&start={start_date}&end={end_date}
  &pageSize={page_size}&page={page_number} -> Event[]
- POST /bookings/:eventId -> bookingId

```
{
  "ticketIds": string[],
  "paymentDetails": ...
}
```

GET /search?term={term}&location={location}&type={type}&date={date} -> Partial<Event>[]


Elasticsearch
- index on name, description, venue, performer, date, etc.
- node query caching enabled
Search Service
CDC
Client
CDN
API Gtw & Auth
- rate limiting
- routing
- LB
view(eventId)
Event cache
Redis
Event Service
Database (PostgreSQL)
Event
- id
- venueId
- performerId
- tickets[]
- name
- description
- ...
Venue
- id
- location
- seatMap
- ...
Performer
- id
- ...
Ticket
- id
- eventId
- seat
- price
- status: available, booked
- userId
Booking
- id
- userId
- tickets
reserve(ticketId, userId)
confirm(ticketId, userId, paymentDetails)
For new booking, add entry in DB with status in-progress
controlled admit rate
Booking Service
Ticket Lock (Redis)
Distributed Lock
{ticketId: userId} TTL 10 min
Booking Service will lock that ticket by adding it to our Redis Distributed Lock with a TTL of 10 minutes (this is how long we will hold the ticket for).
Virtual waiting queue (Redis Sorted Set)
Stripe

Postgres to Elastic writes flow

```
Client --> Event Service --> PostgreSQL
              |
              | CDC
              v
        Kafka/Debezium
              |
              v
        Elasticsearch
              ^
              |
        Search Service
              ^
              |
            Client
```

How can you speed up frequently repeated search queries and reduce load on our search infrastructure?
strategy is:
- CDN for global hot queries,
- Redis for application-level hot queries, and
- Elasticsearch's internal caches underneath.

Interview script
Elasticsearch already provides shard-level query and request caches, so repeated filters and search responses can often be served without recomputing the entire query.

For very hot searches, I can also add a Redis cache in front of Elasticsearch using a normalized query as the key and a short TTL.

At even larger scale, non-personalized searches can be cached at the CDN, for example CloudFront, so requests can be served geographically close to the user without reaching our backend.

The main trade-off is freshness. Search results are eventually consistent anyway, so I'd use short TTLs rather than trying to perfectly invalidate every possible query when an event changes.

So the strategy is:
CDN for global hot queries,
Redis for application-level hot queries, and
Elasticsearch's internal caches underneath.

How does the View API scale to tens of millions of concurrent reads?

Interview script:
For tens of millions of concurrent View API requests, my main goal is to make sure the majority of those requests never hit the database.

I would keep the Event Service stateless and horizontally scale many instances behind a load balancer.

In front of the database, I'd use Redis as a distributed cache. Event details, performer information, venue metadata, and seat-map information are very read-heavy but change infrequently, so they are excellent cache candidates.

The read path becomes: client to load balancer, then Event Service, then Redis. On a cache hit we return immediately. On a miss, we read from the database, populate Redis, and return the response.

I'd use longer TTLs for static data such as venue information and shorter TTLs for more dynamic data. When event metadata changes, the application publishes an invalidation event so the corresponding cache entry is removed or refreshed.

For additional protection, I'd use database read replicas, cache pre-warming for major events, and stampede protection so millions of simultaneous cache misses don't overwhelm the database.

So the scaling strategy is essentially: stateless horizontal scaling at the service layer, aggressive distributed caching, and keeping the database off the hot read path.

How will the system ensure a good user experience during high-demand events with millions simultaneously booking tickets?

Interview script

"For a very high-demand event, I would not allow millions of users to reach the seat-selection and booking services simultaneously.

I'd put a virtual waiting room in front of the booking flow. Users enter a Redis-backed queue, for example a sorted set ordered by arrival time.

I'd maintain an SSE connection to push real-time queue position and estimated wait time to the client. SSE is a good fit because this is primarily server-to-client communication.

An Admission Controller gradually releases users based on the capacity of the downstream booking system. When a user is admitted, we issue a short-lived admission token or store their session in Redis.

The Booking Service validates that admission before allowing seat reservation, so users cannot bypass the queue.

This gives us controlled backpressure, protects the booking and inventory systems from overload, and still gives users real-time feedback while they wait."

How can you speed up frequently repeated search queries and reduce load on our search infrastructure?

Interview Script
I would avoid running complex search queries directly against our transactional PostgreSQL database.

PostgreSQL remains the source of truth, but I'd introduce Elasticsearch as a dedicated search index.

Elasticsearch uses inverted indexes, which makes full-text search, filtering, autocomplete, and fuzzy matching significantly faster than scanning relational data.

For example, if a user searches for 'Tayler Swift,' Elasticsearch can use fuzzy matching to still return 'Taylor Swift.'

To keep the search index synchronized, I'd use CDC—typically Debezium reading PostgreSQL changes, publishing them through Kafka, and a search indexer updating Elasticsearch.

This means search becomes eventually consistent by a few seconds, which is normally acceptable for event discovery, while the transactional database remains authoritative.

So the key idea is: PostgreSQL for correctness, Elasticsearch for fast search, and CDC for synchronization.